Input Reference

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Simulation control . . . 6
Basin parameters . . . 6
Spatial objects and connectivity . . . 6
Structural BMPs . . . 6
Climate . . . 6
Soils and land use . . . 7
Initial conditions . . . 7
Channels, aquifers, reservoirs, wetlands . . . 7
Groundwater . . . 7
Operation and management databases . . . 7
Water allocation . . . 7
Solute transport . . . 7
Pollutants . . . 8
Verification procedure . . . 8
Purpose . . . 8
Source . . . 8
Format . . . 8
Categories and fields . . . 8
Example . . . 10
Related files . . . 10
Purpose . . . 10
Source . . . 10
Format . . . 10
Example . . . 11
Related files . . . 11
Purpose . . . 11
Source . . . 11
Format . . . 11
Example . . . 13
Related files . . . 13
Purpose . . . 14
Source . . . 14
Format . . . 14
Example . . . 15
Related files . . . 15
Purpose . . . 15
Source . . . 15
Format . . . 15
Side effects after read . . . 16
Example . . . 16
Related files . . . 17
Purpose . . . 17
Source . . . 17
Format . . . 17

Example . . . 20
Related files . . . 20
Purpose . . . 20
Source . . . 20
Activation . . . 20
Format . . . 21
Example . . . 24
Calibration . . . 24
Related files . . . 24
Purpose . . . 24
Source . . . 24
Activation . . . 25
Format . . . 25
Example . . . 27
Calibration . . . 27
Related files . . . 27
Purpose . . . 27
Source . . . 28
Format . . . 28
Example . . . 29
Related files . . . 30
Purpose . . . 30
Source . . . 30
Format . . . 30
Example . . . 31
Related files . . . 32
Purpose . . . 32
Source . . . 32
hydrology.hyd . . . 32
topography.hyd . . . 34
field.fld . . . 36
shade_factor.shf . . . 38
Related files . . . 40
Purpose . . . 40
Source . . . 40
recall.rec (master database) . . . 40
salt_recall.rec (salt side database) . . . 41
cs_recall.rec (constituent side database) . . . 42
Element and region files . . . 42
Example . . . 42
Related files . . . 43
Purpose . . . 43
weather-sta.cli . . . 43
weather-wgn.cli . . . 44
pcp.cli . . . 46
tmp.cli . . . 46
slr.cli . . . 47
hmd.cli . . . 47
wnd.cli . . . 48
pet.cli . . . 48
atmodep.cli . . . 49
Atmospheric deposition . . . 50
Related files . . . 51
Purpose . . . 51
Source . . . 52
Format . . . 52

Example . . . 53
Related files . . . 53
Purpose . . . 53
Source . . . 53
Format . . . 53
Example . . . 54
Related files . . . 54
Purpose . . . 55
Source . . . 55
Format . . . 55
Example . . . 55
Related files . . . 55
Purpose . . . 55
Source . . . 56
Format . . . 56
Example . . . 58
Related files . . . 58
Purpose . . . 58
Source . . . 58
Format . . . 58
Example . . . 62
Related files . . . 62
Purpose . . . 62
Source . . . 62
Format . . . 62
Example . . . 63
Related files . . . 64
cntable.lum . . . 64
cons_practice.lum . . . 65
ovn_table.lum . . . 67
Related files . . . 69
Purpose . . . 69
Source . . . 69
channel.cha . . . 69
channel-lte.cha . . . 70
hyd-sed-lte.cha . . . 70
initial.cha . . . 72
temperature.cha . . . 72
Related files . . . 74
Purpose . . . 74
Source . . . 74
hydrology.cha . . . 74
sediment.cha . . . 76
nutrients.cha . . . 78
Related files . . . 81
Purpose . . . 81
Source . . . 81
aquifer.aqu . . . 81
initial.aqu . . . 83
Related files . . . 83
Purpose . . . 84
Source . . . 84
reservoir.res . . . 84
initial.res . . . 84
hydrology.res . . . 85
sediment.res . . . 87

nutrients.res . . . 87
weir.res . . . 88
Related files . . . 90
Purpose . . . 90
Source . . . 90
wetland.wet . . . 90
hydrology.wet . . . 91
Related files . . . 93
Purpose . . . 93
Source . . . 93
File format . . . 93
harv.ops . . . 93
graze.ops . . . 95
irr.ops . . . 97
chem_app.ops . . . 99
fire.ops . . . 101
sweep.ops . . . 103
Example . . . 105
Related files . . . 105
Purpose . . . 105
Source . . . 105
File format . . . 105
Example . . . 106
Related files . . . 107
Purpose . . . 107
Source . . . 107
Format . . . 107
Example . . . 110
Related files . . . 110
Purpose . . . 110
Source . . . 110
Format . . . 110
Example . . . 112
manure_db.frt . . . 112
manure_om.frt . . . 113
Related files . . . 114
Purpose . . . 114
Source . . . 114
Format . . . 114
Example . . . 116
Related files . . . 116
What replaces it . . . 116
Migration . . . 116
Related files . . . 117
Purpose . . . 117
Source . . . 117
Format . . . 117
Example . . . 117
Related files . . . 118
Purpose . . . 118
Source . . . 118
Format . . . 118
Example . . . 119
Related files . . . 119
management.sch . . . 119
puddle.ops . . . 121

transplant.plt . . . 123
co2.out and co2_yr.dat . . . 125
object.prt . . . 125
Purpose . . . 126
Source . . . 127
plant.ini . . . 127
soil_plant.ini . . . 131
om_water.ini . . . 133
Related files . . . 135
Purpose . . . 135
Source . . . 135
tiledrain.str . . . 136
septic.str . . . 138
septic.sep . . . 138
filterstrip.str . . . 139
grassedww.str . . . 141
bmpuser.str . . . 143
satbuffer.str . . . 145
urban.urb . . . 145
Related files . . . 147
Purpose . . . 147
Activation . . . 147
Files . . . 147
Related . . . 148
Purpose . . . 149
Activation . . . 149
Files . . . 149
Related . . . 150
Purpose . . . 150
Activation . . . 150
Files . . . 150
Related . . . 150
Purpose . . . 151
Activation . . . 151
Format conventions . . . 151
File overview . . . 151
`codes.gw`: feature flags and run controls . . . 152
`zones.gw`: aquifer and streambed zones . . . 152
`cells.gw`: cell properties and initial state . . . 153
`cellcon.gw`: cell-to-cell connections . . . 153
`outputs.gw`: output control . . . 153
`chancell.gw`: channel-to-cell links . . . 154
`gwflow.con`: channel-cell spatial connections . . . 154
`hrucell.gw`: HRU-to-cell mapping . . . 154
`wetland.gw`: wetland bed thickness . . . 154
`hru_pump.gw`: HRU pump-observation list . . . 154
Other conditional files . . . 155
`cell_sol.gw`: per-cell initial solute concentrations . . . 155
New flat optional files . . . 155
Example . . . 157
Related . . . 158
Purpose . . . 158
Activation . . . 158
Files . . . 158
Related . . . 160
Purpose . . . 160

Activation . . . 160
Files . . . 160
Related . . . 162
Purpose . . . 162
How it fits together . . . 163
Activation . . . 163
Files . . . 164
Related . . . 168

This section documents every SWAT+ input file. Each page lists:

- the file's purpose
- the Fortran reader and the derived type it populates
- the column layout, units, defaults
- a real example from the reference datasets
- known bugs or quirks in an `Important` section at the bottom

Every page is verified against the SWAT+ source at a specific commit, recorded in the page frontmatter as `verified_at_commit`. The source code is the only authoritative reference; example datasets are downstream and can drift.

Simulation control

- `file.cio`. Master index. Names every other input file.
- `time.sim`. Simulation period and time step.
- `print.prt`. Output selection and aggregation.
- `object.cnt`. Basin name, areas, spatial object counts.

Basin parameters

- `codes.bsn`. Method switches.
- `parameters.bsn`. Numeric coefficients.

Spatial objects and connectivity

- `*.con`. Connectivity files (hru.con, chandeg.con, aquifer.con, reservoir.con, …).
- `hru-data.hru`. Per-HRU pointer table.
- `*.hyd, field.fld, shade_factor.shf`. Per-HRU hydrology, topography, field geometry, and shade factor. Referenced by `hru-data.hru`.
- `recall`. Point sources and inlets. Master `recall.rec` plus salt and constituent side databases and the region/element files.

Structural BMPs

- `*.str, septic.sep, urban.urb`. Tile drains, septic systems and effluent, filter strips, grassed waterways, user-defined BMPs, saturated buffers, urban land cover.

Climate

- `*.cli`. Index files: weather-sta.cli, weather-wgn.cli, pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli, pet.cli, atmodep.cli.
- `*.pcp`. Precipitation data files.
- `*.tmp`. Temperature data files.
- `*.tem`. Legacy temperature file extension.
- `snow.sno`. Snow parameter database.

Soils and land use

- soils.sol. Soil profile and layer properties.
- landuse.lum. Land-use management.
- *.lum lookup tables. Curve number, conservation practice, and overland-flow Manning's n tables referenced by landuse.lum.

Initial conditions

- *.ini. General initial conditions: plant.ini (plant communities), soil_plant.ini (soil water, residue, plant biomass), om_water.ini (organic matter in water bodies). Solute-side init files are covered in their module pages under Solute transport and Pollutants.

Channels, aquifers, reservoirs, wetlands

- *.cha. Legacy channel files.
- sd_channel. SWAT+ routing channel inputs (hydrology.cha, sediment.cha, nutrients.cha).
- *.aqu. Aquifer files (aquifer.aqu, initial.aqu).
- *.res. Reservoir files.
- *.wet. Wetland files.

Groundwater

- gwflow.*. 2D cell-based groundwater module inputs (mesh, channel coupling, tiles, ponds, canals, solutes, heat).

Operation and management databases

- management.sch and minor inputs. Fixed-schedule management, puddle.ops, transplant.plt, co2.out, object.prt.
- *.ops. harv, graze, irr, chem_app, fire, sweep.
- *.dtl. Decision tables (lum.dtl, res_rel.dtl, scen_lu.dtl, flo_con.dtl).
- plants.plt. Plant parameter database.
- fertilizer.frt. Fertilizer composition and the manure type and organic-matter databases (manure_db.frt, manure_om.frt).
- manure_allo.mnu. Manure allocation. Sources, demands, and the rule that distributes manure between them.
- tillage.til. Tillage operations.
- carb_coefs.cbn. Century carbon coefficients.
- treatment.trt. Legacy treatment placeholder.

Water allocation

- water_allocation.wro and the *.wal transport files. Water-rights and inter-object water transfer module: master allocation file plus canal, pipe, tower, treatment, use, out-of-basin source and receiver inputs.

Solute transport

- salt module. rtb-salt inputs: per-object initial conditions and applied loads for the eight salt ions.
- cs module. rtb-cs inputs: master constituents.cs plus per-object initial conditions and applied loads for selenium, boron, and other constituents.

Pollutants

- pesticide module. pesticide.pes parameter database, per-HRU and per-channel initial conditions, and the parent-daughter metabolite chain.
- pathogen module. pathogens.pth parameter database and per-HRU and per-channel initial conditions.
- metals module. Per-HRU initial heavy metal concentrations.

Verification procedure

The procedure for verifying a new or updated page is in SOURCE-OF-TRUTH.md in the docs working folder.

Purpose

file.cio is the master index. SWAT+ opens this file first. Each line tells the model which file to read for one input category. Every other input file in the project is reached through file.cio.

Source

- Reader: src/readcio_read.f90
- File-name fields and defaults: src/input_file_module.f90

Format

- Line 1: title line. Skipped by the reader.
- Lines 2 to 31: one line per input category. Each line starts with a category label (a free-form token that the reader reads but discards), followed by a fixed number of file-name fields whose order matches the corresponding in_* type in input_file_module.f90.
- Line 32: a single line for the optional output path.
- A field set to the literal token null disables the corresponding file.

The category labels written by tools like QSWAT+ and SWAT+ Editor are conventional. The reader does not validate them; only field order and count matter.

Categories and fields

The table lists every category in the order the reader expects them, the number of file-name fields, and the default file names defined in input_file_module.f90.

#	Category label	Fields (defaults from input_file_module.f90)
1	simulation	time.sim, print.prt, object.prt, object.cnt, constituents.cs
2	basin	codes.bsn, parameters.bsn
3	climate	weather-sta.cli, weather-wgn.cli, pet.cli, pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli, atmodep.cli
4	connect	hru.con, hru-lte.con, rout_unit.con, gwflow.con, aquifer.con, aquifer2d.con, channel.con, reservoir.con, recall.con, exco.con, delratio.con, outlet.con, chandeg.con
5	channel	initial.cha, channel.cha, hydrology.cha, sediment.cha, nutrients.cha, channel-lte.cha, hyd-sed-lte.cha, temperature.cha

#	Category label	Fields (defaults from `input_file_module.f90`)
6	reservoir	`initial.res`, `reservoir.res`, `hydrology.res`, `sediment.res`, `nutrients.res`, `weir.res`, `wetland.wet`, `hydrology.wet`
7	routing_unit	`rout_unit.def`, `rout_unit.ele`, `rout_unit.rtu`, `rout_unit.dr`
8	hru	`hru-data.hru`, `hru-lte.hru`
9	exco	`exco.exc`, `exco_om.exc`, `exco_pest.exc`, `exco_path.exc`, `exco_hmet.exc`, `exco_salt.exc`
10	recall	`recall.rec`
11	dr	`delratio.del`, `dr_om.del`, `dr_pest.del`, `dr_path.del`, `dr_hmet.del`, `dr_salt.del`
12	aquifer	`initial.aqu`, `aquifer.aqu`
13	herd	`animal.hrd`, `herd.hrd`, `ranch.hrd`
14	water_rights	`water_allocation.wro`, `element.wro`, `water_rights.wro`
15	link	`chan-surf.lin`, `aqu_cha.lin`
16	hydrology	`hydrology.hyd`, `topography.hyd`, `field.fld`
17	structural	`tiledrain.str`, `septic.str`, `filterstrip.str`, `grassedww.str`, `bmpuser.str`
18	hru_parm_db	`plants.plt`, `fertilizer.frt`, `tillage.til`, `pesticide.pes`, `pathogens.pth`, `metals.mtl`, `salt.slt`, `urban.urb`, `septic.sep`, `snow.sno`
19	ops	`harv.ops`, `graze.ops`, `irr.ops`, `chem_app.ops`, `fire.ops`, `sweep.ops`
20	lum	`landuse.lum`, `management.sch`, `cntable.lum`, `cons_practice.lum`, `ovn_table.lum`
21	chg	`cal_parms.cal`, `calibration.cal`, `codes.sft`, `wb_parms.sft`, `water_balance.sft`, `ch_sed_budget.sft`, `ch_sed_parms.sft`, `plant_parms.sft`, `plant_gro.sft`
22	init	`plant.ini`, `soil_plant.ini`, `om_water.ini`, `pest_hru.ini`, `pest_water.ini`, `path_hru.ini`, `path_water.ini`, `hmet_hru.ini`, `hmet_water.ini`, `salt_hru.ini`, `salt_water.ini`
23	soils	`soils.sol`, `nutrients.sol`, `soils_lte.sol`
24	decision_table	`lum.dtl`, `res_rel.dtl`, `scen_lu.dtl`, `flo_con.dtl`
25	regions	`ls_unit.ele`, `ls_unit.def`, `ls_reg.ele`, `ls_reg.def`, `ls_cal.reg`, `ch_catunit.ele`, `ch_catunit.def`, `ch_reg.def`, `aqu_catunit.ele`, `aqu_catunit.def`, `aqu_reg.def`, `res_catunit.ele`, `res_catunit.def`, `res_reg.def`, `rec_catunit.ele`, `rec_catunit.def`, `rec_reg.def`
26	pcp_path	one path string
27	tmp_path	one path string
28	slr_path	one path string
29	hmd_path	one path string
30	wnd_path	one path string
31	output_path	one path string (read as the whole line after the label)

Categories 1 to 30 are read with list-directed I/O. Category 31 (`output_path`) is read with format `(A)` and the value is taken from the trailing portion of the line after the first space.

Example

Top of `refdata/Ames_sub1/file.cio`:

```
file.cio: AMES
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli           tmp.cli
connect           hru.con           null              null              null              null
channel           null       null       null        null         null       null        null        null
reservoir         null       null       null        null         null       null        null        null
routing_unit      null       null       null        null
hru               hru-data.hru      null
exco              null              null              null              null              null
recall            null
dr                null              null              null              null              null
aquifer           nbull             null
```

Notes on the example:

- Line 2 (`simulation`): only `time.sim`, `print.prt`, and `object.cnt` are in use. Slots 3 and 5 (`object.prt`, `constituents.cs`) are set to `null`.
- Line 4 (`climate`): only stations, weather generator, precipitation, and temperature are active. PET, solar, humidity, wind, and atmospheric deposition are disabled.
- Line 14 (`aquifer`): the dataset writes `nbull` for the initial-aquifer file. This is the file name in this project, not a default and not a typo of `null`. The model opens whatever file name appears here.

Related files

Every input file referenced in `file.cio`. Start from the Input reference index.

Purpose

`time.sim` sets the simulation start date, end date, and the sub-daily time step. All time-driven processes (climate reads, routing, output aggregation) use these values.

Source

- Reader: `src/time_read.f90`
- Type definition: `src/time_module.f90`, `type time_current`

Format

- Line 1: title line. Skipped.
- Line 2: header line. Skipped.
- Line 3: five integer values, list-directed.

#	Name	Type	Description
1	day_start	int	Julian day of year on which the simulation starts (1 to 366)
2	yrc_start	int	Calendar year on which the simulation starts
3	day_end	int	Julian day of year on which the simulation ends
4	yrc_end	int	Calendar year on which the simulation ends
5	step	int	Sub-daily time step code (see below)

Sub-daily time step values

The type definition in `time_module.f90` documents these step codes:

Value	Meaning
0	daily
1	12-hour increment
24	hourly
96	15 minutes
1440	per minute

Defaults and derived values

The reader applies these adjustments after parsing:

- `step <= 0` is forced to `1`.
- `day_start <= 0` is forced to `1`.
- `time%nbyr` is set to `yrc_end - yrc_start + 1`.
- `time%mo`, `time%day_mo`, and `time%mo_start` are computed from `day_start` via the `xmon` helper.
- `time%yrc` is initialised to `yrc_start`.

Example

`refdata/Ames_sub1/time.sim`:

```
time.sim:
day_start  yrc_start   day_end   yrc_end      step
        0       1975         0      2020         0
```

This runs from 1 Jan 1975 through 31 Dec 2020 (the zero values for `day_start` and `day_end` are interpreted as the first and last day of the year). The `step` value of `0` is forced to `1` by the reader.

Related files

- `print.prt`. Controls which outputs are written within this simulation window.

Purpose

`print.prt` controls every output the model writes. It sets the print window in calendar time, the daily print interval, the average-annual print intervals, three database-output toggles, four "other output" toggles, and one row per output object with four print-frequency flags (daily, monthly, yearly, average annual).

If `print.prt` is missing or set to `null` in `file.cio`, the model still runs but writes no time-stepped output.

Source

- Reader: `src/basin_print_codes_read.f90`
- Type definitions: `src/basin_module.f90`. See `type basin_print_codes` (`pco`) and `type print_interval`.

Format

The file has fixed top-of-file blocks followed by one row per output object. There are two object-row modes, selected by the `use_obj_lbls` flag in the third block. The reader skips a header line before each block.

Block 1: window

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: six integers.

#	Name	Description	Default if unset
1	nyskip	number of warm-up years to skip in average-annual summaries	0
2	day_start	julian day to start daily printing	forced to 1 if 0
3	yrc_start	calendar year to start daily printing	`time%yrc` from `time.sim`
4	day_end	julian day to end daily printing	forced to 366 if 0
5	yrc_end	calendar year to end daily printing	`time%yrc + time%nbyr`
6	interval	days between daily prints	forced to 1 if `<= 0`

Block 2: average-annual print intervals

- Line 4: header (skipped).
- Line 5: `aa_numint` followed by `aa_numint` year values. If `aa_numint = 0`, only the value is read.

`aa_numint` is the number of end-years to use for average-annual outputs. When it is greater than zero, average-annual statistics are produced once for each interval defined by the listed end years.

Block 3: database output toggles

- Line 6: header (skipped).
- Line 7: three single-character flags.

Name	Values	Meaning
csvout	`n`, `y`	also write `.csv` versions of outputs
use_obj_lbls	`n`, `y`	object-row mode (see Object rows)
cdfout	`n`, `y`	also write netCDF (`.nc`) versions of outputs

Block 4: other output toggles

- Line 8: header (skipped).
- Line 9: four single-character flags.

Name	Values	Meaning
crop_yld	`a`, `y`, `b`, `n`	crop yields: `a` average annual, `y` yearly, `b` both, `n` no print
mgtout	`n`, `y`	write `mgt_out.txt` (management operations log)
hydcon	`n`, `y`	write `hydcon.out` (hydrograph connections)
fdcout	`n`, `y`	flow duration curve. Read but the type comment notes "NOT ACTIVE"

Object rows

- Line 10: header (`objects daily monthly yearly avann`), skipped.
- Lines 11 and following: one row per output object. Each row has five fields: a name and four flags (`d`, `m`, `y`, `a`) with values `n` or `y`.

The reader interprets the rows in one of two modes:

- `use_obj_lbls = n` (positional): the reader expects rows in a fixed order. The name column is read but discarded. Missing or extra rows cause silent skips of later objects when the loop hits end-of-file.
- `use_obj_lbls = y` (labelled): rows can be in any order. The reader matches the first column against a list of recognised names. Repeated rows for the same object are flagged via `print_prt_error`. Unknown names cause the model to print an error and `error stop`.

Recognised object names (labelled mode)

Group	Names
Basin	basin_wb, basin_nb, basin_ls, basin_pw, basin_aqu, basin_res, basin_cha, basin_sd_cha, basin_psc
Region	region_wb, region_nb, region_ls, region_pw, region_aqu, region_res, region_sd_cha, region_psc, water_allo
LSU	lsunit_wb, lsunit_nb, lsunit_ls, lsunit_pw
HRU	hru_wb, hru_nb, hru_ls, hru_pw, hru_cb, hru_cb_vars
HRU-lte	hru-lte_wb, hru-lte_nb, hru-lte_ls, hru-lte_pw
Other	channel, channel_sd, aquifer, reservoir, recall, hyd, ru, pest
Salt	basin_salt, hru_salt, ru_salt, aqu_salt, channel_salt, res_salt, wetland_salt
Constituents	basin_cs, hru_cs, ru_cs, aqu_cs, channel_cs, res_cs, wetland_cs
gwflow	gwflow_wb, gwflow_flux, gwflow_heat, gwflow_solute, gwflow_obs, gwflow_pump

Each row's four flags map to the `print_interval` fields `d`, `m`, `y`, `a` (daily, monthly, yearly, average annual). Value `y` enables that aggregation. Value `n` disables it.

The HRU carbon rows `hru_cb` and `hru_cb_vars` drive the legacy CSU carbon output files (alongside the standardised per-family carbon outputs). SWAT+ Editor does not write these rows; the CSU team adds them by hand. The `hru_cb` flag letter selects detail (`l` = full per-layer set with diagnostics, `y` = light profile set). See the carbon outputs reference.

Example

`refdata/Ames_sub1/print.prt`:

```
print.prt:
nyskip      day_start  yrc_start  day_end   yrc_end   interval
0           0               1975         0      2020          1
aa_int_cnt
0
csvout        use_obj_lbls  cdfout
n             n             n
crop_yld      mgtout        hydcon        fdcout
b             y             n             n
objects                  daily       monthly        yearly         avann
basin_wb                     n             n             n             y
basin_nb                     n             n             n             y
basin_ls                     n             n             n             y
basin_pw                     n             n             n             y
basin_aqu                    n             n             n             y
...
```

Window: print over 1975 to 2020, daily interval 1, no warm-up. `aa_int_cnt = 0` means a single average-annual interval over the whole period. CSV and netCDF off. Crop yields in both average-annual and yearly form. Management log on. `use_obj_lbls = n` so the rows are in fixed order.

Related files

- `time.sim`. Defines the simulation period that bounds these print windows.
- Output reference. Documents each output file produced when the corresponding row is enabled.

Purpose

`object.cnt` declares the basin name, the landscape and total areas, and how many of each spatial object type (HRU, channel, aquifer, reservoir, and so on) the project contains. The model uses these counts to allocate the `ob`, `obcs`, `obcs_alloc`, and `obom` arrays before any object data is read.

If `object.cnt` is missing or set to `null` in `file.cio`, the model prints `Cannot find object.cnt input file` and stops.

Source

- Reader: src/basin_read_objs.f90
- Type definitions:
  - src/basin_module.f90, `type basin_inputs` (`bsn`)
  - src/hydrograph_module.f90, `type spatial_objects` (`sp_ob`)

Format

- Line 1: title line. Skipped.
- Line 2: header line. Skipped.
- Line 3: 21 values read into `bsn` and `sp_ob`, list-directed.

#	Field	Maps to	Type	Description
1	name	`bsn%name`	char(25)	basin name (free-form)
2	ls_area	`bsn%area_ls_ha`	real	landscape area, hectares
3	tot_area	`bsn%area_tot_ha`	real	total area, hectares
4	objs	`sp_ob%objs`	int	total object count (sum of all object types)
5	hru	`sp_ob%hru`	int	number of HRUs
6	hru_lte	`sp_ob%hru_lte`	int	number of HRU-lte objects
7	ru	`sp_ob%ru`	int	number of routing units
8	gwflow	`sp_ob%gwflow`	int	number of gwflow cells
9	aqu	`sp_ob%aqu`	int	number of aquifer objects
10	chan	`sp_ob%chan`	int	number of legacy channels
11	res	`sp_ob%res`	int	number of reservoirs
12	recall	`sp_ob%recall`	int	number of recall objects
13	exco	`sp_ob%exco`	int	number of export-coefficient objects
14	dr	`sp_ob%dr`	int	number of delivery-ratio objects
15	canal	`sp_ob%canal`	int	number of canals
16	pump	`sp_ob%pump`	int	number of pumps
17	outlet	`sp_ob%outlet`	int	number of outlets
18	chandeg	`sp_ob%chandeg`	int	number of SWAT-deg channels (SWAT+ routing)
19	aqu2d	`sp_ob%aqu2d`	int	2D aquifer count (currently not used per the type comment)
20	herd	`sp_ob%herd`	int	herd count (currently not used per the type comment)
21	wro	`sp_ob%wro`	int	water-rights object count (currently not used per the type comment)

gwflow side effect

After reading, if `bsn_cc%gwflow == 1` (set via `codes.bsn`), the reader looks for `gwflow.chancells`. If that file exists and contains river cells, the reader:

- sets `sp_ob%gwflow` to the number of river cells,
- recomputes `sp_ob%objs` to `sp_ob%objs + nriv - sp_ob%aqu`,
- sets `in_con%gwflow_con` to `gwflow.con`,
- sets `sp_ob%aqu` to 0 and `in_con%aqu_con` to `null`.

If `bsn_cc%gwflow == 1` but `gwflow.chancells` is missing or empty, gwflow is silently turned off.

Example

`refdata/Ames_sub1/object.cnt`:

```
object.cnt:
name                    ls_area      tot_area       obj       hru      lhru       rtu       mfl       aqu
demo                      1.           1            12         12         0         0         0         0
```

Notes on the example:

- `name = demo`, `ls_area = 1` ha, `tot_area = 1` ha. The fields are floats; the trailing `.` makes that explicit in this dataset.
- `objs = 12` and `hru = 12`. Every spatial object in this project is an HRU.
- The header uses shortened labels (`obj`, `lhru`, `rtu`, `mfl`, `dlr`, `can`, `pmp`, `out`, `lcha`, `hrd`). These are tool conventions; the reader does not parse the header.

Related files

- `codes.bsn`. Sets `bsn_cc%gwflow`, which triggers the gwflow side effect above.
- `file.cio`. The `simulation` category points to `object.cnt`.
- Object-specific connectivity files (`hru.con`, `chandeg.con`, `aquifer.con`, `reservoir.con`, …). These must agree with the counts in `object.cnt`.

Purpose

`codes.bsn` holds the method-selection switches that apply to the whole basin: the PET method, the routing method, the carbon model, the soil P model, and so on. It is read into `bsn_cc` (`type basin_control_codes`) early in the run and then consulted by every process module.

Source

- Reader: `src/basin_read_cc.f90`
- Type definition: `src/basin_module.f90`, `type basin_control_codes`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: 26 fields, list-directed, read into `bsn_cc` in declaration order.

#	Field	Type	Default	Description
1	petfile	char(16)	`pet.cli`	Potential ET filename. Used when `pet = 3` (read PET from file).
2	wwqfile	char(16)	`""`	Watershed stream water-quality filename.
3	pet	int	0	PET method. `0` Priestley-Taylor, `1` Penman-Monteith, `2` Hargreaves. (Method `3` triggers reading `pet.cli`.)
4	nam1	int	0	Not used.
5	crk	int	0	Crack flow. `1` to compute flow in cracks.
6	swift_out	int	0	SWIFT output. `0` do not write, `1` write `swift_hru.inp`.
7	sed_det	int	0	Peak rate method. `0` NRCS dimensionless hydrograph with PRF, `1` half-hour rainfall intensity method.

#	Field	Type	Default	Description
8	rte	int	0	Water routing method. `0` variable storage, `1` Muskingum.
9	deg	int	0	Not used.
10	wq	int	0	Not used.
11	nostress	int	0	Plant stress. `0` all stresses applied, `1` turn off all plant stress, `2` turn off nutrient stress only.
12	cn	int	0	Not used.
13	cfac	int	0	Not used.
14	cswat	int	0	Carbon model. `0` static (legacy mineralization), `1` C-FARM (reserved, not implemented), `2` dynamic CENTURY/SWAT-C model.
15	lapse	int	0	Lapse-rate adjustment for precip and temperature. `0` no, `1` yes.
16	uhyd	int	1	Unit hydrograph. `0` triangular, `1` gamma function.
17	sed_ch	int	0	Not used.
18	tdrn	int	0	Tile drainage equation. `0` drawdown-days, `1` DRAINMOD.
19	wtdn	int	0	Shallow water-table depth. `0` original routine (fill to upper limit), `1` DRAINMOD water-table depth.
20	sol_p_model	int	0	Soil P model. `0` original SWAT P model, `1` Vadas and White (2010).
21	gampt	int	0	Infiltration method. `0` curve number, `1` Green-Ampt.
22	atmo	char(1)	`a`	Not used.
23	smax	int	0	Not used.
24	qual2e	int	0	Instream nutrient routing. `0` QUAL2E, `1` QUAL2E with simplified transformations.
25	gwflow	int	0	gwflow module. `0` inactive, `1` active. Triggers extra logic in `object.cnt`.
26	idc_till	int	3	Tillage method when `cswat = 2`. `1` DSSAT, `2` EPIC, `3` Kemanian, `4` DNDC.

The "Not used" fields are read from the file but the model does not consult them. Tools commonly write `0`.

Side effects after read

If `pet = 3`, the reader opens `pet.cli` and reads a title and header line to position the file pointer. The actual PET values are read later by the climate routines.

Example

`refdata/Ames_sub1/codes.bsn`:

`codes.bsn`:

pet_file	wq_file	pet	event	crack	swift_out	sed_det	rte_cha	deg_cha
null	null	2	0	0	0	0	0	0

The header column names written by SWAT+ Editor (`event`, `rte_cha`, `i_fpwet`, …) differ from the field names in the source (`crk`, `rte`, `qual2e`, …). Order is what the reader cares about. Mapping the example to the table:

- `pet = 2`: Hargreaves.
- `cswat = 2`: dynamic CENTURY/SWAT-C carbon model. (Header label `carbon`.) The example file above still carries `carbon = 1`, which is now the reserved C-FARM slot and runs no carbon model; datasets that want the dynamic model must set `carbon = 2`.
- `uhyd = 1`: gamma-function unit hydrograph.
- `idc_till = 3`: Kemanian tillage method.

The `null` strings in columns 1 and 2 disable the PET and water-quality file inputs.

Related files

- `parameters.bsn`. Numeric basin parameters consumed by the methods selected here.
- `object.cnt`. Reads gwflow state when `gwflow = 1`.
- `pet.cli`. Used when `pet = 3`.

Purpose

`parameters.bsn` holds the basin-wide numeric coefficients (initial soil water, surface runoff lag, nutrient cycling rates, channel routing factors, lapse rates, CO2, and so on). Values are read into `bsn_prm` (`type basin_parms`) and consulted by hydrology, nutrient, erosion, and routing routines.

Source

- Reader: `src/basin_read_prm.f90`
- Type definition: `src/basin_module.f90`, `type basin_parms`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: 44 values, list-directed, read into `bsn_prm` in declaration order. The last two values are integers; the rest are reals.

#	Field	Type	Default	Units	Description
1	evlai	real	3.0	none	leaf area index above which no soil evaporation occurs
2	ffcb	real	0.0	fraction	initial soil water content as a fraction of field capacity
3	surlag	real	4.0	days	surface runoff lag time
4	adj_pkr	real	1.0	none	peak rate adjustment factor in the subbasin
5	prf	real	484.0	none	peak rate factor for the peak rate equation
6	spcon	real	0.0	none	not used
7	spexp	real	0.0	none	not used
8	cmn	real	0.003	1/day	rate factor for mineralization of active organic N
9	n_updis	real	20.0	none	nitrogen uptake distribution parameter
10	p_updis	real	20.0	none	phosphorus uptake distribution parameter
11	nperco	real	0.10	0..1	nitrate percolation coefficient
12	pperco	real	10.0	0..1	phosphorus percolation coefficient
13	phoskd	real	175.0	none	phosphorus soil partitioning coefficient
14	psp	real	0.40	none	phosphorus availability index
15	rsdco	real	0.05	none	residue decomposition coefficient
16	percop	real	0.5	0..1	pesticide percolation coefficient
17	msk_co1	real	0.75	none	Muskingum storage time constant for bankfull depth
18	msk_co2	real	0.25	none	Muskingum storage time constant for low flow
19	msk_x	real	0.20	none	Muskingum weighting factor for inflow vs outflow
20	nperco_lchtile	real	0.5	none	N concentration coefficient for tile flow and bottom-layer leaching
21	evrch	real	0.60	none	reach evaporation adjustment factor
22	scoef	real	1.0	0..1	channel storage coefficient
23	cdn	real	1.40	none	denitrification exponential rate coefficient
24	sdnco	real	1.30	none	denitrification threshold (fraction of field capacity)
25	bact_swf	real	0.15	fraction	fraction of manure containing active colony-forming units
26	tb_adj	real	0.0	none	adjustment factor for sub-daily unit hydrograph base time
27	cn_froz	real	0.000862	none	parameter for frozen-soil adjustment on infiltration/runoff
28	dorm_hr	real	-1.0	hours	dormancy time threshold
29	plaps	real	0.0	mm/km	precipitation lapse rate per km of elevation difference
30	tlaps	real	6.5	deg C/km	temperature lapse rate per km of elevation difference
31	nfixmx	real	20.0	kg/ha/day	maximum daily nitrogen fixation
32	decr_min	real	0.01	1/day	minimum daily residue decay
33	rsd_covco	real	0.75	none	residue cover factor for computing cover fraction
34	urb_init_abst	real	1.0	none	maximum initial abstraction for urban areas (Green-Ampt)

#	Field	Type	Default	Units	Description
35	petco_pmpt	real	100.0	percent	PET adjustment for Penman-Monteith and Priestley-Taylor
36	uhalpha	real	1.0	none	alpha coefficient for the gamma-function unit hydrograph
37	eros_spl	real	0.0	none	coefficient of splash erosion (typical range 0.9 to 3.1)
38	rill_mult	real	0.0	none	rill erosion coefficient
39	eros_expo	real	0.0	none	exponential coefficient for overland flow
40	c_factor	real	0.0	none	cover and management factor scaling for overland-flow erosion
41	ch_d50	real	0.0	mm	median particle diameter of the main channel
42	co2	real	400.0	ppm	CO2 concentration at start of simulation
43	day_lag_mx	int	0	days	maximum days to lag hydrographs for HRU, RU, and channels on non-draining soils
44	igen	int	5	none	random generator code. `0` use defaults, `1` generate new numbers every run

Example

refdata/Ames_sub1/parameters.bsn:

```
parameters.bsn: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
  lai_noevap       sw_init      surq_lag      adj_pkrt  adj_pkrt_sed       lin_sed       exp_sed      orgn
     3.00000       0.50000       4.00000       1.00000       1.00000       0.00000       1.00000       0.0
```

Notes:

- Tool header names (`lai_noevap`, `sw_init`, `surq_lag`, `adj_pkrt`, `orgn_min`, `n_uptake`, `evap_adj`, `denit_exp`, `denit_frac`, `man_bact`, `adj_uhyd`, `n_fix_max`, `rsd_decay`, `rsd_cover`, `uhyd_alpha`, `splash`, `rill`, `surq_exp`, `cov_mgt`, `cha_d50`, `day_lag_max`) do not match the field names in the source (`evlai`, `ffcb`, `surlag`, `adj_pkr`, `cmn`, `n_updis`, `evrch`, `cdn`, `sdnco`, `bact_swf`, `tb_adj`, `nfixmx`, `decr_min`, `rsd_covco`, `uhalpha`, `eros_spl`, `rill_mult`, `eros_expo`, `c_factor`, `ch_d50`, `day_lag_mx`). The reader uses position only.
- Position 8 in this dataset is `orgn_min = 0.0`. In the source this is `cmn`, with default `0.003`. The example overrides the default to zero, which disables active organic-N mineralization.
- Position 7 is `exp_sed = 1.0`. This is `spexp` in the source, marked "not used".
- Positions 28 to 30 (`dorm_hr`, `plaps`, `tlaps`) are all zero in this dataset. The default for `tlaps` is `6.5` deg C/km; this dataset disables temperature lapse.

Related files

- codes.bsn. Method switches that select which of these parameters are active.

Purpose

`carbon.bsn` holds the basin-wide scalar coefficients for the SWAT+ carbon module. The reader fills `org_frac`, `cb_wtr_coef`, `man_coef`, and `org_con` (all defined in carbon_module.f90), the basin-scope tillage variables in tillage_data_module.f90 (`bio_consf`, `till_consf`, `bmix_a/b/c`, `tillmix_a/b/c`, `till_eff_days`), the `photo_degrade_factor` in plant_data_module.f90, and the five residue-decomposition tunables (`n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref`) promoted from hard-coded constants into module-scope reals in `carbon_module.f90`.

`carbon.bsn` replaces the legacy `carb_coefs.cbn` (keyword-style) and works alongside the per-layer file carbon_lyr.bsn, which holds the rate constants and CO2 fractions that differ between the surface and subsurface layer groups.

Source

- Reader: src/carbon_bsn_read.f90
- Filename slot: src/input_file_module.f90, `type input_basin`, field `carbon_bsn = "carbon.bsn"`.
- Target types: `organic_fractions`, `carbon_water_coef`, `manure_coef`, `organic_controls` in src/carbon_module.f90; module-scope reals `n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref` in the same module; `bio_consf`, `till_consf`, `till_eff_days`, `bmix_a/b/c`, `tillmix_a/b/c` in src/tillage_data_module.f90; `photo_degrade_factor` in src/plant_data_module.f90.

Activation

The reader runs only when `bsn_cc%cswat == 2` (set in codes.bsn). When `cswat == 2` the file is **required**: if `carbon.bsn` (or its companion `carbon_lyr.bsn`) is missing or its data row cannot be parsed, the reader writes an error message to stdout and unit 9001 and calls `error stop`. There is no silent fallback to compiled defaults. `carbon_bsn_read.f90` returns early for any other `cswat` value, so `cswat = 1` (the reserved C-FARM slot) does not read this file.

The dynamic carbon model activates on `cswat = 2`, matching legacy SWAT numbering (1 = C-FARM, 2 = CENTURY). It was previously `cswat = 1`; existing `codes.bsn` files left at `carbon = 1` now run with the carbon model off and must be changed to `carbon = 2`.

`carbon.bsn` is listed in the BASIN row of `file.cio` alongside `codes.bsn` and `parameters.bsn` (three columns). `carbon_lyr.bsn` rides at a fixed name beside it and does not appear in `file.cio`.

> **Note**
>
> The legacy `cbn_diag` first column is retired. Output gating now lives entirely in `print.prt` through the `hru_cb_*` and `lsu_cb_*` rows (see the carbon outputs reference).

Format

- Line 1: title (skipped).
- Line 2: column header (skipped).
- Line 3: 28 values, list-directed, read into the target fields in the order below. (The 28th value, `mathers_method`, was added when carbonDev merged to main; 27-value files from before the merge fail to parse and must add the trailing column.)

Values are space-delimited. Position is significant; the column header is decorative.

#	Field	Type	Target	Units	Description
1	init_seq	real	org_frac%frac_seq	fraction	initial fraction of total soil carbon held in the sequestered pool
2	init_microb	real	org_frac%frac_hum_microb	fraction	initial microbial-humus fraction of non-sequestered carbon
3	init_slow	real	org_frac%frac_hum_slow	fraction	initial slow-humus fraction of non-sequestered carbon
4	init_passive	real	org_frac%frac_hum_passive	fraction	initial passive-humus fraction of non-sequestered carbon
5	koc_c	real	cb_wtr_coef%prmt_21	none	partition coefficient for carbon loss in water and sediment (formerly prmt_21)
6	solc_ratio	real	cb_wtr_coef%prmt_44	0..1	ratio of soluble C in surface runoff to soluble C in percolate (formerly prmt_44)
7	till_eff_days	real	till_eff_days	days	number of days a tillage event remains effective
8	manure_c_frac	real	man_coef%rtof	none	manure organic N and P partition between fresh and stable pools (formerly rtof)
9	bio_consol	real	bio_consf	none	biological consolidation factor for soil moisture mixing
10	till_consol	real	till_consf	none	tillage consolidation factor for soil moisture mixing
11	tmpf_eqn	int	org_con%tmpf	none	temperature factor selector (1, 2, or 3) used in cbn_zhang2.f90
12	watf_eqn	int	org_con%watf	none	water factor selector (1 or 2) used in cbn_zhang2.f90
13	t_cbn_min	real	org_con%tn	deg C	minimum temperature bound for tmpf2 (lower limit of biological activity)
14	t_cbn_opt	real	org_con%top	deg C	optimum temperature for tmpf2
15	t_cbn_max	real	org_con%tx	deg C	maximum temperature bound for tmpf2 (upper limit of biological activity)
16	bmix_a	real	bmix_a	none	biological mixing curve coefficient A (mgt_tillfactor.f90 tillf option 4)
17	bmix_b	real	bmix_b	none	biological mixing curve coefficient B
18	bmix_c	real	bmix_c	none	biological mixing curve coefficient C
19	tillmix_a	real	tillmix_a	none	tillage mixing curve coefficient A
20	tillmix_b	real	tillmix_b	none	tillage mixing curve coefficient B
21	tillmix_c	real	tillmix_c	none	tillage mixing curve coefficient C
22	sfc_rsd_photodeg	real	photo_degrade_factor	1/day	surface residue photodegradation rate factor
23	n_act_frac	real	n_act_frac	fraction	fraction of organic N held in the active humus pool (drives active-to-stable flow in nut_nminrl.f90)
24	cnr_cap	real	cnr_cap	none	upper cap on residue C:N ratio before computing the decomposition factor
25	cnr_ref	real	cnr_ref	none	reference C:N ratio where the C:N decomposition factor equals 1
26	cpr_cap	real	cpr_cap	none	upper cap on residue C:P ratio before computing the decomposition factor
27	cpr_ref	real	cpr_ref	none	reference C:P ratio where the C:P decomposition factor equals 1

#	Field	Type	Target	Units	Description
28	`mathers_method`	int	`org_frac%mathers_method`	flag	slow-humus pool initialization: `0` original humus-slow init (default), `1` Mathers method (clay+silt based, from "Updating carbon pool initialization with DSSAT-CENTURY"). Used in `soil_nutcarb_init`.

The reader does not interpret column 1-4 as a probability simplex; the four `init_*` values are used as supplied. The four target fields hold the fractional partitioning of soil carbon into sequestered, microbial-humus, slow-humus, and passive-humus pools at simulation start.

The `tmpf_eqn` (col 11) and `watf_eqn` (col 12) integers select between the temperature and water response equations encoded in `cbn_zhang2.f90`. They are integers in the file but the reader does not enforce that; the consuming code treats them as `int(real)`.

Example

`refdata/Lafayette/carbon.bsn`:

```
carbon.bsn: basin-wide scalar carbon parameters (one record)
   init_seq   init_microb   init_slow   init_passive   koc_c   solc_ratio   till_eff_days   manure_c_frac
       0.95         0.02         0.44           0.54    1000          0.5           100            0.50         0.15
```

Calibration

Eighteen of the 28 fields are wired in `cal_parm_select.f90` and may be touched through `calibration.cal`: `init_seq`, `init_microb`, `init_slow`, `init_passive`, `koc_c`, `solc_ratio`, `manure_c_frac`, `bio_consol`, `till_consol`, `sfc_rsd_photodeg`, `n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref`, `t_cbn_min`, `t_cbn_opt`, `t_cbn_max`. The six bmix/tillmix curve coefficients are also wired. The selector integers (`tmpf_eqn`, `watf_eqn`) and the integer-valued `till_eff_days` are not calibrated.

> **Note**
>
> `proc_cal` must run **before** `soil_nutcarb_init` for the `init_*` calibration values to take effect. The current `main.f90` already orders these correctly; do not reorder.

Related files

- `carbon_lyr.bsn`. Per-layer rate constants and CO2 allocation fractions.
- `codes.bsn`. The `cswat` switch that gates this file.
- `print.prt`. Output gating for the carbon families.
- Soil carbon (model theory). Equations and pool structure.
- Parameter change files. Calibration entries for the wired carbon parameters.

Purpose

`carbon_lyr.bsn` holds the carbon coefficients that differ between soil layer groups. It populates `carbdb(layer_id)` (rate constants for the litter, microbial, slow, and passive pools) and `org_allo(layer_id)` (the CO2 fractions produced by each pool transformation). The two layer groups currently in use are:

- `layer 1` (surface litter and topsoil)
- `layer 2` (all subsurface layers)

The reader indexes directly into the `carbdb(:)` and `org_allo(:)` arrays. Both are sized `dimension(2)` in `carbon_module.f90` today; a future expansion to additional layer groups requires only a larger array allocation, not a reader change.

Source

- Reader: same routine as `carbon.bsn`, `src/carbon_bsn_read.f90`. The per-layer block is parsed in the second half of the subroutine.
- Target types: `carbon_inputs` (`carbdb`) and `organic_allocations` (`org_allo`) in `src/carbon_module.f90`.
- Filename: hardcoded in the reader as `carbon_lyr.bsn`. **Not listed in `file.cio`** — it rides at a fixed name beside `carbon.bsn`.

Activation

Read only when `bsn_cc%cswat == 2` (the dynamic CENTURY/SWAT-C model; the activation value moved from 1 to 2). Like `carbon.bsn`, the file is **required**: an absent file, an unparseable row, or fewer data rows than the size of `carbdb` (currently 2) triggers `error stop` with a clear message.

Format

- Line 1: title (skipped).
- Line 2: column header (skipped).
- Lines 3..: one row per layer group. Each row begins with an integer `layer` id (1 = top, 2 = subsurface) followed by 11 real coefficients. The reader loops until end-of-file. Rows with an out-of-range `layer` id (less than 1 or greater than `size(carbdb)`) are skipped with a warning to unit 9001 and do not count toward the required row count.

#	Field	Type	Target	Units	Description
1	`layer`	int	(array index)	none	layer group id; 1 = top (surface litter + topsoil), 2 = all subsurface layers
2	`hp_rate`	real	`carbdb(layer)%hp_rate`	1/day	passive humus transformation rate under optimal conditions
3	`hs_rate`	real	`carbdb(layer)%hs_rate`	1/day	slow humus transformation rate under optimal conditions
4	`microb_rate`	real	`carbdb(layer)%microb_rate`	1/day	microbial biomass transformation rate under optimal conditions
5	`meta_rate`	real	`carbdb(layer)%meta_rate`	1/day	metabolic litter transformation rate under optimal conditions
6	`str_rate`	real	`carbdb(layer)%str_rate`	1/day	structural litter transformation rate under optimal conditions
7	`microb_top_rate`	real	`carbdb(layer)%microb_top_rate`	1/day	top-layer microbial activity adjustment
8	`hs_hp`	real	`carbdb(layer)%hs_hp`	none	slow-to-passive humus allocation coefficient
9	`a1co2`	real	`org_allo(layer)%a1co2`	fraction	decomposed metabolic litter routed to CO2
10	`asco2`	real	`org_allo(layer)%asco2`	fraction	decomposed slow humus routed to CO2
11	`apco2`	real	`org_allo(layer)%apco2`	fraction	decomposed passive humus routed to CO2
12	`abco2`	real	`org_allo(layer)%abco2`	fraction	decomposed microbial biomass routed to CO2

Example

refdata/Lafayette/carbon_lyr.bsn:

```
carbon_lyr.bsn: layer-keyed carbon coefficients (one row per layer group; 1 = top, 2 = subsurface; extend
   layer       hp_rate       hs_rate   microb_rate   meta_rate   str_rate   microb_top_rate   hs_hp   a1co
       1      1.2e-05      2.92e-04        0.0164      0.0405     0.0107            0.0164    0.05    0.60
       2      1.2e-05      1.81e-04          0.02      0.0507     0.0134              0.02    0.05    0.55
```

Calibration

All 11 per-layer fields are wired in `cal_parm_select.f90`. The calibration target is the `carbdb(ly)` or `org_allo(ly)` element selected by the `ly` index in the calibration record (see Parameter change files):

- `hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp` target `carbdb(ly)%*`.
- `a1co2`, `asco2`, `apco2`, `abco2` target `org_allo(ly)%*`.

Each `case` branch in `cal_parm_select.f90` guards `ly` to the range `[1, size(carbdb)]` (or `[1, size(org_allo)]`), so out-of-range layer indices in a calibration row are silently no-op rather than crashing.

Related files

- `carbon.bsn`. Basin-wide scalar carbon parameters.
- `codes.bsn`. The `cswat` switch.
- Soil carbon (model theory). Pool definitions and Century-style transformation equations.

Purpose

The `.con` files describe the spatial-object topology of the project. Every spatial object (HRU, channel, aquifer, reservoir, routing unit, recall, export coefficient, delivery ratio, outlet, gwflow cell) gets one row in its corresponding `.con` file. The row gives the object its sequential number, name, GIS id, drainage area, location, elevation, parameter pointers, weather station, optional constituent and overbank pointers, and the list of downstream objects that receive its outflow. The model uses these rows to build the routing graph in `hyd_connect`.

Each object type has its own `.con` file but every `.con` file is read by the same routine, `hyd_read_connect`, and is stored in the same array `ob`. The names of the files are listed in `type input_con` in `input_file_module.f90`:

File	Object type passed to `hyd_read_connect`	Default name
`hru.con`	`hru`	`in_con%hru_con`
`hru-lte.con`	`hru_lte`	`in_con%hruez_con`
`rout_unit.con`	`ru`	`in_con%ru_con`
`gwflow.con`	`gwflow`	`in_con%gwflow_con`
`aquifer.con`	`aqu`	`in_con%aqu_con`
`channel.con`	`chan`	`in_con%chan_con`
`chandeg.con`	`chandeg`	`in_con%chandeg_con`
`reservoir.con`	`res`	`in_con%res_con`
`recall.con`	`recall`	`in_con%rec_con`
`exco.con`	`exco`	`in_con%exco_con`
`delratio.con`	`dr`	`in_con%delr_con`
`outlet.con`	`outlet`	`in_con%out_con`

A 13th name, `aquifer2d.con` (`in_con%aqu2d_con`), is declared in `type input_con` but is not currently passed to `hyd_read_connect` from `hyd_connect.f90`.

Source

- Reader: src/hyd_read_connect.f90
- Dispatcher: src/hyd_connect.f90 (calls `hyd_read_connect` once per object type)
- Type definition: src/hydrograph_module.f90 (`type object_connectivity`, array `ob`)
- File name pointers: src/input_file_module.f90 (`type input_con`)

Format

- Line 1: title. Skipped.
- Line 2: header. Skipped.
- Line 3 and beyond: one record per object. The loop runs from `nspu1` to `nspu1 + nspu - 1`, where `nspu` is the count for this object type taken from `sp_ob` (set in `object.cnt`). The number of rows in the file must equal `nspu`.

Each record has a fixed leading block of 13 fields followed by `out_tot` connection tuples. The reader makes two list-directed reads of the same line: the first pass reads up to `src_tot`, then if `src_tot > 0` the reader backspaces and re-reads with the connection tuples appended.

Fixed leading block

#	Column (typical header)	Maps to	Type	Description
1	id / NUMB	`ob(i)%num`	int	Object number within this object type. Sequential. Becomes the spatial-object pointer used everywhere else (`hru` column of `hru-data.hru`, channel id in `chandeg.con`, etc).
2	name / NAME	`ob(i)%name`	char(16)	Object name. Free-form.
3	gis_id / GIS_ID	`ob(i)%gis_id`	int*8	GIS identifier from the source watershed delineation.
4	area / AREA_HA	`ob(i)%area_ha`	real (ha)	Drainage area. For HRU, routing unit, and recall objects this value is also copied into `ob(i)%area_ha_calc`. For other objects `area_ha_calc` is initialized to 0 and is filled in by `hyd_connect` from the routing topology.
5	lat / LAT	`ob(i)%lat`	real (deg)	Latitude.
6	lon / LONG	`ob(i)%long`	real (deg)	Longitude.
7	elev / ELEV	`ob(i)%elev`	real (m)	Elevation.
8	hru / aqu / lcha / rtu / RECALL	`ob(i)%props`	int	Pointer to the parameter record for this object. For `hru.con` it points into `hru-data.hru`; for `chandeg.con` into the SWAT-deg channel database; for `aquifer.con` into `aquifer.aqu`; and so on.
9	wst / WST	`ob(i)%wst_c`	char(50)	Weather station name. Resolved by `search(wst_n, db_mx%wst, wst_c, wst)` to the integer `ob(i)%wst` after the record is read.
10	cst / CONSTIT	`ob(i)%constit`	int	Pointer to constituent data (pesticides, pathogens, metals, salts). `0` means no constituent data.

#	Column (typical header)	Maps to	Type	Description
11	ovfl / OVERFLOW	`ob(i)%props2`	int	Overbank-connectivity pointer to a landscape unit. `0` means no overbank connection.
12	rule / RULESET	`ob(i)%ruleset`	char(16)	Name of a decision table in `flo_con.dtl` that controls the outflow fraction. Empty string means no rule.
13	out_tot / OUT_TOT	`ob(i)%src_tot`	int	Number of downstream (source) connections that follow on this line. May be 0.

Per-connection tuple

If `out_tot > 0`, the reader expects `out_tot` groups of four values after column 13, all on the same line:

Offset	Column (typical header)	Maps to	Type	Description
+1	obj_typ / OBTYP_OUT	`ob(i)%obtyp_out(k)`	char(3)	Type of the downstream object. Values used in `hyd_connect`: `hru`, `ru`, `aqu`, `cha`, `sdc` (SWAT-deg channel), `res`, `rec`, `exc`, `dr`, `out`.
+2	obj_id / HTYPNO_OUT	`ob(i)%obtypno_out(k)`	int	Sequential number of the downstream object within its type.
+3	hyd_typ / HTYP_OUT	`ob(i)%htyp_out(k)`	char(3)	Which outflow hydrograph from this object flows to the target. Values include `tot` (total), `rhg` (recharge), `sur` (surface), `lat` (lateral), `til` (tile). The set depends on the source object type; see `type hd_tot` in `hydrograph_module.f90`.
+4	frac / FRAC_OUT	`ob(i)%frac_out(k)`	real	Fraction of that hydrograph routed to the target. The fractions for one source object should sum to 1.0. The reader does not check this.

If `out_tot == 0`, the reader allocates `ob(i)%obtypno_out(1)` and sets it to 0. No connections are recorded.

gwflow side effect

After the connection block is read, if `bsn_cc%gwflow == 1` and any of the connections target an aquifer (`obtyp_out(k) == "aqu"`), the reader decrements `ob(i)%src_tot` by 1. The aquifer connection is left in the arrays but `src_tot` no longer counts it. This is part of the gwflow override that replaces standard aquifer routing with grid-cell routing (see `object.cnt`, gwflow side effect).

Example

`refdata/0su_1hru/aquifer.con`:

```
aquifer.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       aqu
       1  aqu001                   1      10.00000      35.64206     127.38513     322.26395         1
```

Notes:

- One aquifer object, named `aqu001`, with `area_ha = 10.0` and `out_tot = 1`.
- The single connection sends `tot` (total) outflow from this aquifer to SWAT-deg channel number 1 (`sdc 1 tot 1.00000`), fraction 1.0.

`refdata/Osu_1hru/rout_unit.con` shows a row with `out_tot = 2`:

```
rout_unit.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       rtu
       1  rtu001                   1       10.0000      35.52014     127.32787     113.51276         1
```

The routing unit `rtu001` sends total flow to SWAT-deg channel 1 (fraction 1.0) and recharge flow (`rhg`) to aquifer 1 (fraction 1.0). The two fractions need not sum to 1.0 because they apply to different hydrograph types (`tot` and `rhg`).

`refdata/Ames_sub1/hru.con` is the trivial case with `out_tot = 0` on every row:

```
hru.con: AMES
      id  name                gis_id          area           lat           lon          elev       hru
       1  hru0001                  1         1.005         41.20        -96.63         347.5         1
       2  hru0002                  1         1.005         41.20        -96.63         347.5         2
```

When `out_tot = 0` the row stops at column 13 and no connection tuples follow. Routing for these HRUs is then expected to be supplied through a separate routing unit definition.

Related files

- `object.cnt`. Sets `sp_ob%hru`, `sp_ob%chan`, etc, which determine how many rows `hyd_read_connect` expects in each `.con` file.
- `hru-data.hru`. The `hru` column in `hru.con` points into the rows of this file.
- `hru-lte.hru`, `rout_unit.def`, `aquifer.aqu`, `channel.cha`, `chandeg.cha`, `reservoir.res`, `recall.rec`, `exco.exc`, `delratio.del`. Each `.con` file points into one of these property files via the `props` column.
- `flo_con.dtl`. Decision tables named in the `ruleset` column.
- `weather-sta.cli`. Source of the names in the `wst` column.
- `gwflow.chancells`. When `bsn_cc%gwflow == 1`, the aquifer connection is silently dropped from `src_tot` after reading.

Purpose

`hru-data.hru` is the pointer table for every HRU in the project. Each record names one HRU and lists the parameter sets it draws from: topography, hydrology, soils, land use management, soil and plant initialization, surface storage, snow, and field. The names in this file are looked up in the corresponding parameter files (`topography.hyd`, `hydrology.hyd`, `soils.sol`, `landuse.lum`, `plant.ini`, `snow.sno`, `field.fld`). The integer indices resolved by those lookups are stored in `hru_db(i)%dbs` and used throughout the simulation to fetch HRU properties.

If the file is missing or set to `null` in `file.cio`, the reader allocates `hru_db(0:0)` and no HRU data is loaded.

Source

- Reader: `src/hru_read.f90`
- Type definition: `src/hru_module.f90` (`type hru_databases_char`, `type hydrologic_response_unit_db`)
- File name pointer: `src/input_file_module.f90` (`in_hru%hru_data`, default `"hru-data.hru"`)

Format

- Line 1: title. Skipped.
- Line 2: header. Skipped.

- Line 3 and beyond: one record per HRU. The reader makes two passes. The first pass scans the file to find the largest `id` and allocates `hru_db(0:imax)`. The second pass reads each record list-directed into `k, hru_db(i)%dbsc`, where `k` is a throwaway read of the id and `i` is the id parsed from the same line.

Record layout: 10 list-directed values per HRU.

#	Column	Maps to	Type	Description
1	id	record index `i`	int	HRU number. Used as the array index into `hru_db`. Does not have to be sequential. The largest `id` in the file sets the array size.
2	name	`hru_db(i)%dbsc%name`	char(40)	HRU name. Free-form identifier.
3	topo	`hru_db(i)%dbsc%topo`	char(40)	Topography record name. Looked up in `topography.hyd`. Missing names are logged to `diagnostics.out` as "not found (topography.hyd)".
4	hydro	`hru_db(i)%dbsc%hyd`	char(40)	Hydrology record name. Looked up in `hydrology.hyd`. Missing names are logged as "not found (hydrograph.hyd)".
5	soil	`hru_db(i)%dbsc%soil`	char(40)	Soil record name. Looked up in `soils.sol`. Missing names are logged as "not found (soils.sol)".
6	lu_mgt	`hru_db(i)%dbsc%land_use_mgt`	char(40)	Land use and management record name. Looked up in `landuse.lum`. Missing names are logged as "not found (landuse.lum)".
7	soil_plant_init	`hru_db(i)%dbsc%soil_plant_init`	char(40)	Soil and plant initialization record name. Looked up in `plant.ini`. Missing names are logged as "not found (plant.ini)". A match also cross-walks the initial soil-test, pesticide, pathogen, heavy-metal, salt, and constituent records named inside the `plant.ini` entry.
8	surf_stor	`hru_db(i)%dbsc%surf_stor`	char(40)	Surface storage (reservoir or wetland) record name. The reader stores the name but does not resolve a pointer for it in `hru_read`. Use `null` if the HRU has no surface storage.
9	snow	`hru_db(i)%dbsc%snow`	char(40)	Snow record name. Looked up in `snow.sno`. The literal `null` is allowed and suppresses the not-found warning.
10	field	`hru_db(i)%dbsc%field`	char(40)	Field record name. Looked up in `field.fld`. The literal `null` is allowed and suppresses the not-found warning.

The reader does not enforce a count line. The number of HRUs comes from the maximum `id` seen during the scan pass, so a gap in the numbering allocates unused slots in `hru_db`.

Example

`refdata/Ames_sub1/hru-data.hru`:

```
hru-data.hru:
      id  name                      topo            hydro            soil           lu_mgt   soil_
       1  hru0001              topohru0001          hyd0001        soil_01-h1       cosy_lum        
       2  hru0002              topohru0002          hyd0002        soil_02          mntill_corn_lum 
       3  hru0003              topohru0003          hyd0003        soil_03-h3       cosy_lum        
       4  hru0004              topohru0004          hyd0004        soil_04          cosy_lum        
```

Notes on the example:

- HRU 2 uses `mntill_corn_lum` while the rest use `cosy_lum`. The land use record drives tillage, plant cover, and management operations.

- `surf_stor` and `field` are `null` for every HRU. None of the HRUs has a pond, wetland, or tile-drained field assigned.
- `snow001` is the same record for every HRU. The snow record is a per-elevation-band parameter set.

Related files

- `object.cnt`. The `hru` count must be at least the largest `id` in this file.
- `hru.con`. One row per HRU, in the same id order.
- `topography.hyd`, `hydrology.hyd`, `soils.sol`, `landuse.lum`, `plant.ini`, `snow.sno`, `field.fld`. Targets of the name pointers above.
- `file.cio`. The `hru` category line points to this file.

Purpose

This page documents the per-HRU hydrology and topography parameter files: `hydrology.hyd`, `topography.hyd`, and `field.fld`, plus the closely related shade-factor file `shade_factor.shf`. File names for the first three are set in `type input_hydrology` in `src/input_file_module.f90`; the shade file is set in `type shade_factor` in the same module.

Records in `hydrology.hyd`, `topography.hyd`, and `field.fld` are referenced by name from `hru-data.hru` (columns `hydro`, `topo`, and the field-name slot). Every HRU points at one record from each. Records in `shade_factor.shf` are used by the channel temperature routine in `src/ch_temp.f90`.

Each file is optional. If the slot in `file.cio` is `null` or the file is missing, the reader allocates a zero-length array and skips on. All four readers skip two header lines, then read one record per parameter set.

Source

- File names: `src/input_file_module.f90`, `type input_hydrology`, `type shade_factor`
- Readers:
  - `src/hydrol_read.f90` (`hydrology.hyd`)
  - `src/topo_read.f90` (`topography.hyd`)
  - `src/field_read.f90` (`field.fld`)
  - `src/shade_factor_read.f90` (`shade_factor.shf`)
- Type definitions: `src/hydrology_data_module.f90`, `src/topography_data_module.f90`, `src/hydrograph_module.f90`

hydrology.hyd

Per-HRU hydrology parameter sets (lateral flow, canopy, evaporation compensation, percolation, enrichment ratios). Read into `hyd_db` (`type hydrology_db` in `src/hydrology_data_module.f90`).

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“”	parameter set name
2	lat_ttime	real	days	0.0	exponential of the lateral flow travel time (0..120)
3	lat_sed	real	g/L	0.0	sediment concentration in lateral flow
4	canmx	real	mm H2O	0.0	maximum canopy storage
5	esco	real	none	0.0	soil evaporation compensation factor (0..1)
6	epco	real	none	0.0	plant water uptake compensation factor (0..1)
7	erorgn	real	none	0.0	organic N enrichment ratio (0 = model calculates per event)
8	erorgp	real	none	0.0	organic P enrichment ratio (0 = model calculates per event)
9	cn3_swf	real	none	0.0	soil water at CN3 (0 = field capacity, 0.99 = near saturation)
10	biomix	real	none	0.0	biological mixing efficiency, applied at end of each calendar year
11	perco	real	none	0.0	percolation coefficient, linear adjustment to daily perc (0..1)
12	lat_orgn	real	ppm	0.0	organic N concentration in lateral flow
13	lat_orgp	real	ppm	0.0	organic P concentration in lateral flow
14	pet_co	real	none	1.0	radiation coefficient used in Hargreaves equation
15	latq_co	real	none	0.3	lateral soil flow coefficient, linear adjustment to daily lateral flow (0..1)

Example from `refdata/Osu_1hru/hydrology.hyd`:

```
hydrology.hyd: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                 lat_ttime       lat_sed       can_max          esco          epco   orgn_enrich   org
hyd0001                0.00000       0.00000       1.00000       0.95000       0.50000       0.00000
```

Editor column names (`can_max`, `harg_pet`, `bio_mix`) differ from source field names. The reader uses position only.

topography.hyd

Per-HRU topography parameter sets. Read into `topo_db` (`type topography_db` in `src/topography_data_module.f90`).

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	slope	real	m/m	0.02	average slope steepness in HRU
3	slope_len	real	m	50.0	average slope length for erosion
4	lat_len	real	m	50.0	slope length for lateral subsurface flow
5	dis_stream	real	m	100.0	average distance to stream
6	dep_co	real	none	1.0	deposition coefficient

Example from `refdata/Osu_1hru/topography.hyd`:

```
topography.hyd: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        slp       slp_len       lat_len      dist_cha         depos
topohru0001             0.01078      10.00000      10.00000     121.00000       0.00000
```

The same type and array are also used for landscape units; `hru-data.hru` and the subbasin objects both reference entries in `topo_db` by name.

field.fld

Per-field geometry used by the wind-erosion routine. Read into `field_db` (`type fields_db` in `src/topography_data_module.f9`

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	length	real	m	500.0	field length for wind erosion
3	wid	real	m	100.0	field width for wind erosion
4	ang	real	deg	30.0	field angle for wind erosion

Example from `refdata/Osu_1hru/field.fld` (first four records shown):

```
field.fld: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       len            wd           ang
fld011                500.00000     100.00000      30.00000
fld012                500.00000     100.00000      30.00000
fld031                500.00000     100.00000      30.00000
fld032                500.00000     100.00000      30.00000
```

The reader uses a `read .. backspace .. read` pattern, but the file format is one record per line, same as the other files on this page.

shade_factor.shf

Time- and landscape-unit-resolved shade factor used by the channel water-temperature routine. Read into `shf_db` (`type shade_factor_data` in `src/hydrograph_module.f90`). Activated per channel by `sf_on = 1` in `temperature.cha`; when `sf_on = 0` the channel uses the calibration-file value instead.

#	Column	Type	Units	Default	Description
1	jday	int	day of year	0	day of year the record applies to
2	lsu	int	none	0	landscape unit number
3	value	real	none	0.0	shade factor value

In `src/ch_temp.f90` the array is scanned for the matching `(jday, lsu)` pair and the `value` is assigned to `ssff`. There is no name column.

No example dataset in `refdata/` ships a `shade_factor.shf` file.

Related files

- `file.cio`. The `hru` line selects `hydrology.hyd`, `topography.hyd`, and `field.fld`; the shade file is listed separately.
- `hru-data.hru`. Every HRU references one entry from each of `hydrology.hyd`, `topography.hyd`, and `field.fld`.
- `temperature.cha`. Controls whether `shade_factor.shf` is consulted (`sf_on` flag).

Purpose

The recall module reads externally provided time series and exposes them as spatial objects in the SWAT+ object graph. A recall object is a point source, an outlet, or a basin inlet whose flow and constituent loads are read from disk instead of simulated. Each recall has its own time step (subdaily, daily, monthly, or annual). The hydrograph delivered by a recall feeds the receiving object through `recall.con`, exactly the same way that an HRU or a channel feeds its downstream link.

Inputs come in two layers:

- A master database file (`recall.rec`) lists every recall object and points to the per-object side files for organic and mineral constituents and, optionally, for pesticides, pathogens, heavy metals, salts, and other constituents.
- Optional parallel database files (`salt_recall.rec`, `cs_recall.rec`) carry the time series of salt-ion and constituent loads when those modules are active.
- Optional spatial-element files (`rec_catunit.def`, `rec_reg.def`, `rec_catunit.ele`) group recall objects into regions used for output aggregation and soft calibration.

Source

- Master reader: `src/recall_read.f90` (`recalldb_read` and `recall_read`)
- Salt reader: `src/recall_read_salt.f90`
- Constituent reader: `src/recall_read_cs.f90`
- Element reader: `src/rec_read_elements.f90`
- Type definitions: `src/recall_module.f90` (`type recall_databases`, `type constituent_file_data`), `src/hydrograph_module.f90` (`type recall_hydrograph_inputs`), `src/constituent_mass_module.f90` (`type recall_salt_inputs`, `type recall_cs_inputs`, `type recall_pesticide_inputs`).

recall.rec (master database)

Lists every recall object and names the side files that hold its time-series data.

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per recall object. Two passes: the first finds the maximum object number, the second reads the records.

Each record is read into `recall_db(i)` (`type recall_databases`):

#	Field	Type	Description
1	id	int	Sequential id. Used as the array index.
2	name	char(13)	Recall object name. Referenced by `recall.con` and by the `obj_typ = rec` slots in other connectivity files.

#	Field	Type	Description
3	org_min	record	Organic and mineral side file. Three subfields read together: `name` (filename), `units` (`mass` or `conc`), `tstep` (`sub`, `day`, `mo`, `yr`).
4	pest	record	Pesticide side file (same three subfields).
5	path	record	Pathogen side file.
6	hmet	record	Heavy-metal side file.
7	salt	record	Salt side file.
8	constit	record	Constituent (other) side file.

`type constituent_file_data` has three character fields read in order: `name(25)`, `units(13)`, `tstep(13)`. List-directed reads consume three tokens per record slot. After reading each row, the master reader calls `recall_read(i)` which opens the file in `recall_db(i)%org_min%name` and reads the time series.

Per-object time series

`recall_read` opens the organic-mineral side file and dispatches on `tstep`:

`tstep`	Storage	Records per year
`sub`	`recall(irec)%hyd_flo(step*366, nbyr)` plus `recall(irec)%hd(366, nbyr)`	Subdaily. Flow is converted from m^3/s to m^3 per subdaily step.
`day`	`recall(irec)%hd(366, nbyr)`	One row per Julian day.
`mo`	`recall(irec)%hd(12, nbyr)`	One row per month.
`yr`	`recall(irec)%hd(1, nbyr)`	One row per year.

Side-file format (`day`, `mo`, `yr`):

- Line 1: title.
- Line 2: `nbyr` (number of years).
- Line 3: header (skipped).
- Lines 4+: per time step, `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then a 22-field `hyd_output` record (flow plus organic, mineral, and physical constituents).

Each `hyd_output` row supplies, in order: `flo`, `sed`, `orgn`, `sedp`, `no3`, `solp`, `chla`, `nh3`, `no2`, `cbod`, `dox`, `san`, `sil`, `cla`, `sag`, `lag`, `grv`, `temp`. Flow is in m^3/s. Sediment is in metric tons. Nutrients and other constituents are in kg. See `type hyd_output` for the full column list.

The reader fast-forwards the side file to the row whose `iyr` matches `time%yrc` (simulation start year). Rows before the simulation start are skipped; rows after `time%yrc_end` end the read. Each row sets `recall(irec)%start_yr` to the first matching year and `recall(irec)%end_yr` to the last row read.

pest.com (recall pesticide side database)

When `pest.com` exists in the working directory, `recall_read` reads it as a parallel pesticide database. It is structured like `recall.rec`: a master file naming per-pesticide files. Each per-pesticide file has its own `nbyr` and time step (1=day, 2=mo, 3=yr) and fills `rec_pest(i)%hd_pest(istep, nbyr)`. The exact column count of the per-pesticide payload depends on `db_mx%pestcom` (the number of pesticides in the constituent database).

salt_recall.rec (salt side database)

Mirror of `recall.rec` for salt loads, read when the salt module is active.

- Line 1: title.
- Line 2: header.
- Lines 3+: one record per recall object. Fields: `id`, `name`, `typ` (1=day, 2=mo, 3=yr, 4=exco crosswalk), `filename`.

For `typ` in `1..3`, the side file is opened with:

- Line 1: title. If the literal token is `Incoming` the source is flagged as originating from outside the watershed (`pts_type = 2`); otherwise `pts_type = 1` (within the watershed).
- Line 2: `nbyr`.
- Line 3: header.
- Lines 4+: `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then `cs_db%num_salts` real values (one per salt ion). The order of ions follows `constituents.cs`.

Each side file fills `rec_salt(i)%hd_salt(istep, nbyr)%salt(1..num_salts)`. For `typ = 4`, the salt record is crosswalked to an `exco` entry; no side file is opened.

cs_recall.rec (constituent side database)

Same structure as `salt_recall.rec`, but for the other-constituent module. Fields per record: `id`, `name`, `typ`, `filename`. Side files have the same five-line header pattern; payload rows carry `cs_db%num_cs` real values that fill `rec_cs(i)%hd_cs(istep, nbyr)%cs(1..num_cs)`. The `Incoming` flag in the title is honoured identically.

Element and region files

Three files in `file.cio`'s `regions` slot group recall objects for output aggregation and soft calibration. Names default to `rec_catunit.def`, `rec_reg.def`, and `rec_catunit.ele` (set by `in_regs%def_psc`, `in_regs%def_psc_reg`, and `in_regs%ele_psc`). All three are optional.

rec_catunit.def (output regions)

Defines the regions used to print recall output. Format:

- Line 1: title.
- Line 2: number of regions, `mreg`.
- Line 3: header.
- Lines 4+: per region: `id`, `name`, `area_ha`, `nspu`, then `nspu` element ids.

When `nspu > 0`, the listed element ids are passed to `define_unit_elements` and stored in `pcu_out(i)%num`. When `nspu = 0`, the region defaults to every recall in the basin (`sp_ob%recall` elements).

rec_reg.def (calibration regions)

Same format as `rec_catunit.def`. Populates `pcu_reg(i)` instead of `pcu_out(i)`. Used for soft-calibration grouping and per-type output.

rec_catunit.ele (element table)

Lists every element that can appear in `rec_catunit.def` or `rec_reg.def`. Format:

- Line 1: title.
- Line 2: header.
- Lines 3+: per element: `id`, `name`, `obtyp`, `obtypno`, `bsn_frac`, `ru_frac`, `reg_frac`.

`obtyp` is the object-type tag (typically `rec`) and `obtypno` is the recall object number. The three fractions are the share of the element in the basin, in its routing unit, and in its region.

Example

`refdata/Osu_1hru/recall.rec`:

```
recall.rec
NUM     NAME    TYPE     FNAME    OBJ_TYP     OBJ_NAME       flo      sed     orgn     sedp      no3     solp
   1    PSTYR    3      RECYR.        hru         hru1   12120.5        0        0        0        0        0
```

```
   1       1          1     2011         hru          hru2     9000.7        0        0        0        0        0
```

This dataset stores one recall object (PSTYR) whose annual hydrograph lives in a side file. The trailing 22-column row is the actual hyd_output payload that the side file would contain. In a clean layout, the side file (RECYR. or similar) is its own file: a title line, a year count, a header, then one or more annual rows.

Related files

- recall.con. Spatial-connectivity entries for each recall object. Links the recall to its receiving spatial object.
- file.cio. Names the master recall.rec (in_rec%recall_rec) and the element files (in_regs%def_psc, in_regs%def_psc_reg, in_regs%ele_psc).
- salt module, cs module. Document the salt and other-constituent modules whose recall side databases parallel recall.rec.

Purpose

The .cli files are index files. Each one names a set of climate inputs that SWAT+ should load. They sit between file.cio, which names the index files themselves, and the actual time series files (*.pcp, *.tmp, *.slr, *.hmd, *.wnd, *.pet, weather-generator stations, atmospheric deposition).

The full set of index files is defined by type input_cli in src/input_file_module.f90:

Slot	Default name	Reader	Purpose
weat_sta	weather-sta.cli	cli_staread.f90	Weather stations: one row per station, columns are pointers to a wgn station and to per-variable gages.
weat_wgn	weather-wgn.cli	cli_wgnread.f90	Weather generator parameters: location plus 12 monthly statistics.
pet_cli	pet.cli	cli_petmeas.f90	List of measured PET station files.
pcp_cli	pcp.cli	cli_pmeas.f90	List of measured precipitation station files.
tmp_cli	tmp.cli	cli_tmeas.f90	List of measured temperature station files.
slr_cli	slr.cli	cli_smeas.f90	List of measured solar radiation station files.
hmd_cli	hmd.cli	cli_hmeas.f90	List of measured relative humidity station files.
wnd_cli	wnd.cli	cli_wmeas.f90	List of measured wind speed station files.
atmo_cli	atmodep.cli	cli_read_atmodep.f90	Atmospheric N deposition: control record plus one or more stations.

In file.cio the slot value is either a filename or the literal token null. Every reader skips its work when the slot is null or when the file does not exist. The per-variable readers (pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli, pet.cli) all have the same shape: title, header, then one filename per line. The actual time-series files are documented separately.

All readers skip a title line and a header line at the top of the file. Records are list-directed (free format), so columns are separated by whitespace.

weather-sta.cli

One record per weather station. Each row binds a station name to a weather generator and to gage names for each climate variable.

Source

- Reader: src/cli_staread.f90
- Type: src/climate_module.f90, `type weather_station` (column 1) and `type weather_codes_station_char` (columns 2 to 9)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: 9 fields per record, list-directed.

The reader call is `read (107,*,iostat=eof) wst(i)%name, wst(i)%wco_c`. The fields of `wco_c` are filled in declaration order.

#	Column	Type	Description
1	name	char(50)	Weather station name. Referenced by hru.con and other spatial-object files.
2	wgn	char(50)	Name of the weather generator station in `weather-wgn.cli`.
3	pgage	char(50)	Name of the precipitation gage in `pcp.cli`, or `sim` to generate.
4	tgage	char(50)	Name of the temperature gage in `tmp.cli`, or `sim` to generate.
5	sgage	char(50)	Name of the solar radiation gage in `slr.cli`, or `sim` to generate.
6	hgage	char(50)	Name of the relative humidity gage in `hmd.cli`, or `sim` to generate.
7	wgage	char(50)	Name of the wind speed gage in `wnd.cli`, or `sim` to generate.
8	petgage	char(50)	Name of the measured PET station in `pet.cli`, or `null`.
9	atmodep	char(50)	Name of the atmospheric deposition station in `atmodep.cli`, or `null`.

Each gage name is matched against the station list of the corresponding `.cli` file. A missing match (and a value other than `sim` for the generated variables, or `null` for pet/atmodep) is written to the error log at unit 9001.

Example

`refdata/Ames_sub1/weather-sta.cli`:

```
weather-sta.cli:
name                            wgn              pcp              tmp              slr              hm
wgn_sta1                    wgn_001         ames.pcp         ames.tem              sim              si
```

The header column at position 8 is labelled `wnd_dir` by SWAT+ Editor. The reader stores that value in `petgage` (the PET gage slot).

weather-wgn.cli

One record per weather generator station. Each record is a station header line plus a 12-row monthly statistics block.

Source

- Reader: src/cli_wgnread.f90
- Type: src/climate_module.f90, `type weather_generator_db`

Format

- Line 1: title (skipped).
- Per station, 14 lines:
  - Line A: station name, latitude, longitude, elevation, rain_yrs.
  - Line B: header for the monthly block (skipped).
  - Lines C1..C12: one line per month, 14 values each.

Number of stations is determined by reading to end of file.

Station header line:

#	Field	Type	Units	Description
1	name	char	-	Generator station name. Matched by `wgn` in `weather-sta.cli`.
2	lat	real	degrees	Latitude of the generator station.
3	long	real	degrees	Longitude of the generator station.
4	elev	real	m	Elevation of the generator station.
5	rain_yrs	real	years	Number of years of record used to compute `rainhmx`.

Monthly block, 12 rows (Jan to Dec), 14 columns:

#	Field	Type	Units	Description
1	tmpmx	real	deg C	Average daily maximum air temperature.
2	tmpmn	real	deg C	Average daily minimum air temperature.
3	tmpstdmx	real	deg C	Standard deviation of daily maximum air temperature.
4	tmpstdmn	real	deg C	Standard deviation of daily minimum air temperature.
5	pcpmm	real	mm	Total precipitation in the month.
6	pcpstd	real	mm/day	Standard deviation of daily precipitation.
7	pcpskw	real	none	Skew coefficient of daily precipitation.
8	pr_wd	real	none	Probability of a wet day after a dry day.
9	pr_ww	real	none	Probability of a wet day after a wet day.
10	pcpd	real	days	Average number of days of precipitation in the month.
11	rainhmx	real	mm	Maximum 0.5-hour rainfall in the month.
12	solarav	real	MJ/m^2/day	Average daily solar radiation.
13	dewpt	real	deg C	Average daily dew point temperature.
14	windav	real	m/s	Average daily wind speed.

Example

`refdata/Ames_sub1/weather-wgn.cli`:

```
weather-wgn.cli: Ames
wgn001      37.48     -100.83   887.00    10
   TMPMX   TMPMN  TMPSTDMX TMPTDMN  PCPMM  PCPSTD  PCPSKW   PR_WD   PR_WW   PCPD  RAINHMX SOLARAV  DEWPT
    7.61   -8.13    8.06    5.99     7.4     3.3    0.19    0.07    0.25    2.65     5.1   10.39   -8.14
   11.07   -5.53    8.06    5.34    10.2     5.1    2.67    0.08    0.27    2.86     1.8   13.41   -5.58
   15.22   -1.87    7.99    5.22    28.1     8.9    1.64    0.11    0.34    4.43     7.6   17.85   -4.71
   21.42    4.12    6.42    4.82    32.0     7.9    1.65    0.14    0.34    5.25    24.6    21.7    0.93
```

```
 25.97    9.93    5.68    4.29    83.7    12.4    1.71    0.21    0.42    8.24    26.9   23.09    7.81
 31.78   15.51    4.96    3.62    71.6    13.0     0.9    0.17    0.41    6.71    40.4   26.15   12.49
 34.28   18.34    3.68    2.51    65.0    12.4    1.25    0.18    0.33    6.56    37.3   25.85   14.72
 33.17   17.22    3.89    2.59    65.0    13.0    1.26    0.19    0.27     6.4    32.0   22.54   14.29
 29.04   12.36    5.12    4.25    43.5    13.5    3.71    0.13    0.31    4.76    18.0   20.32    9.38
 23.43    5.54    6.17    4.63    29.8    11.9    1.31    0.08    0.32    3.26    10.4   15.04    3.48
 14.34   -1.71    6.76    4.75    20.1     6.9    0.58    0.09    0.34     3.6    10.4   15.46   -2.96
  9.17   -6.26    7.09    5.01     8.0     3.6    1.24    0.07    0.24    2.61    16.3    9.76   -6.38
```

This file holds one generator station (`wgn001`).

pcp.cli

List of measured precipitation station files. Each row points to one `*.pcp` file in the project directory (or under the path set by `in_path_pcp%pcp`).

Source

- Reader: src/cli_pmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`pcp` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.pcp` time-series file. The same string is used as the station key (`pcp_n(i)`) that `weather-sta.cli` matches against.

The reader makes two passes over the file. The first pass reads the field into `pcp_n(i)` (the lookup name). The second pass reads the same field into `pcp(i)%filename` and opens that file.

Example

`refdata/Ames_sub1/pcp.cli`:

```
pcp.cli
FILENAME
ames.pcp
```

One station, file `ames.pcp` in the project directory.

tmp.cli

List of measured temperature station files.

Source

- Reader: src/cli_tmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`tmp` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.tmp` time-series file. Used as the station key (`tmp_n(i)`).

Optional weather path prefix from `in_path_tmp%tmp` (see `file.cio`).

Example

`refdata/Ames_sub1/tmp.cli`:

```
tmp.cli
FILENAME
ames.tmp
```

slr.cli

List of measured solar radiation station files.

Source

- Reader: src/cli_smeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`slr` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.slr` time-series file. Used as the station key (`slr_n(i)`).

Example

```
slr.cli
FILENAME
ames.slr
```

The reference dataset `refdata/Ames_sub1` does not ship a `slr.cli` because the weather-station rows use `sim` for solar radiation. The example shape above is what the reader expects when the file is present.

hmd.cli

List of measured relative humidity station files.

Source

- Reader: src/cli_hmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`hmd` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.hmd` time-series file. Used as the station key (`hmd_n(i)`).

Example

```
hmd.cli
FILENAME
ames.hmd
```

wnd.cli

List of measured wind speed station files.

Source

- Reader: src/cli_wmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`wnd` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.wnd` time-series file. Used as the station key (`wnd_n(i)`).

Example

```
wnd.cli
FILENAME
ames.wnd
```

pet.cli

List of measured potential ET station files. Used when codes.bsn sets `pet = 3`.

Source

- Reader: src/cli_petmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`petm` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.pet` time-series file. Used as the station key (`petm_n(i)`).

Example

```
pet.cli
FILENAME
ames.pet
```

atmodep.cli

Atmospheric N deposition for one or more stations. Unlike the per-variable index files, this file contains the deposition values themselves, not pointers to data files.

Source

- Reader: src/cli_read_atmodep.f90
- Types: src/climate_module.f90, `type atmospheric_deposition_control` (line 3) and `type atmospheric_deposition` (per-station block)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: control record, 5 fields.
- Then, per station, a block whose layout depends on `timestep`.

Control record (line 3):

#	Field	Type	Description
1	num_sta	int	Number of deposition stations to read.
2	timestep	char(2)	`aa` (average annual, scalar values), `mo` (monthly time series), or `yr` (annual time series).
3	mo_init	int	First month of the time series (1 to 12). Used when `timestep = mo`.
4	yr_init	int	First year of the time series. Used for `mo` and `yr`.
5	num	int	Number of records per station (length of the time series). Set to `1` for `aa`.

For `timestep = aa`, each station occupies 5 lines:

Line	Type	Description
1	char(50)	Station name (used as `atmo_n(i)`).
2	real	`nh4_rf` - average annual ammonia concentration in rainfall (mg/L).
3	real	`no3_rf` - average annual nitrate concentration in rainfall (mg/L).
4	real	`nh4_dry` - average annual ammonia dry deposition (kg/ha/yr).
5	real	`no3_dry` - average annual nitrate dry deposition (kg/ha/yr).

For `timestep = mo`, each station occupies 5 lines (name plus four monthly series of length `num`):

Line	Description
1	Station name.

Line	Description
2	`nh4_rfmo(1..num)` - ammonia in rainfall by month (mg/L).
3	`no3_rfmo(1..num)` - nitrate in rainfall by month (mg/L).
4	`nh4_drymo(1..num)` - ammonia dry deposition by month (kg/ha/yr).
5	`no3_drymo(1..num)` - nitrate dry deposition by month (kg/ha/yr).

For `timestep = yr`, each station occupies 5 lines (name plus four annual series of length `num`):

Line	Description
1	Station name.
2	`nh4_rfyr(1..num)` - ammonia in rainfall by year (mg/L).
3	`no3_rfyr(1..num)` - nitrate in rainfall by year (mg/L).
4	`nh4_dryyr(1..num)` - ammonia dry deposition by year (kg/ha/yr).
5	`no3_dryyr(1..num)` - nitrate dry deposition by year (kg/ha/yr).

When `num_sta = 0` (no stations) or the file is named `null` in `file.cio` or absent, the reader allocates an empty array and skips deposition.

Example

Minimal `aa` form, one station:

```
atmodep.cli
num_sta  timestep  mo_init  yr_init  num
      1        aa        1     2000    1
station_001
1.0
0.2
0.0
0.0
```

Atmospheric deposition

Three readers populate the atmospheric deposition state. They share the per-station list set up by `atmodep.cli` and the control record `atmodep_cont` (`type atmospheric_deposition_control`). The three files cover N, salts, and other constituents.

atmodep.cli (nitrogen)

Documented above. Read by src/cli_read_atmodep.f90. Fills `atmodep(0:num_sta)` (`type atmospheric_deposition`) with NH4 and NO3 wet (rainfall) concentrations and dry deposition rates. The reader also computes `atmodep_cont%ts` (the index into the time series corresponding to the first simulation step) for `mo` and `yr` time steps, and sets `atmodep_cont%first = 0` once aligned.

salt_atmo.cli (salt ions)

Read by src/cli_read_atmodep_salt.f90 when `cs_db%num_salts > 0`. Per-station wet and dry deposition for each salt ion in `constituents.cs`. Filled into `atmodep_salt(iadep)%salt(isalt)` (`type atmospheric_deposition_cs` within `type object_deposition_cs`).

Layout:

- Lines 1 to 6: skipped. The reader consumes six lines as commentary before processing stations.
- Per station, block layout depends on `atmodep_cont%timestep` (set by `atmodep.cli`):

For `timestep = aa`:

Line	Description
1	Station name (character, ignored for cross-matching; positional).
2..1+num_salts	`salt_ion_label`, then one real: `rf` (rainfall concentration, mg/L) for each ion.
…next num_salts	`salt_ion_label`, then one real: `dry` (dry deposition, kg/ha/yr) for each ion.

For `timestep = mo`:

Line	Description
1	Station name.
2..1+num_salts	`salt_ion_label`, then `num` reals: monthly wet concentrations (mg/L).
…next num_salts	`salt_ion_label`, then `num` reals: monthly dry deposition (kg/ha/yr).

For `timestep = yr`: same shape as monthly with annual series of length `num`.

When the file exists, the reader sets the module-level flag `salt_atmo = "y"`. When `cs_db%num_salts = 0` or the file is missing, salt deposition is skipped.

cs_atmo.cli (other constituents)

Read by src/cli_read_atmodep_cs.f90 when `cs_db%num_cs > 0`. Same shape as `salt_atmo.cli` but without per-ion labels.

Layout:

- Lines 1 to 3: skipped.
- Per station:

For `timestep = aa`:

Line	Description
1	Station name (real-typed scratch read; positional).
2..1+num_cs	One real per line: wet concentration (mg/L) for each constituent.
…next num_cs	One real per line: dry deposition (kg/ha) for each constituent.

For `timestep = mo` and `timestep = yr`, each constituent gets one line of `num` reals for wet, and one line of `num` reals for dry, in the same order as `constituents.cs`.

When the file exists, the reader sets `cs_atmo = "y"`.

Related files

- file.cio. Names every `.cli` file via `type input_cli`.
- codes.bsn. `pet = 3` triggers reading `pet.cli`.
- `*.pcp`, `*.tmp`, `*.slr`, `*.hmd`, `*.wnd`, `*.pet`. The actual time-series files listed in the per-variable `.cli` files.
- salt module, cs module. Define the salt and other-constituent lists that drive `salt_atmo.cli` and `cs_atmo.cli`.

Purpose

A `*.pcp` file holds the measured precipitation time series for a single gauge. One file per station. The filename is listed in pcp.cli and the same filename string is matched as the gauge key referenced from `pgage` in weather-sta.cli. The reader supports either daily values or subdaily values at a fixed time step.

Source

- Reader: src/cli_pmeas.f90
- Type definition: src/climate_module.f90, `type climate_measured_data` (the `pcp` array)

Format

The reader uses list-directed reads (free format, whitespace-separated). The first two lines are skipped. The third line is the station header. From line four to end of file are the time-step records.

Line 1: title

Single string, skipped.

Line 2: column header

Skipped. SWAT+ Editor writes `NBYR  TSTEP  LAT  LONG  ELEV`.

Line 3: station header

Read by `read (108,*,iostat=eof) pcp(i)%nbyr, pcp(i)%tstep, pcp(i)%lat, pcp(i)%long, pcp(i)%elev`.

#	Field	Type	Units	Description
1	nbyr	int	years	Number of years of record in the file. Used to size the in-memory time-series array.
2	tstep	int	minutes	Time step. `0` means daily. A positive value is the subdaily interval in minutes and must match `time%step` from time.sim.
3	lat	real	degrees	Latitude of the gauge.
4	long	real	degrees	Longitude of the gauge.
5	elev	real	m	Elevation of the gauge.

Records (line 4 onward)

The record layout depends on `tstep`.

For daily records (`tstep = 0`):

#	Field	Type	Units	Description
1	year	int	calendar year	Calendar year of the record.
2	jday	int	day-of-year	Julian day (1 to 365 or 366).
3	pcp	real	mm	Daily precipitation depth. Use a value at or below `-97` to flag missing data; the reader replaces it with a generated value.

For subdaily records (`tstep > 0`):

#	Field	Type	Units	Description
1	year	int	calendar year	Calendar year.
2	jday	int	day-of-year	Julian day.
3	mo	int	month	Month number.
4	day_mo	int	day	Day of the month.
5	ihr	int	step index	Sub-day step index (1 to `1440 / tstep`).
6	pcp	real	mm	Precipitation depth for the time step.

Records are read in order. The reader advances year-by-year using the change in `jday` (it expects the year to change after `jday = 365` or `366`).

Missing data

The daily reader treats any value at or below `-97` as missing. Downstream in `cli_precip_control.f90`, when `precip_next <= -97.` the value is replaced by the weather generator. The classic flag written by SWAT+ Editor and the legacy tools is `-99`.

Example

First lines of `refdata/Ames_sub1/ames.pcp`:

```
AME.pcp
 NBYR    TSTEP       LAT      LONG      ELEV
   121        0     42.04    -93.89    316
1900 1 -99
1900 2 -99
1900 3 -99
...
1983 184 0
1983 185 15.5
1983 186 0
```

Station header: 121 years, daily time step (`0`), latitude 42.04, longitude -93.89, elevation 316 m. Early years are missing (`-99`) and will be filled by the weather generator.

Related files

- `pcp.cli`. Lists each `*.pcp` filename.
- `weather-sta.cli`. Binds a precipitation gauge to a weather station via `pgage`.
- `file.cio`. Names the `pcp.cli` index and the optional path prefix `in_path_pcp%pcp` applied to every `*.pcp` filename.
- `time.sim`. For subdaily files, `time%step` must equal the file's `tstep`.

Purpose

A `*.tmp` file holds the measured daily temperature time series for a single gauge. One file per station. The filename is listed in `tmp.cli` and the same filename string is matched as the gauge key referenced from `tgage` in `weather-sta.cli`. The reader records daily maximum and minimum air temperature.

Source

- Reader: `src/cli_tmeas.f90`
- Type definition: `src/climate_module.f90`, `type climate_measured_data` (the `tmp` array; `ts` holds daily max, `ts2` holds daily min)

Format

List-directed reads (free format, whitespace-separated). The first two lines are skipped. The third line is the station header. From line four to end of file are the daily records.

Line 1: title

Single string, skipped.

Line 2: column header

Skipped. SWAT+ Editor writes `NBYR  TSTEP  LAT  LONG  ELEV`.

Line 3: station header

Read by `read (108,*,iostat=eof) tmp(i)%nbyr, tmp(i)%tstep, tmp(i)%lat, tmp(i)%long, tmp(i)%elev`.

#	Field	Type	Units	Description
1	nbyr	int	years	Number of years of record in the file.
2	tstep	int	minutes	Time step indicator. Stored but the reader expects daily values regardless.
3	lat	real	degrees	Latitude of the gauge.
4	long	real	degrees	Longitude of the gauge.
5	elev	real	m	Elevation of the gauge.

Records (line 4 onward)

Daily values only:

#	Field	Type	Units	Description
1	year	int	calendar year	Calendar year of the record.
2	jday	int	day-of-year	Julian day (1 to 365 or 366).
3	tmax	real	deg C	Daily maximum air temperature. Stored in `tmp(i)%ts(jday, year_index)`.
4	tmin	real	deg C	Daily minimum air temperature. Stored in `tmp(i)%ts2(jday, year_index)`.

Records are read sequentially. The reader detects the year change after `jday = 365` or `366` and increments its internal year index.

While reading, the routine accumulates monthly sums into `tmp(i)%max_mon` and `tmp(i)%min_mon` (per the `xmon` call) and at end of file divides by the total day count to get the average monthly maximum and minimum. A `-99` flag is treated like any other value at this stage, so missing-data flags carry into the monthly statistics unless the file is clean over the simulation window.

Example

First lines of `refdata/Ames_sub1/ames.tem` (the temperature time series for the Ames reference dataset; the same record layout applies to a `*.tmp` file):

```
AME.Tmp
 NBYR    TSTEP      LAT     LONG     ELEV
   121        0    42.04   -93.89   316
1900    1   -99 -99
1900    2   -99 -99
...
1983    185 32.2    10.6
1983    186 26.7    11.1
```

Station header: 121 years, time step `0`, latitude 42.04, longitude -93.89, elevation 316 m. Early years carry the `-99` missing-data flag for both max and min.

Related files

- `tmp.cli`. Lists each temperature gauge filename.
- `weather-sta.cli`. Binds a temperature gauge to a weather station via `tgage`.
- `file.cio`. Names the `tmp.cli` index and the optional path prefix `in_path_tmp%tmp` applied to every temperature filename.
- `*.tem`. Same record layout, alternate filename extension used by some toolchains.

Purpose

A `*.tem` file is a temperature time-series file with the same record layout as a *.tmp. The `.tem` extension is used by the reference datasets shipped with SWAT+ (for example `refdata/Ames_sub1/ames.tem`) and by some external preprocessors. There is no separate reader for `.tem`. The temperature reader at `src/cli_tmeas.f90` opens whatever filename is supplied; the extension is not inspected.

Source

- Reader: src/cli_tmeas.f90
- Type definition: src/climate_module.f90, `type climate_measured_data`

The reader is reached only when the filename appears in tmp.cli. The `tgage` slot in weather-sta.cli is matched against the filename listed in `tmp.cli` via the `tmp_n` lookup array, so both files must agree on the string (including the `.tem` extension if used).

Format

Identical to a daily `*.tmp` file. List-directed reads, whitespace-separated.

- Line 1: title (skipped).
- Line 2: column header (skipped). Conventionally `NBYR TSTEP LAT LONG ELEV`.
- Line 3: station header (`nbyr`, `tstep`, `lat`, `long`, `elev`).
- Line 4 onward: one record per day with year, Julian day, daily maximum temperature, daily minimum temperature.

See *.tmp for the field tables.

Example

`refdata/Ames_sub1/ames.tem` (excerpt):

```
AME.Tmp
 NBYR    TSTEP      LAT     LONG     ELEV
   121        0    42.04   -93.89   316
1900    1   -99 -99
1900    2   -99 -99
...
1983    185 32.2    10.6
1983    186 26.7    11.1
```

Related files

- *.tmp. Same content, alternate extension. The reader is shared.
- tmp.cli. Lists each temperature filename (with whatever extension).
- weather-sta.cli. The `tgage` column must match a filename entry in `tmp.cli`.

Purpose

`snow.sno` is the snow parameter database. Each row defines a named set of snow accumulation and melt parameters. HRUs reference a row by name through the `snow` column of hru-data.hru, which the reader resolves to an index into the `snodb` array. If `hru-data.hru` carries `null` for an HRU's snow entry, that HRU uses the default `snow_parameters` values.

Source

- Reader: src/snowdb_read.f90
- Type definition: src/hru_module.f90, `type snow_parameters` (the `snodb` array)
- Filename slot: `in_parmdb%snow` in src/input_file_module.f90 (default `snow.sno`).

Format

List-directed reads (free format, whitespace-separated). The first two lines are skipped. From line three to end of file each line is one snow parameter row. The reader counts records on a first pass, allocates `snodb(0:imax)`, then reads each row into `snodb(isno)`.

- Line 1: title (skipped). Editor convention: `snow.sno: written by SWAT+ editor ...`.
- Line 2: column header (skipped). Editor convention: `name fall_tmp melt_tmp melt_max melt_min tmp_lag snow_h2o cov50 snow_init`.
- Line 3+: one row per parameter set, 9 fields each.

The fields appear in the order declared in `type snow_parameters`:

#	Field	Type	Units	Default	Description
1	name	char(40)	-	(none)	Parameter set name. Referenced from the `snow` column of `hru-data.hru`.
2	falltmp	real	deg C	0.0	Snowfall temperature. Precipitation falls as snow when the mean air temperature is at or below this threshold.
3	melttmp	real	deg C	0.5	Snowmelt base temperature. Snowmelt only occurs when the snowpack temperature exceeds this value.
4	meltmx	real	mm/deg C/day	4.5	Maximum melt rate factor for snow during the year (occurs near 21 June).
5	meltmn	real	mm/deg C/day	0.5	Minimum melt rate factor for snow during the year (occurs near 21 December).
6	timp	real	none	0.8	Snowpack temperature lag factor (0 to 1). Higher values give more weight to the current day's air temperature.
7	covmx	real	mm H2O	25.0	Snow water content above which the ground is fully covered by snow.
8	cov50	real	none	0.5	Fraction of `covmx` at which the ground is 50% snow-covered. Shape factor for the snowpack areal depletion curve.
9	init_mm	real	mm H2O	0.0	Initial snow water content at the start of the simulation.

The defaults above are the values assigned in `type snow_parameters` at declaration. They are not values the reader supplies, so every row of `snow.sno` must list all nine fields.

Example

`refdata/Ames_sub1/snow.sno`:

```
snow.sno: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                  fall_tmp      melt_tmp      melt_max      melt_min       tmp_lag      snow_h2o
snow001                1.00000       0.50000       4.50000       4.50000       1.00000       1.00000
```

One parameter set named `snow001`. Note that the example value for `meltmn` (column 5) is `4.5`, identical to `meltmx`. The defaults shipped in source set `meltmn = 0.5`.

Related files

- hru-data.hru. References a `snow.sno` row by name in its `snow` column. `null` selects the type defaults.
- file.cio. Names the `snow.sno` file in the `init` group via `in_parmdb%snow`. Set to `null` to skip reading (no snow database is loaded; HRUs that point at a snow name will fall back as described above).

Purpose

`soils.sol` defines every soil class used in the watershed. Each class has a profile header (name, number of layers, hydrologic group, rooting depth, anion exclusion fraction, crack volume, texture string) and one row per soil layer describing depth, bulk density, available water capacity, saturated hydraulic conductivity, organic carbon, particle-size fractions, rock content, albedo, USLE K, electrical conductivity, calcium carbonate, and pH. HRUs reference these classes through `hru-data.hru`. Values feed `soil_phys_init`, `soil_nutcarb_init`, runoff, percolation, erosion, and plant uptake routines.

Source

- Reader: src/soil_db_read.f90
- Type definition: src/soil_data_module.f90, `type soil_database` (which holds `type soil_profile_db` plus an allocatable array of `type soilayer_db`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Each soil is a multi-record block:
  - One profile record with 7 fields.
  - `nly` layer records that follow, each with 14 fields. `nly` is read from the second field of the profile record and used to allocate the layer array.
- Records are list-directed (free format). Column positions in the reference example are cosmetic indentation only; the reader does not depend on them.

Profile record (7 fields)

Read into `soildb(i)%s` (`type soil_profile_db`).

#	Field	Type	Units	Default	Description
1	snam	char(20)	none	blank	soil class name. Referenced from `hru-data.hru`
2	nly	int	none	1	number of soil layers that follow this record
3	hydgrp	char(16)	none	“A”	hydrologic soil group (A, B, C, or D)
4	zmx	real	mm	1500.0	maximum rooting depth
5	anion_excl	real	fraction	0.5	fraction of porosity from which anions are excluded
6	crk	real	fraction	0.01	crack volume potential of the soil
7	texture	char(16)	none	blank	texture descriptor string (free-form label)

Layer record (14 fields, repeated `nly` times)

Read into `soildb(i)%ly(j)` (`type soilayer_db`) for `j = 1..nly`.

#	Field	Type	Units	Default	Description
1	z	real	mm	1500.0	depth from soil surface to the bottom of the layer
2	bd	real	Mg/m^3	1.3	moist bulk density
3	awc	real	mm H2O / mm soil	0.2	available water capacity
4	k	real	mm/hr	10.0	saturated hydraulic conductivity
5	cbn	real	%	2.0	organic carbon content of the layer
6	clay	real	%	10.0	clay fraction of mineral material
7	silt	real	%	60.0	silt fraction of mineral material
8	sand	real	%	30.0	sand fraction of mineral material
9	rock	real	%	0.0	rock fragment content of the layer
10	alb	real	none	0.1	moist soil albedo
11	usle_k	real	none	0.2	USLE soil erodibility (K) factor
12	ec	real	dS/m	0.0	electrical conductivity
13	cal	real	%	0.0	CaCO3 content
14	ph	real	none	0.0	soil pH

The reader applies one fix-up after reading: if the first layer is thinner than 20 mm and either the profile has only one layer or the second layer is deeper than 20 mm, the depth of layer 1 is forced to 20 mm.

Example

Excerpt from `refdata/Ames_sub1/soils.sol`. The first soil class has four layers:

```
soils.sol Ames
   name                NLY  HYD_GRP        ZMX     ANION_EXCL      CRK       TEXTURE        DEPTH         BD
soil_01                  4        B   2000.000          0.500    0.500       L- L- L- L
                                                                                             50.00      1.260
                                                                                            150.00      1.420
                                                                                            840.00      1.500
                                                                                           2000.00      1.600
```

Notes:

- The profile record `soil_01 4 B 2000.000 0.500 0.500 "L- L- L- L"` declares 4 layers. The next four physical lines are the layer records.
- Each layer record has 14 numeric values. The `TEXTURE` token on the profile line (`L- L- L- L` here) is one space-separated token only because the reader reads `texture` as a single character field; the writer pads it with hyphen-separated horizon codes.
- Header tokens in the file (`DEPTH`, `BD`, `AWC`, `K`, `CBN`, `CLAY`, `SILT`, `SAND`, `ROCK`, `ALB`, `USLE_K`, `EC`, `CAL`, `PH`) are read past as a single skipped line. The column order is fixed by the source.

Related files

- `hru-data.hru`. Each HRU points to a soil class name in `soils.sol`.
- `nutrients.sol`. Soil chemistry initialization paired with a soil class.
- `plants.plt`. Rooting depth in `soils.sol` (`zmx`) caps plant root depth.

Purpose

`landuse.lum` is the master table of land use and management classes. Each row defines one class by name and lists string pointers to records in other files: the plant community (`plants.ini`), management schedule (`management.sch`), curve number entry (`cntable.lum`), conservation practice (`cons_practice.lum`), urban land use (`urban.urb`), overland-flow Manning's n class (`ovn_table.lum`), tile drainage (`tiledrain.str`), septic (`septic.str`), filter strip (`filterstrip.str`), grassed waterway (`grassedww.str`), and user BMP (`bmpuser.str`). HRUs reference a row in this table through `hru-data.hru`. After reading, the reader resolves every non-`null` string pointer to an integer index stored in `lum_str`; unresolved names are logged to `simulation.out` (unit 9001).

Source

- Reader: `src/landuse_read.f90`
- Type definition: `src/landuse_data_module.f90`, `type land_use_management` (instances stored in `lum`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per land use class. Records are list-directed (free format). All 14 fields are read on every row.
- Any pointer field that should be empty must be the literal string `null`. The reader treats `null` as "no link" and does not try to resolve it.

#	Field	Type	Default	Description
1	name	char(40)	blank	name of the land use and management class. Referenced from `hru-data.hru`
2	cal_group	char(40)	blank	calibration group label. Not currently used by the solver
3	plant_cov	char(40)	blank	plant community pointer into `plants.ini`. Resolved to `lum_str%plant_cov`
4	mgt_ops	char(40)	blank	management schedule pointer into `management.sch`. Resolved to `lum_str%mgt_ops`
5	cn_lu	char(40)	blank	curve number entry pointer into `cntable.lum`. Resolved to `lum_str%cn_lu`
6	cons_prac	char(40)	blank	conservation practice pointer into `cons_practice.lum`. Resolved to `lum_str%cons_prac`
7	urb_lu	char(40)	blank	urban land use pointer into `urban.urb` (e.g. `URLD`, `URHD`). Resolved during urban init
8	urb_ro	char(40)	blank	urban runoff model. `usgs_reg` for USGS regression equations, `buildup_washoff` for build-up/wash-off
9	ovn	char(40)	blank	overland-flow Manning's n class pointer into `ovn_table.lum`
10	tiledrain	char(40)	blank	tile drainage pointer into `tiledrain.str`. Resolved to `lum_str%tiledrain`
11	septic	char(40)	blank	septic pointer into `septic.str`. Resolved to `lum_str%septic`
12	fstrip	char(40)	blank	filter strip pointer into `filterstrip.str`. Resolved to `lum_str%fstrip`
13	grassww	char(40)	blank	grassed waterway pointer into `grassedww.str`. Resolved to `lum_str%grassww`
14	bmpuser	char(40)	blank	user-defined BMP pointer into `bmpuser.str`. Resolved to `lum_str%bmpuser`

The reader logs a line to `simulation.out` for every non-`null` pointer that fails to match a name in the target file. The simulation continues with the link unresolved (index zero), which means the corresponding behaviour is skipped.

Example

Excerpt from `refdata/Ames_sub1/landuse.lum`:

```
landuse.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                          cal_group          plnt_com                                  mgt
cosy_lum                           null         cosy_comm                              mgt_01
agrr_lum                           null         agrr_comm                           AGRR_rot
frst_lum                           null         frst_comm                              null
urld_lum                           null              null                              null
```

Notes:

- The header tokens written by SWAT+ Editor (`plnt_com`, `cn2`, `urban`, `ov_mann`, `tile`, `sep`, `vfs`, `grww`, `bmp`) do not match the field names in the source (`plant_cov`, `cn_lu`, `urb_lu`, `ovn`, `tiledrain`, `septic`, `fstrip`, `grassww`, `bmpuser`). The reader skips the header and uses position only.
- `urld_lum` is an urban class. Its `plant_cov` and `mgt_ops` are both `null`; the row links instead to `URLD` in `urban.urb` and selects `buildup_washoff` as the urban runoff model.
- `frst_lum` has no management schedule (`mgt_ops = null`). Forest classes typically rely on the plant community initialization alone.

Related files

- hru-data.hru. Each HRU points to one `landuse.lum` row by name.
- cntable.lum. Curve number table referenced by the `cn_lu` field. Each row holds a name and four curve numbers (hydrologic conditions A through D).
- cons_practice.lum. Conservation practice table referenced by `cons_prac`. Holds USLE P factor and maximum slope length per practice.
- ovn_table.lum. Overland-flow Manning's n table referenced by `ovn`. Holds mean, minimum, and maximum n per class.
- plants.ini. Target of `plant_cov`.
- management.sch. Target of `mgt_ops`.
- `urban.urb`. Target of `urb_lu`.
- `tiledrain.str`, `septic.str`, `filterstrip.str`, `grassedww.str`, `bmpuser.str`. Structural BMP files targeted by the last five pointer columns.

Three lookup files supply numeric parameters that landuse.lum references by name: `cntable.lum` (SCS curve numbers), `cons_practice.lum` (USLE P factor and maximum slope length), and `ovn_table.lum` (overland-flow Manning's n). Each file has the same shape: a title line, a header line, then one record per row. Rows are read with list-directed I/O, so any column after the last typed field is ignored by the solver.

cntable.lum

Curve number table. Each entry holds four SCS-CN values for hydrologic soil groups A, B, C, and D.

Source

- Reader: src/cntbl_read.f90
- Type: src/landuse_data_module.f90, `type curvenumber_table` (instances stored in `cn`)
- Sized into `db_mx%cn_lu` after the file is read.

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per curve number entry. Fields are list-directed.

#	Field	Type	Default	Description
1	name	char(40)	blank	curve number entry name. Referenced from `landuse.lum` field `cn_lu`
2	cn(1)	real	30.0	curve number for hydrologic soil group A
3	cn(2)	real	55.0	curve number for hydrologic soil group B
4	cn(3)	real	70.0	curve number for hydrologic soil group C
5	cn(4)	real	77.0	curve number for hydrologic soil group D

Curve numbers are dimensionless and bounded by 0 and 100. SWAT+ Editor writes additional `description`, `treat`, and `cond_cov` columns after `cn(4)`. The reader does not consume them.

Example

Excerpt from `refdata/Ames_sub1/cntable.lum`:

```
cntable.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                      cn_a          cn_b          cn_c          cn_d  description
fal_bare               77.00000      86.00000      91.00000      94.00000  Fallow
rc_strow_g             67.00000      78.00000      85.00000      89.00000  Row_crops
wood_f                 36.00000      60.00000      73.00000      79.00000  Woods
urban                  98.00000      98.00000      98.00000      98.00000  Paved_parking_lots_roofs_drivewa
```

cons_practice.lum

Conservation practice table. Supplies the USLE support practice factor (P) and the maximum slope length used for length-slope factor calculations.

Source

- Reader: src/cons_prac_read.f90
- Type: src/landuse_data_module.f90, `type conservation_practice_table` (instances stored in `cons_prac`)
- Sized into `db_mx%cons_prac` after the file is read.

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per practice. Fields are list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	blank	-	practice name. Referenced from `landuse.lum` field `cons_prac`
2	pfac	real	1.0	-	USLE support practice factor (P). Dimensionless, 0 to 1
3	sl_len_mx	real	1.0	m	maximum slope length for the practice

SWAT+ Editor writes a `description` column after `sl_len_mx`. The reader does not consume it.

Example

Excerpt from `refdata/Ames_sub1/cons_practice.lum`:

```
cons_practice.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                     usle_p   slp_len_max  description
up_down_slope          1.00000     121.00000  Up_and_down_slope
cross_slope            0.75000     121.00000  Cross_slope_tillage
contour_farming        0.50000     121.00000  Contour_tillage
strip_contour          0.25000     121.00000  Strip_cropping_contour
ter_3-8_sodout         0.50000      76.00000  terraces_3-8%_slopes_sod-outlet
```

ovn_table.lum

Overland-flow Manning's n table. Supplies a mean, minimum, and maximum n per land use class for sheet-flow routing across the HRU.

Source

- Reader: src/overland_n_read.f90
- Type: src/landuse_data_module.f90, `type overlandflow_n_table` (instances stored in `overland_n`)
- Sized into `db_mx%ovn` after the file is read.

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per class. Fields are list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	blank	-	class name. Referenced from `landuse.lum` field `ovn`
2	ovn	real	0.5	s m^(-1/3)	mean Manning's n for overland flow
3	ovn_min	real	0.5	s m^(-1/3)	minimum Manning's n for overland flow
4	ovn_max	real	0.5	s m^(-1/3)	maximum Manning's n for overland flow

SWAT+ Editor writes a `description` column after `ovn_max`. The reader does not consume it.

Example

Excerpt from `refdata/Ames_sub1/ovn_table.lum`:

```
ovn_table.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                  ovn_mean       ovn_min       ovn_max  description
convtill_nores         0.09000       0.06000       0.12000  Conventional_tillage_no_residue
convtill_res           0.19000       0.16000       0.22000  Conventional_tillage_residue
notill_2-9res          0.30000       0.17000       0.47000  No_till_2-9_t/ha_residue
forest_med             0.60000       0.50000       0.70000  Forest_medimum_good
urban_asphalt          0.01100       0.01100       0.01100  Urban_asphalt
```

Related files

- landuse.lum. Master land-use management table. Its `cn_lu`, `cons_prac`, and `ovn` columns name rows in these three files.
- hru-data.hru. Selects a `landuse.lum` row per HRU, which in turn selects rows in these three tables.

Purpose

This page documents the legacy channel input files: `channel.cha` (original reach pointer), `channel-lte.cha` (SWAT+ deg routing pointer), `hyd-sed-lte.cha` (combined hydrology and sediment table used by the deg routine), `initial.cha` (initial-state pointer), and `temperature.cha` (water temperature parameters). All file names are set in `type input_cha` in `src/input_file_module.f90`. Each file is independently optional: if the slot in `file.cio` is `null` or the file is missing, the reader allocates an empty array and skips on.

The three companion tables that these pointer files reference (`hydrology.cha`, `sediment.cha`, `nutrients.cha`) are documented on the sd_channel page.

Source

- File names: src/input_file_module.f90, `type input_cha`
- Readers:
  - src/ch_read.f90 (`channel.cha`)
  - src/sd_channel_read.f90 (`channel-lte.cha`)
  - src/sd_hydsed_read.f90 (`hyd-sed-lte.cha`)
  - src/ch_read_init.f90 (`initial.cha`)
  - src/ch_read_temp.f90 (`temperature.cha`)
- Type definitions: src/channel_data_module.f90, src/sd_channel_module.f90

Every reader skips two header lines, then reads one record per channel definition.

channel.cha

Pointer file for the original SWAT-style reach routing. Each record names a channel and points to entries in `initial.cha`, `hydrology.cha`, `sediment.cha`, and `nutrients.cha`. Read into `ch_dat_c` (`type channel_data_char_input`).

#	Column	Type	Description
1	id	int	record number (echoed back)
2	name	char(16)	channel name
3	init	char(16)	name of an `initial.cha` entry, or `null`
4	hyd	char(16)	name of a `hydrology.cha` entry, or `null`
5	sed	char(16)	name of a `sediment.cha` entry, or `null`

#	Column	Type	Description
6	nut	char(16)	name of a `nutrients.cha` entry, or `null`

After reading, the reader resolves each character pointer to an integer index by scanning `ch_init`, `ch_hyd`, `ch_sed`, and `ch_nut`. Missing names are written to the diagnostics file (unit 9001) with messages like `"<name> not found (hydrology.cha)"`. The reader does not abort; the index is left at 0 and downstream code uses the default record.

No example dataset shipping with SWAT+ uses `channel.cha`. The two reference datasets at `refdata/Osu_1hru` and (if present) `refdata/Ames_sub1` configure this slot as `null` in `file.cio` and use `channel-lte.cha` instead.

channel-lte.cha

Pointer file for SWAT+ deg routing. Listed under `chandeg.con`. Each record points to entries in `initial.cha`, `hyd-sed-lte.cha`, an optional sediment table, and `nutrients.cha`. Read into `sd_dat` (`type swatdeg_datafiles`).

#	Column	Type	Description
1	id	int	record number used to position the array element
2	name	char(16)	channel name
3	cha_ini	char(16)	name of an `initial.cha` entry, or `null`
4	cha_hyd	char(16)	name of a `hyd-sed-lte.cha` entry, or `null`
5	cha_sed	char(16)	name of a sediment-nutrient entry (resolved against `sed_nut.cha` when present)
6	cha_nut	char(16)	name of a `nutrients.cha` entry, or `null`

The `cha_hyd` column is matched against names in both `sd_chd` (`hyd-sed-lte.cha`) and `sd_chd1` (`sed_nut.cha`). The `cha_sed` column is read but not resolved to an index by `sd_channel_read`; sediment behaviour follows the `hyd-sed-lte.cha` parameters.

Example from `refdata/Osu_1hru/channel-lte.cha`:

```
channel-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                        cha_ini           cha_hyd           cha_sed           cha_nut
       1  cha01                      initcha1          hydcha01             null           nutcha1
       2  cha03                      initcha1          hydcha03             null           nutcha1
       3  cha04                      initcha1          hydcha04             null           nutcha1
```

hyd-sed-lte.cha

Combined hydrology and bank-erosion parameters for the SWAT+ deg routing routine. One record per channel hydrology profile. Read into `sd_chd` (`type swatdeg_hydsed_data`).

#	Column	Type	Units	Default	Description
1	name	char(25)	none	“”	parameter set name (referenced by `channel-lte.cha`)
2	order	int	none	0	stream order
3	chw	real	m	0.0	channel width
4	chd	real	m	0.0	channel depth
5	chs	real	m/m	0.0	channel slope
6	chl	real	km	0.0	channel length
7	chn	real	none	0.0	channel Manning’s n
8	chk	real	mm/hr	0.0	channel bottom hydraulic conductivity
9	bank_exp	real	none	0.0	bank erosion exponent (column `erod_fact` in editor output)
10	cov	real	0..1	0.0	channel cover factor
11	sinu	real	none	0.0	sinuosity (channel length / straight-line length)
12	vcr_coef	real	none	0.0	critical velocity coefficient (column `eq_slp` in editor output)
13	d50	real	mm	0.0	median sediment diameter
14	ch_clay	real	percent	0.0	clay percent of bank and bed
15	carbon	real	percent	0.0	carbon percent of bank and bed
16	ch_bd	real	t/m^3	0.0	dry bulk density
17	chss	real	none	0.0	channel side slope
18	bankfull_flo	real	m^3/s	0.0	bank-full flow rate (column `bed_load` in editor output)
19	fps	real	m/m	0.000001	flood plain slope
20	fpn	real	none	0.1	flood plain Manning’s n
21	n_conc	real	mg/kg	0.0	nitrogen concentration in channel bank
22	p_conc	real	mg/kg	0.0	phosphorus concentration in channel bank
23	p_bio	real	fraction	0.0	fraction of bank P that is bioavailable

Example from `refdata/Osu_1hru/hyd-sed-lte.cha`:

```
hyd-sed-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                          order            wd            dp           slp           len          mann
hydcha01                          4      27.17726       0.99164       0.00146       3.42760       0.05000
```

Editor column names differ from source field names. The reader uses position only.

initial.cha

Initial state pointer file. Each record names a set of initial-condition files for organic-mineral mass, pesticides, pathogens, heavy metals, and salts. Read into `ch_init` (`type channel_init_datafiles`).

#	Column	Type	Description
1	name	char(16)	initial-state set name
2	org_min	char(16)	initial organic-mineral input file name, or `null`
3	pest	char(16)	initial pesticide input file name, or `null`
4	path	char(16)	initial pathogen input file name, or `null`
5	hmet	char(16)	initial heavy metals input file name, or `null`
6	salt	char(16)	initial salt input file name, or `null`

The reader uses unformatted list-directed reads with no leading integer id column. The record is read directly into the derived type, so the first token on each line is the `name`.

Example from `refdata/Osu_1hru/initial.cha`:

```
initial.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        org_min             pest             path             hmet             salt
initcha1                    no_init             null             null             null             null
```

A separate file `initial.cha_cs` is read by `ch_read_init_cs` for salt and conservative-species initial conditions. The file name is hard-coded in the reader (not in `type input_cha`).

temperature.cha

Water temperature parameters used by the channel temperature routine. Read into `w_temp` (`type water_temperature_data`). The `name` field is `character(len=13)` in the source; longer names will be truncated.

#	Column	Type	Units	Default	Description
1	name	char(13)	none	“”	parameter set name
2	sno_mlt	real	none	1.0	coefficient on snowmelt temperature contribution
3	gw	real	none	1.0	coefficient on groundwater temperature contribution
4	sur_lat	real	none	1.0	coefficient on surface and lateral flow temperature contribution
5	sno_lag	real	days	2.0	air temperature lag to snowmelt (1..3)
6	gw_lag	real	days	250.0	air temperature lag to groundwater flow (200..365)
7	surf_lag	real	days	5.0	air temperature lag to surface runoff (2..5)
8	lat_lag	real	days	10.0	air temperature lag to lateral flow (5..10)
9	lat_lag_coef	real	none	0.75	lateral air-lag coefficient
10	surf_lag_coef	real	none	0.75	surface air-lag coefficient (also used for snow)
11	gw_lag_coef	real	none	0.75	groundwater air-lag coefficient
12	hex_coef1	real	none	0.75	dew point calibration
13	hex_coef2	real	none	1.5	channel geometry calibration
14	sf_on	int	none	0	shade factor file activation. `1` = read from file, `0` = use value from calibration file
15	ssff	real	0..1	0.5	shade factor value (used when `sf_on = 0`)

No example dataset in `refdata/` contains `temperature.cha`. The slot is `null` in `file.cio` for the included reference datasets.

Related files

- `file.cio`. The `channel` line selects which of these files are read.
- `chandeg.con`. Connectivity file that references `channel-lte.cha` entries by name.
- `channel.con`. Connectivity file that references `channel.cha` entries by name.
- sd_channel companion tables (`hydrology.cha`, `sediment.cha`, `nutrients.cha`).

Purpose

This page documents the three per-channel parameter tables that hold hydraulic, sediment, and nutrient coefficients used inside the channel routing modules: `hydrology.cha`, `sediment.cha`, and `nutrients.cha`. Each is a flat database. Records are referenced by name from the pointer files `channel.cha` (legacy) and `channel-lte.cha` (SWAT+ deg). When the SWAT+ deg routine routes water and constituents through reaches listed under `chandeg.con`, the parameters in these tables drive the in-channel processes.

File names are declared in `type input_cha` in `src/input_file_module.f90` (`hyd`, `sed`, `nut`). Each file is optional. If the slot in `file.cio` is `null` or the file is missing, the reader allocates an empty array and the matching pointer is left at index 0.

Source

- File names: `src/input_file_module.f90`, `type input_cha`
- Readers:
  - `src/ch_read_hyd.f90`
  - `src/ch_read_sed.f90`
  - `src/ch_read_nut.f90`
- Type definitions: `src/channel_data_module.f90`, types `channel_hyd_data`, `channel_sed_data`, `channel_nut_data`

Every reader skips two header lines, then reads one record per parameter set.

hydrology.cha

Hydraulic and bank parameters for the original SWAT-style reach routine. Read into `ch_hyd` (`type channel_hyd_data`). The reader rewrites a few fields after read: `alpha_bnk` is replaced with `exp(-alpha_bnk)`, `s` is floored at 0.0001, `n` is clamped to `[0.01, 0.70]`, `l` is floored at 0.0010, `wdr` defaults to 3.5 if non-positive, and `side` defaults to 2.0 if non-positive.

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	w	real	m	2.0	average width of main channel
3	d	real	m	0.5	average depth of main channel
4	s	real	m/m	0.01	average slope of main channel
5	l	real	km	0.1	main channel length in subbasin
6	n	real	none	0.05	Manning’s n
7	k	real	mm/hr	0.01	effective hydraulic conductivity of channel alluvium
8	wdr	real	m/m	6.0	channel width-to-depth ratio
9	alpha_bnk	real	days	0.03	alpha factor for bank storage recession (read as days, stored as `exp(-alpha_bnk)`)
10	side	real	none	0.0	channel side slope (horizontal per unit vertical)

No example dataset in `refdata/` ships with `hydrology.cha`. The included reference datasets configure this slot as `null` in `file.cio` and use `hyd-sed-lte.cha` instead.

sediment.cha

Channel sediment routing parameters. Read into `ch_sed` (`type channel_sed_data`). The 12 monthly `erod` values follow the scalar fields. The reader clamps several values: cover and erodibility factors are bounded by the routing equation choice, bank and bed `d50` default to 50 and 500 micrometers if non-positive, bank and bed bulk density default to 1.40 and 1.50 g/cc if non-positive, and bank and bed jet-test erodibility default to 0.2 (or `0.2 / sqrt(tc_*)` if a critical shear stress is given). If all 12 monthly `erod` values are effectively zero, the reader fills the monthly array with `cov1`.

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“”	parameter set name
2	eqn	int	none	0	sediment routing method. `0` original SWAT, `1` Bagnold, `2` Kodatie, `3` Molinas-Wu, `4` Yang
3	cov1	real	none	0.1	channel erodibility factor (0..1 when `eqn = 0`, 0..25 otherwise)
4	cov2	real	none	0.1	channel cover factor (same bounds as `cov1`)
5	bnk_bd	real	g/cc	0.0	bulk density of channel bank sediment (1.1..1.9)
6	bed_bd	real	g/cc	0.0	bulk density of channel bed sediment (1.1..1.9)
7	bnk_kd	real	cm^3/N/s	0.0	bank erodibility from jet test
8	bed_kd	real	cm^3/N/s	0.0	bed erodibility from jet test
9	bnk_d50	real	micrometers	0.0	bank median particle diameter (clamped to 10000 max)
10	bed_d50	real	micrometers	0.0	bed median particle diameter (clamped to 10000 max)
11	tc_bnk	real	N/m^2	0.0	critical shear stress of channel bank
12	tc_bed	real	N/m^2	0.0	critical shear stress of channel bed
13..24	erod(1..12)	real	none	0.0	monthly channel erodibility factor. `0.0` = non-erosive, `1.0` = no resistance to erosion

No example dataset in `refdata/` ships with `sediment.cha`.

nutrients.cha

In-stream nutrient cycling parameters. Read into `ch_nut` (`type channel_nut_data`). After read, several rate constants are scaled by the routing time step (`time%step`) so that values defined per day are converted to per-substep. The light half-saturation `k_l` is converted from kJ/(m^2 min) to MJ/(m^2 hr) by multiplying by 0.06. Defaults are applied (in place of zero) for every coefficient.

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“”	parameter set name
2	onco	real	ppm	0.0	channel organic N concentration
3	opco	real	ppm	0.0	channel organic P concentration
4	rs1	real	m/day	1.0	local algal settling rate at 20 deg C
5	rs2	real	(mg disP-P)/(m^2 day)	0.05	benthos source rate for dissolved P at 20 deg C
6	rs3	real	(mg NH4-N)/(m^2 day)	0.5	benthos source rate for ammonia at 20 deg C
7	rs4	real	1/day	0.05	organic N settling rate at 20 deg C
8	rs5	real	1/day	0.05	organic P settling rate at 20 deg C
9	rs6	real	1/day	2.5	settling rate for arbitrary non-conservative constituent
10	rs7	real	(mg ANC)/(m^2 day)	2.5	benthal source rate for arbitrary non-conservative constituent
11	rk1	real	1/day	1.71	CBOD deoxygenation rate at 20 deg C
12	rk2	real	1/day	1.0	reaeration rate (Fickian) at 20 deg C
13	rk3	real	1/day	2.0	CBOD loss rate due to settling at 20 deg C
14	rk4	real	(mg O2)/(m^2 day)	0.0	sediment oxygen demand at 20 deg C
15	rk5	real	1/day	1.71	coliform die-off rate
16	rk6	real	1/day	1.71	decay rate for arbitrary non-conservative constituent
17	bc1	real	1/hr	0.55	NH3 to NO2 biological oxidation rate at 20 deg C
18	bc2	real	1/hr	1.1	NO2 to NO3 biological oxidation rate at 20 deg C
19	bc3	real	1/hr	0.21	organic N to ammonia hydrolysis rate at 20 deg C
20	bc4	real	1/hr	0.35	organic P to dissolved P decay rate at 20 deg C
21	lao	real	none	2.0	QUAL2E light averaging option. Only option 2 is implemented in SWAT
22	igropt	int	none	2	algal growth limiting equation. 1 multiplicative, 2 limiting nutrient, 3 harmonic mean
23	ai0	real	ug chla / mg alg	50.0	ratio of chlorophyll-a to algal biomass
24	ai1	real	mg N / mg alg	0.08	nitrogen fraction of algal biomass
25	ai2	real	mg P / mg alg	0.015	phosphorus fraction of algal biomass
26	ai3	real	mg O2 / mg alg	1.60	rate of O2 production per unit of algal photosynthesis
27	ai4	real	mg O2 / mg alg	2.0	rate of O2 uptake per unit of algal respiration
28	ai5	real	mg O2 / mg N	3.5	rate of O2 uptake per unit of NH3 oxidation
29	ai6	real	mg O2 / mg N	1.07	rate of O2 uptake per unit of NO2 oxidation
30	mumax	real	1/hr	2.0	maximum specific algal growth rate at 20 deg C
31	rhoq	real	1/day	2.5	algal respiration rate
32	tfact	real	none	0.3	fraction of solar radiation that is photosynthetically active
33	k_l	real	kJ/(m^2 min) read, stored as MJ/(m^2 hr)	0.75	light half-saturation coefficient
34	k_n	real	mg N / L	0.02	Michaelis-Menten N half-saturation constant

#	Column	Type	Units	Default	Description
35	k_p	real	mg P / L	0.025	Michaelis-Menten P half-saturation constant
36	lambda0	real	1/m	1.0	non-algal portion of the light extinction coefficient
37	lambda1	real	1/(m * ug chla/L)	0.03	linear algal self-shading coefficient
38	lambda2	real	(1/m)(ug chla/L)^(-2/3)	0.054	nonlinear algal self-shading coefficient
39	p_n	real	none	0.5	algal preference factor for ammonia

Example from `refdata/Osu_1hru/nutrients.cha`:

```
nutrients.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      plt_n         ptl_p       alg_stl      ben_disp      ben_nh3n      ptln_stl      
nutcha1                 0.00000       0.00000       1.00000       0.05000       0.50000       0.05000
```

Editor column names (`plt_n`, `ptl_p`, `alg_stl`, …) do not match the source field names (`onco`, `opco`, `rs1`, …). The reader uses position only.

Notes on the Osu_1hru example:

- Position 11 in this dataset (`cbn_bod_co = 1.71`) is `rk1` in the source.
- Position 13 (`cbn_bod_stl = 0.36`) is `rk3`. The source default is 2.0; this dataset overrides it.
- Position 14 (`ben_bod = 2.0`) is `rk4`. The source default is 0.0; this dataset overrides it.
- Positions 21 and 22 (`q2e_lt = 2`, `q2e_alg = 2`) are `lao` (read as real) and `igropt` (read as int).

Related files

- `*.cha` legacy channel files (`channel.cha`, `channel-lte.cha`, `hyd-sed-lte.cha`, `initial.cha`, `temperature.cha`). The pointer files that select records from `hydrology.cha`, `sediment.cha`, and `nutrients.cha` by name.
- `chandeg.con`. Connectivity file for SWAT+ deg routing.
- `channel.con`. Connectivity file for the original SWAT-style routing.
- `file.cio`. The `channel` line lists the file names that occupy each slot in `type input_cha`.

Purpose

The aquifer group has two input files declared in `type input_aqu` (`src/input_file_module.f90`):

- `aquifer.aqu` defines one record per aquifer object. Each record sets baseflow recession parameters, initial water table depth, initial chemistry, and the recharge and revap controls used by the lumped (non-gwflow) aquifer module. Records are read into `aqudb` (an array of `type aquifer_database`).
- `initial.aqu` is a small table of named initial-condition pointers. Each entry points to organic-mineral, pesticide, pathogen, heavy-metal, and salt initialization files. The `init` column of each row in `aquifer.aqu` selects one of these names.

Source

- Readers: `src/aqu_read.f90`, `src/aqu_read_init.f90`
- Type definitions: `src/aquifer_module.f90`, `type aquifer_database` and `type aquifer_init_data_char`
- Filename declarations: `src/input_file_module.f90`, `type input_aqu`

aquifer.aqu

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per aquifer with list-directed I/O. The first integer on each record is the aquifer id; the reader uses it as the index into `aqudb`. The remaining columns are read into the fields of `type aquifer_database` in declaration order.

#	Field	Type	Units	Default	Description
1	id	int	none	-	aquifer id. Used as the array index into `aqudb`
2	aqunm	char(16)	none	“”	aquifer name
3	aqu_ini	char(16)	none	“”	initial-condition pointer. Cross-walks with `name` in `initial.aqu`
4	flo	real	mm	0.05	flow from aquifer in the current time step
5	dep_bot	real	m	0.0	depth from mid-slope surface to bottom of aquifer
6	dep_wt	real	m	0.0	initial depth from mid-slope surface to water table
7	no3	real	ppm N	0.0	initial nitrate-N concentration in aquifer
8	minp	real	ppm P	0.0	initial mineral phosphorus concentration in aquifer
9	cbn	real	percent	0.5	initial organic carbon in aquifer
10	flo_dist	real	m	50.0	average flow distance to stream or downstream object
11	bf_max	real	mm	0.0	maximum daily baseflow when all channels are contributing
12	alpha	real	1/day	0.0	lag factor for the groundwater recession curve
13	revap_co	real	none	0.0	revap coefficient. `evap = pet * revap_co`
14	seep	real	fraction	0.0	fraction of recharge that seeps from the aquifer to the deep store
15	spyld	real	$m^{3/m}3$	0.0	specific yield of the aquifer
16	hlife_n	real	days	30.0	half-life of nitrogen in groundwater
17	flo_min	real	m	0.0	water table depth above which return flow occurs
18	revap_min	real	m	0.0	water table depth above which revap occurs

Example

`refdata/Osu_1hru/aquifer.aqu`:

```
aquifer.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                          init        gw_flo       dep_bot        dep_wt         no3_n
       1  aqu011                     initaqu1       0.05000      10.00000       6.00000       0.00000
       2  aqu012                     initaqu1       0.05000      10.00000       6.00000       0.00000
      21  aqu_deep010                initaqu1       0.00000      10.00000      20.00000       0.00000
```

Notes:

- The tool header names (`gw_flo`, `no3_n`, `sol_p`, `carbon`, `alpha_bf`, `revap`, `rchg_dp`, `spec_yld`, `hl_no3n`) differ from the source field names (`flo`, `no3`, `minp`, `cbn`, `alpha`, `revap_co`, `seep`, `spyld`, `hlife_n`). Column order is what matters.
- The `init` column points to `initaqu1`, which must exist as a row in `initial.aqu`.
- In this dataset all shallow aquifers share the same property values. The last record (`aqu_deep010`) represents the deep aquifer with no baseflow (`flo = 0`, `flo_min = 0`, `revap_min = 0`).

initial.aqu

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per initial-condition set with list-directed I/O. Records are read into `aqu_init_dat_c` (an array of `type aquifer_init_data_char`).

#	Field	Type	Description
1	name	char(16)	initial set name. Cross-walks with `aqu_ini` in `aquifer.aqu`
2	org_min	char(16)	name of organic-mineral initial-state record
3	pest	char(16)	name of pesticide initial-state record
4	path	char(16)	name of pathogen initial-state record
5	hmet	char(16)	name of heavy-metal initial-state record
6	salt	char(16)	name of salt initial-state record

The literal `null` is accepted in slots that have no initial-state file.

Example

`refdata/Osu_1hru/initial.aqu`:

```
initial.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       org_min             pest             path             hmet             salt
initaqu1                   no_init             null             null             null             null
```

Notes:

- The tool writes a trailing `description` column. The source type has no `description` field. List-directed reads consume tokens left to right, so the extra trailing token is harmless. Do not rely on the description being read.
- `no_init` is itself a name that must resolve in the organic-mineral initial file. `null` is treated as "no file".

Related files

- aquifer.con. Connectivity entries that map each aquifer id to its spatial location and downstream object.
- `aqu_catunit.ele` and `aqu_catunit.def`. Catchment-unit elements that group aquifers.
- Organic-mineral, pesticide, pathogen, heavy-metal, and salt initial-state files referenced by `initial.aqu`.

Purpose

The reservoir group has six input files declared in `type input_res` (`src/input_file_module.f90`):

- `reservoir.res` is the object table. One record per reservoir, naming the hydrology, sediment, nutrient, release, and initial-condition records that apply.
- `initial.res` is a table of named initial-condition pointers (organic-mineral, pesticide, pathogen, heavy-metal, salt) shared with the wetland module.
- `hydrology.res` is the hydrology database. One record per named hydrology profile.
- `sediment.res` is the sediment-quality database.
- `nutrients.res` is the nutrient settling and loss-rate database.
- `weir.res` is the weir outflow database for reservoirs that release through a weir.

All five database files are referenced by name from `reservoir.res`. Names are not positional. The reader builds an index after all files are loaded.

Source

- Readers:
  - src/res_read.f90
  - src/res_read_init.f90
  - src/res_read_hyd.f90
  - src/res_read_sed.f90
  - src/res_read_nut.f90
  - src/res_read_weir.f90
- Type definitions: src/reservoir_data_module.f90
- Filename declarations: src/input_file_module.f90, `type input_res`

reservoir.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per reservoir with list-directed I/O. The first integer on each record is the reservoir id; the reader uses it as the index into `res_dat_c`. The remaining columns are read into `type reservoir_data_char_input` in declaration order.

#	Field	Type	Description
1	id	int	reservoir id. Used as the array index into `res_dat_c`
2	name	char(25)	reservoir name
3	init	char(25)	initial-state pointer. Cross-walks with `init` in `initial.res`
4	hyd	char(25)	hydrology pointer. Cross-walks with `name` in `hydrology.res`
5	release	char(25)	release rule pointer. Either a decision-table name in `res_rel.dtl`, or a name prefixed with `ctbl_` for a conditional table
6	sed	char(25)	sediment pointer. Cross-walks with `name` in `sediment.res`
7	nut	char(25)	nutrient pointer. Cross-walks with `name` in `nutrients.res`

The `release` lookup branches on the literal prefix `ctbl_`. If present, the name is matched against `ctbl(:)%name` and the release mode is set to `c`. Otherwise it is matched against `dtbl_res(:)%name` and the release mode is set to `d`.

initial.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per initial-condition set with list-directed I/O. Records are read into `res_init_dat_c` (an array of `type reservoir_init_data_char`).

#	Field	Type	Description
1	init	char(25)	initial set name. Cross-walks with `init` in `reservoir.res` (and `wetland.wet`)
2	org_min	char(25)	name of organic-mineral initial-state record
3	pest	char(25)	name of pesticide initial-state record
4	path	char(25)	name of pathogen initial-state record
5	hmet	char(25)	name of heavy-metal initial-state record
6	salt	char(25)	name of salt initial-state record

The literal `null` is accepted in slots that have no initial-state file.

Example

`refdata/Osu_1hru/initial.res`:

```
initial.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        org_min             pest             path             hmet             salt
initwet1                    no_init             null             null             null             null
```

The first column header in the example is `name` but the source field is `init` (the value it must match in `reservoir.res`). The source uses position. The editor also writes a trailing `description` column that is not read.

hydrology.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per hydrology profile with list-directed I/O. Records are read into `res_hyddb` (an array of `type reservoir_hyd_data`) in declaration order.

#	Field	Type	Units	Default	Description
1	name	char(25)	none	“default”	hydrology profile name
2	iyres	int	none	0	simulation year the reservoir becomes operational
3	mores	int	none	0	month the reservoir becomes operational
4	psa	real	ha	0.0	surface area when filled to principal spillway
5	pvol	real	ha-m	0.0	volume needed to fill to principal spillway. Converted to m^3 by `res_initial`
6	esa	real	ha	0.0	surface area when filled to emergency spillway
7	evol	real	ha-m	0.0	volume needed to fill to emergency spillway. Converted to m^3 by `res_initial`
8	k	real	mm/hr	0.01	hydraulic conductivity of the reservoir bottom
9	evrsv	real	none	0.7	lake evaporation coefficient
10	br1	real	none	0.0	volume-surface area coefficient (estimated by the model if zero)
11	br2	real	none	0.0	volume-surface area coefficient (estimated by the model if zero)

The reader fills in defaults when fields are zero or non-positive:

- If `pvol + evol > 0` and `pvol <= 0`, sets `pvol = 0.9 * evol`.
- If both are zero or negative, sets `pvol = 60000.0`.
- If `evol <= 0`, sets `evol = 1.11 * pvol`.
- If `psa <= 0`, sets `psa = 0.08 * pvol`.
- If `esa <= 0`, sets `esa = 1.5 * psa`.
- If `evrsv <= 0`, sets `evrsv = 0.6`.

Note the default for `evrsv` in the type definition is `0.7`, but the runtime default applied when the field is zero or negative is `0.6`.

sediment.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per sediment profile with list-directed I/O. Records are read into `res_sed` (an array of `type reservoir_sed_data`).

#	Field	Type	Units	Description
1	name	char(25)	none	sediment profile name
2	nsed	real	mg/L	normal amount of sediment in the reservoir
3	d50	real	um	median particle size of suspended and benthic sediment
4	carbon	real	percent	organic carbon in suspended and benthic sediment
5	bd	real	t/m^3	bulk density of benthic sediment
6	sed_stlr	real	none	sediment settling rate
7	velsetlr	real	m/day	sediment settling velocity

Example

`refdata/Osu_1hru/sediment.res`:

```
sediment.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    sed_amt           d50        carbon            bd       sed_stl       stl_vel
sedwet1                   10.00           5.00       0.04            0.8          0.15       0.0002
```

The editor writes `sed_amt`, `sed_stl`, `stl_vel`; the source fields are `nsed`, `sed_stlr`, `velsetlr`. Column order is what matters.

nutrients.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per nutrient profile with list-directed I/O. Records are read into `res_nut` (an array of `type reservoir_nut_data`). After reading, the reader divides the four `setlr` rates and the two `solr` rates by 365 (m/yr to m/day).

#	Field	Type	Units (file)	Description
1	name	char(25)	none	nutrient profile name
2	ires1	int	day of year	start of mid-year settling season
3	ires2	int	day of year	end of mid-year settling season
4	nsetlr1	real	m/year	nitrogen mass loss rate, mid-year period
5	nsetlr2	real	m/year	nitrogen mass loss rate, remainder of year
6	psetlr1	real	m/year	phosphorus mass loss rate, mid-year period
7	psetlr2	real	m/year	phosphorus mass loss rate, remainder of year
8	nsolr	real	m/year	loss rate for soluble nitrogen (NO3, NH3, NO2)
9	psolr	real	m/year	loss rate for soluble phosphorus
10	theta_n	real	none	temperature adjustment for nitrogen settling

#	Field	Type	Units (file)	Description
11	theta_p	real	none	temperature adjustment for phosphorus settling
12	conc_nmin	real	ppm	minimum nitrogen concentration for settling
13	conc_pmin	real	ppm	minimum phosphorus concentration for settling

Example

`refdata/Osu_1hru/nutrients.res`:

```
nutrients.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name               mid_start   mid_end     mid_n_stl         n_stl     mid_p_stl         p_stl       chla_c
nutwet1                    5        10       5.50000       5.50000      10.00000      10.00000       1.00000
```

Notes:

- Editor headers `mid_n_stl`, `n_stl`, `mid_p_stl`, `p_stl` correspond to source fields `nsetlr1`, `nsetlr2`, `psetlr1`, `psetlr2`.
- Editor headers `chla_co` and `secchi_co` correspond to source fields `nsolr` and `psolr` by position. These are the soluble nitrogen and phosphorus loss rates, not chlorophyll or Secchi-depth coefficients.

weir.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per weir profile with list-directed I/O. Records are read into `res_weir` (an array of `type reservoir_weir_outflow`).

#	Field	Type	Units	Default	Description
1	name	char(25)	none	“”	weir profile name
2	c	real	none	1.84	weir discharge linear coefficient
3	k	real	none	2.6	weir discharge exponential coefficient
4	w	real	m	2.5	weir width
5	h	real	m	0.0	height of the weir above the bottom of the impoundment

Example

`refdata/Osu_1hru/weir.res`:

```
weir.res: Reservoir weir inputs - asdf;lj
NAME   Linear_C    Exp_K     Width(m)   Depth(m)
weir1   1.83       1.50      5.00       0.0
```

Related files

- reservoir.con. Connectivity entries that place each reservoir in the routing topology.
- `res_rel.dtl`. Decision tables referenced from the `release` column of `reservoir.res`.
- wetland.wet. Wetlands share the `initial.res` table and the `hydrology.wet`, `sediment.res`, `nutrients.res`, and `weir.res` databases.

Purpose

The wetland group has two input files declared in `type input_res` (`src/input_file_module.f90`):

- `wetland.wet` is the wetland object table. One record per wetland, naming the initial-condition set, hydrology profile, release rule, sediment profile, nutrient profile, and weir profile that apply.
- `hydrology.wet` is the wetland hydrology database. One record per named hydrology profile. The geometry is HRU-relative (fraction of HRU area, depth in mm) rather than absolute area and volume as for reservoirs.

Wetlands reuse the reservoir sediment, nutrient, weir, and initial-condition databases. The wetland records in `wetland.wet` point at the same `sediment.res`, `nutrients.res`, `weir.res`, and `initial.res` tables described in reservoir.res.

Source

- Readers: src/wet_read.f90, src/wet_read_hyd.f90
- Type definitions: src/reservoir_data_module.f90, `type reservoir_data_char_input` (shared with reservoirs) and `type wetland_hyd_data`
- Filename declarations: src/input_file_module.f90, `type input_res`

wetland.wet

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per wetland with list-directed I/O. The first integer on each record is the wetland id (used as the index into `wet_dat_c`). The remaining columns are read into `type reservoir_data_char_input` in declaration order (the same type used by reservoirs).

#	Field	Type	Description
1	id	int	wetland id. Used as the array index into `wet_dat_c`
2	name	char(25)	wetland name
3	init	char(25)	initial-state pointer. Cross-walks with `init` in `initial.res`
4	hyd	char(25)	hydrology pointer. Cross-walks with `name` in `hydrology.wet`
5	release	char(25)	release rule pointer (decision-table name)
6	sed	char(25)	sediment pointer. Cross-walks with `name` in `sediment.res`
7	nut	char(25)	nutrient pointer. Cross-walks with `name` in `nutrients.res`

The example dataset writes a trailing `WEIR` column. The base type has six character fields (name, init, hyd, release, sed, nut). A weir pointer is held in the extension type `reservoir_data` (`character (len=25) :: weir`). The list-directed reader for `wetland.wet` does not consume a separate weir token into the base record; the source links wetland weirs through dedicated reservoir-conditions input. See the Important note below.

Example

`refdata/Osu_1hru/wetland.wet`:

```
wetland.       wet: written by SWAT+ edi#############for SWAT+ rev.60.5.4
     id                name         init          hyd          rel          sed          nut         WEIR
       1           paddy0001     initwet2        paddy         weir      sedwet1      nutwet1        weir1
```

Notes:

- The title line in this dataset is partially corrupted ("wetland. wet:"). The reader skips it as a single text line, so the content does not matter as long as it stays on one line.
- The `rel` column value `weir` is matched against decision-table names. If no match exists in `res_rel.dtl` (or in the `ctbl_` conditional tables), the wetland will be flagged as missing a release rule.

hydrology.wet

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per hydrology profile with list-directed I/O. Records are read into `wet_hyddb` (an array of `type wetland_hyd_data`) in declaration order.

#	Field	Type	Units	Default	Description
1	name	char(25)	none	“default”	hydrology profile name
2	psa	real	fraction	0.0	fraction of HRU area at the principal spillway. Surface-inlet riser flow starts at this level
3	pdep	real	mm	0.0	average water depth at the principal spillway
4	esa	real	fraction	0.0	fraction of HRU area at the emergency spillway. Overflow into the ditch starts at this level
5	edep	real	mm	0.0	average water depth at the emergency spillway
6	k	real	mm/hr	0.01	hydraulic conductivity of the wetland bottom
7	evrsv	real	none	0.7	wetland evaporation coefficient
8	acoef	real	none	1.0	volume-surface area coefficient for HRU impoundment
9	bcoef	real	none	1.0	volume-depth coefficient for HRU impoundment
10	ccoef	real	none	1.0	volume-depth coefficient for HRU impoundment
11	frac	real	none	0.5	fraction of HRU that drains into the impoundment

The reader fills defaults when fields are zero or non-positive:

- If `psa <= 0`, sets `psa = 0.08 * wet_hyd(ires)%pdep` (uses the previously initialized `wet_hyd`, not the just-read `wet_hyddb`).
- If `esa <= 0`, sets `esa = 1.5 * wet_hyd(ires)%psa`.
- If `evrsv <= 0`, sets `evrsv = 0.6`.

Example

`refdata/Osu_1hru/hydrology.wet`:

```
hydrology.wet: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                     hru_ps          dp_ps         hru_es          dp_es              k           evap    vol
paddy                      1.00         150.00          1.00         150.00           0.5            0.75
```

Editor headers `hru_ps`, `dp_ps`, `hru_es`, `dp_es`, `evap`, `vol_area_co`, `vol_dp_a`, `vol_dp_b`, `hru_frac` correspond to source fields `psa`, `pdep`, `esa`, `edep`, `evrsv`, `acoef`, `bcoef`, `ccoef`, `frac`. Column order is what matters.

Related files

- reservoir.res and initial.res. Shared tables. `wetland.wet` records point at `initial.res` names, and the sediment, nutrient, and weir lookups use the same databases as reservoirs.
- HRU connectivity. Each wetland is attached to an HRU; the geometry in `hydrology.wet` is expressed as fractions of HRU area.

Purpose

The six `.ops` files declared in `type input_ops` are parameter databases for the management operations that `management.sch` and the `lum.dtl` decision tables can invoke. Each file is a named list of parameter sets that other files reference by `name`.

All six readers follow the same structure: skip two header lines, count records, allocate, rewind, skip two, read one record per row into the corresponding `*_db` array.

Source

- Readers: `src/mgt_read_harvops.f90`, `src/mgt_read_grazeops.f90`, `src/mgt_read_irrops.f90`, `src/mgt_read_chemapp.f90`, `src/mgt_read_fireops.f90`, `src/mgt_read_sweepops.f90`.
- Type definitions: `src/mgt_operations_module.f90`.

File format

For every file:

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3 and following: one record per operation. List-directed read. The reader counts records on a first pass, allocates the database array, rewinds, and reads the values.

The "Common" first column is the operation name; other files reference it.

harv.ops

Reader: `mgt_read_harvops.f90` → `harvop_db` (`type harvest_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	typ	char(40)	“”	none	`grain`, `biomass`, `residue`, `tree`, or `tuber`
3	hi_ovr	real	0	none	harvest index override at harvest
4	eff	real	0	fraction	harvest efficiency. Fraction of harvested yield removed; the remainder is left as residue
5	bm_min	real	0	kg/ha	minimum biomass required to allow harvest

graze.ops

Reader: `mgt_read_grazeops.f90` → `grazeop_db` (`type grazing_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	fertnm	char(40)	“”	none	manure name pointing into `fertilizer.frt`
3	manure_id	int	0	none	resolved id of `fertnm` in `fertilizer.frt`
4	eat	real	0	(kg/ha)/day	dry biomass removed by grazing per day
5	tramp	real	0	(kg/ha)/day	dry biomass removed by trampling per day
6	manure	real	0	(kg/ha)/day	dry manure deposited per day
7	biomin	real	0	kg/ha	minimum plant biomass to allow grazing

irr.ops

Reader: `mgt_read_irrops.f90` → `irrop_db` (`type irrigation_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	amt_mm	real	25.4	mm	application amount
3	eff	real	0	none	in-field application efficiency
4	surq	real	0	fraction	surface runoff ratio
5	dep_mm	real	0	mm	depth of application for subsurface irrigation
6	salt	real	0	mg/kg	total salt concentration in irrigation water
7	no3	real	0	mg/kg	nitrate concentration
8	po4	real	0	mg/kg	phosphate concentration

chem_app.ops

Reader: `mgt_read_chemapp.f90` → `chemapp_db` (`type chemical_application_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	form	char(40)	“”	none	`solid` or `liquid`
3	op_typ	char(40)	“”	none	`spread`, `spray`, `inject`, or `direct`
4	app_eff	real	0	none	application efficiency
5	foliar_eff	real	0	none	foliar efficiency
6	inject_dep	real	0	mm	injection depth
7	surf_frac	real	0	fraction	fraction in upper 10 mm of soil
8	drift_pot	real	0	none	drift potential
9	aerial_unif	real	0	none	aerial uniformity

fire.ops

Reader: `mgt_read_fireops.f90` → `fire_db` (`type fire_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	cn2_upd	real	0	none	change applied to SCS curve number II value
3	fr_burn	real	0	fraction	fraction of cover burned

sweep.ops

Reader: `mgt_read_sweepops.f90` → `sweepop_db` (`type streetsweep_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	eff	real	0	none	removal efficiency of the sweeping operation
3	fr_curb	real	0	none	availability factor: fraction of curb length that is sweepable

Example

`refdata/Ames_sub1/harv.ops`:

```
harv.ops written by SWAT+ editor v2.3.3 on 2023-10-25
name                  typ                hi_ovr        eff        bm_min
harv_grain            grain              0.0           1.0        0.0
harv_residue          residue            0.0           1.0        0.0
```

The other five files follow the same shape with their own column sets.

Related files

- `landuse.lum` and `management.sch` reference operations by `name`.
- `*.dtl` decision tables can fire operations by `name`.
- `fertilizer.frt` provides the `manure_id` resolution target for `graze.ops`.

Purpose

A decision-table file holds one or more tables. Each table evaluates a set of conditions every time step and fires the matching action when an alternative is satisfied. SWAT+ uses decision tables to drive land-use management (planting, harvest, fertiliser, irrigation), reservoir releases, scenario land-use changes, and flow control.

The four decision-table files declared in `type input_condition` are:

- `lum.dtl`: land-use management actions.
- `res_rel.dtl`: reservoir release rules.
- `scen_lu.dtl`: scenario-driven land-use changes.
- `flo_con.dtl`: flow control rules.

All four use the same file format. They differ only in which decision-table array they fill (`dtbl_lum`, `dtbl_res`, `dtbl_scen`, `dtbl_flo`).

Source

- Readers: `src/dtbl_lum_read.f90`, `src/dtbl_flocon_read.f90` (and matching readers for reservoir-release and scenario tables).
- Type definitions: `src/conditional_module.f90`. Each table is a `decision_table` instance; its `cond`, `alt`, and `act` arrays hold conditions, alternatives, and actions.

File format

A single file holds `mdtbl` tables.

- Line 1: title (skipped).
- Line 2: `mdtbl`, the number of decision tables in the file.
- Line 3: blank line (skipped).

For each of the `mdtbl` tables:

- One header line (skipped).
- One name line: `name`, `conds`, `alts`, `acts` (table name, number of conditions, number of alternatives, number of actions).
- One header line for the conditions block (skipped).
- `conds` lines, one per condition. Each line names a conditional variable plus `alts` alternative thresholds. See Condition row below.
- One header line for the actions block (skipped).
- `acts` lines, one per action. See Action row below.

Condition row

Each condition row is read into a `condition_var` record with the following fields, then `alts` alternative thresholds are read into the `alt(ic, 1..alts)` array.

Fields on a condition row:

#	Field	Type	Description
1	var	char	conditional variable name (for example `land_use`, `plant_gro`, `phu_plant`, `soil_water`, `prob_unif`)
2	obj	char	object the variable applies to
3	obj_num	int	object number
4	lim_var	char	limit-variable name
5	lim_op	char	comparison operator
6	lim_const	real	comparison constant
7+	alt(1..alts)	char	one alternative threshold per alternative column

When `var == "prob_unif"`, the reader rereads the row and treats the second token as the probability fraction `frac_app` rather than the standard condition layout.

Action row

#	Field	Type	Description
1	act_typ	char	action type (for example `plant`, `harvest`, `fert`, `irrigate`, `lu_change`)
2	obj	char	target object
3	obj_num	int	target object number
4	name	char	action name pointing into the matching ops or input database
5	option	char	option tag
6	const	real	numeric constant for the action
7	const2	real	second numeric constant
8	fp	char	flag or pointer
9+	act_hit(1..alts)	char	one outcome flag per alternative column

Notes

- The reader allocates `cond(conds)`, `alt(conds, alts)`, `act(acts)`, and `act_hit(alts)` based on the header values. A mismatch between declared counts and actual rows truncates or overruns these arrays.
- After all tables are read, `lum.dtl` does a second pass to count HRUs whose `land_use_mgt_c` matches any `land_use` condition. The counts feed probabilistic operations.

Example

Top of `refdata/Ames_sub1/lum.dtl`:

```
lum.dtl: written by SWAT+ editor for SWAT+ rev.60.5.7
8

corn_grain
name                 conds   alts   acts
pl_hv_corn               2      2      2
var                   obj   obj_num   lim_var   lim_op   lim_const   alt1   alt2
plant_gro           plant         1   tree      >        0           y      n
phu_plant           plant         1                               >  1.0    n      y
act_typ             obj   obj_num   name                 option   const   const2   fp   outcome1   outcom
plant               hru   0         corn                 below    0       0        y    y          n
harvest_kill        hru   0         harv_grain_corn      now      0       0        y    n          y
```

Reading this:

- File holds 8 tables. The first is named `pl_hv_corn` with 2 conditions, 2 alternatives, 2 actions.
- Condition 1 checks `plant_gro` on plant object 1 against `tree > 0`. Alt 1 fires when `y`, alt 2 fires when `n`.
- Condition 2 checks `phu_plant > 1.0`. Alt 1 fires when `n`, alt 2 fires when `y`.
- Action 1: `plant` HRU operation named `corn` with option `below`. Fires under alt 1, not alt 2.
- Action 2: `harvest_kill` HRU operation named `harv_grain_corn` with option `now`. Fires under alt 2, not alt 1.

The exact column layout varies a little between datasets because the reader is fully list-directed.

Related files

- `landuse.lum`. Plant and management names referenced by actions.
- `*.ops`. Operation parameter databases that the actions resolve `name` against.
- `management.sch`. Fixed schedules used alongside decision tables.

Purpose

`plants.plt` holds one parameter set per plant species the model can grow. Each record fills a `plant_db` entry in the `pldb` array. Plant growth, phenology, biomass-to-energy conversion, root and canopy geometry, nutrient uptake, and residue partition fractions all come from this file. HRU plant communities (`plant.ini`) and management operations (`plant.ops`, `landuse.lum`) reference these records by name.

Source

- Reader: `src/plant_parm_read.f90`
- Type definition: `src/plant_data_module.f90`, `type plant_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per plant. List-directed read into `pldb(ic)` (`type plant_db`). The last sub-object is `res_part_fracs` (`type residue_partition_fracs`), which contributes three trailing fields.

The reader has a conditional branch:

```
if (bsn_cc%nam1 == 0) then
  read (104,*,iostat=eof) pldb(ic)
else
  read (104,*,iostat=eof) pldb(ic), pl_class(ic)
end if
```

`bsn_cc%nam1` is a switch in `codes.bsn`. With the default `nam1 = 0`, the standard column list below applies. With `nam1 /= 0`, one extra `pl_class` token (a 25-character name like `row_crop`, `tree`, `grass`) is read at the end of each record.

#	Field	Type	Units	Default	Description
1	plantnm	char(40)	none	“”	plant name (used as the key from other files)
2	typ	char(18)	none	“”	plant category: `warm_annual`, `cold_annual`, `warm_annual_tuber`, `cold_annual_tuber`, `perennial`
3	trig	char(18)	none	“”	phenology trigger: `moisture_gro` or `temp_gro`
4	nfix_co	real	none	0.0	nitrogen fixation coefficient (0.5 for legumes, 0 for non-legumes)
5	days_mat	int	days	110	days to maturity. If 0, heat units run the entire growing season
6	bio_e	real	(kg/ha)/(MJ/m^2)	15.0	biomass-energy ratio
7	hvsti	real	(kg/ha)/(kg/ha)	0.76	harvest index: crop yield over above-ground biomass
8	blai	real	none	5.0	maximum potential leaf area index
9	frgrw1	real	fraction	0.05	fraction of growing season at the 1st point on the optimal LAI curve
10	laimx1	real	fraction	0.05	fraction of maximum LAI at the 1st point on the optimal LAI curve
11	frgrw2	real	fraction	0.4	fraction of growing season at the 2nd point on the optimal LAI curve
12	laimx2	real	fraction	0.95	fraction of maximum LAI at the 2nd point on the optimal LAI curve
13	dlai	real	fraction	0.99	fraction of growing season when LAI starts to decline
14	dlai_rate	real	none	1.0	exponent governing LAI decline rate
15	chtmx	real	m	6.0	maximum canopy height
16	rdmx	real	m	3.5	maximum root depth
17	t_opt	real	deg C	30.0	optimal temperature for plant growth
18	t_base	real	deg C	10.0	minimum (base) temperature for plant growth
19	cnyld	real	kg N / kg yield	0.0015	fraction of nitrogen in yield
20	cpyld	real	kg P / kg yield	0.0003	fraction of phosphorus in yield
21	pltnfr1	real	kg N / kg biomass	0.006	nitrogen uptake parameter 1 (emergence)
22	pltnfr2	real	kg N / kg biomass	0.002	nitrogen uptake parameter 2 (mid-season)
23	pltnfr3	real	kg N / kg biomass	0.0015	nitrogen uptake parameter 3 (maturity)
24	pltpfr1	real	kg P / kg biomass	0.0007	phosphorus uptake parameter 1 (emergence)
25	pltpfr2	real	kg P / kg biomass	0.0004	phosphorus uptake parameter 2 (mid-season)
26	pltpfr3	real	kg P / kg biomass	0.0003	phosphorus uptake parameter 3 (maturity)
27	wsyf	real	(kg/ha)/(kg/ha)	0.01	lower bound of harvest index under water stress
28	usle_c	real	none	0.001	minimum value of the USLE C factor for water erosion
29	gsi	real	m/s	0.002	maximum stomatal conductance
30	vpdfr	real	kPa	4.0	vapor pressure deficit at which `gmaxfr` is valid
31	gmaxfr	real	none	0.75	fraction of maximum stomatal conductance achieved at `vpdfr`
32	wavp	real	none	8.0	rate of decline in radiation use efficiency per unit VPD

#	Field	Type	Units	Default	Description
33	co2hi	real	uL CO2 / L air	660.0	elevated CO2 concentration at the 2nd point on the RUE-CO2 curve
34	bioehi	real	(kg/ha)/(MJ/m^2)	16.0	biomass-energy ratio at the elevated CO2 level (`co2hi`)
35	rsdco_pl	real	none	0.05	plant residue decomposition coefficient
36	alai_min	real	$m^{2/m}2$	0.75	minimum LAI during winter dormant period
37	laixco_tree	real	none	0.3	coefficient used to estimate maximum LAI during tree growth
38	mat_yrs	int	years	10	years to maturity (forced to at least 1 by the reader)
39	bmx_peren	real	metric tons/ha	1000.0	maximum biomass for forest
40	ext_coef	real	none	0.65	light extinction coefficient
41	leaf_tov_min	real	months	12.0	perennial leaf turnover rate with minimum stress
42	leaf_tov_max	real	months	3.0	perennial leaf turnover rate with maximum stress
43	bm_dieoff	real	fraction	0.0	above-ground biomass that dies off at dormancy
44	rsr1	real	fraction	0.0	initial root-to-shoot ratio at the beginning of the growing season
45	rsr2	real	fraction	0.0	root-to-shoot ratio at the end of the growing season
46	pop1	real	plants/m^2	0.0	plant population at the 1st point on the population-LAI curve
47	frlai1	real	fraction	0.0	fraction of maximum LAI at the 1st point on the population-LAI curve
48	pop2	real	plants/m^2	0.0	plant population at the 2nd point on the population-LAI curve
49	frlai2	real	fraction	0.0	fraction of maximum LAI at the 2nd point on the population-LAI curve
50	frsw_gro	real	fraction	0.5	30-day sum of P-PET that triggers monsoon-season growth in tropical plants
51	aeration	real	none	0.2	aeration stress factor
52	rsd_pctcov	real	none	0.0	residue factor for the percent cover equation
53	rsd_covfac	real	none	0.0	residue factor for the surface cover (C factor) equation
54	res_part_fracs%meta_frac	real	fraction	0.85	metabolic fraction of residue
55	res_part_fracs%str_frac	real	fraction	0.15	structural fraction of residue
56	res_part_fracs%lig_frac	real	fraction	0.12	lignin fraction of residue

When `cswat = 2` in `codes.bsn`, the three residue partition fractions are reinterpreted into above- and below-ground metabolic, structural, and lignin pools (`cswat_1_part_fracs`) in `plant_parm_read`. When `cswat /= 2`, the reader overwrites the read values with hard-coded defaults of 0.85, 0.15, and 0.12.

Example

First two records of `refdata/Ames_sub1/plants.plt`:

```
plants.plt
name  plnt_typ  gro_trig  nfix_co  days_mat  bm_e  harv_idx  lai_pot  frac_hu1  lai_max1  frac_hu2  lai_ma
alfa  perennial  temp_gro  0.5  0  20  0.9  4  0.15  0.01  0.5  0.95  0.9  0.5  0.9  3  20  4  0.02  0.003
cana  warm_annual  temp_gro  0  100  34  0.3  4.5  0.15  0.02  0.45  0.95  0.5  0.3  1.3  1.4  21  5  0.03
```

Notes on the example:

- Header column names written by the editor (`plnt_typ`, `gro_trig`, `bm_e`, `harv_idx`, `lai_pot`, `frac_hu1`, `frac_n_yld`, `frac_n_em`, `stcon_max`, `frac_stcon`, `ru_vpd`, `co2_hi`, `bm_e_hi`, `plnt_decomp`, `lai_min`, `bm_tree_acc`, `yrs_mat`, `bm_tree_max`, `ext_co`, `leaf_tov_mn`, `leaf_tov_mx`, `rt_st_beg`, `rt_st_end`, `plnt_pop1`, `frac_lai1`, `frac_sw_gro`, `avg_lig_frac`, `ab_lig_frac`, `bg_lig_frac`) do not match the field names in the source. The reader uses position only.
- The trailing `description` column is not part of `plant_db`. List-directed reads stop after the type is filled, so the description is silently discarded.
- For `alfa`, `days_mat = 0` is the explicit "use heat units for the full season" signal.

Related files

- `codes.bsn`. `bsn_cc%nam1` selects whether `pl_class` is read after each record. `bsn_cc%cswat` selects how residue partition fractions are interpreted.
- `plant.ini`. Lists which plants are present in each HRU at the start of simulation, keyed by `plantnm`.
- `landuse.lum` and `plant.ops`. Reference plant names from this file in management schedules.

Purpose

`fertilizer.frt` lists every fertilizer product the simulation can apply, with its mass fractions of mineral nitrogen, mineral phosphorus, organic nitrogen, organic phosphorus, and the share of mineral N that is in the NH3 form. Each record fills one `fertilizer_db` entry in the `fertdb` array. Fertilizer operations (in `chem_app.ops` and the management decision tables) reference entries here by name.

Source

- Reader: `src/fert_parm_read.f90`
- Type definition: `src/fertilizer_data_module.f90`, `type fertilizer_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per fertilizer product. List-directed read into `fertdb(it)` (`type fertilizer_db`).

#	Field	Type	Units	Default	Description
1	fertnm	char(16)	none	“”	fertilizer name (key referenced by `chem_app.ops` and management decision tables)
2	fminn	real	kg min-N / kg fertilizer	0.0	fraction of fertilizer that is mineral N (NO3 + NH3)
3	fminp	real	kg min-P / kg fertilizer	0.0	fraction of fertilizer that is mineral P
4	forgn	real	kg org-N / kg fertilizer	0.0	fraction of fertilizer that is organic N
5	forgp	real	kg org-P / kg fertilizer	0.0	fraction of fertilizer that is organic P
6	fnh3n	real	kg NH3-N / kg min-N	0.0	fraction of mineral N content that is NH3

The five real fractions must sum to less than or equal to 1 for a single product. `fminn + fminp + forgn + forgp` is the total nutrient mass per unit fertilizer; the remainder is carrier mass.

Example

First lines of `refdata/Ames_sub1/fertilizer.frt`:

```
fertilizer.frt: written by SWAT+ editor v2.2.2 on 2023-10-17 14:43 for SWAT+ rev.60.5.4
name                      min_n         min_p         org_n         org_p         nh3_n         pathogens  
elem_n                  1.00000       0.00000       0.00000       0.00000       0.00000              null  
elem_p                  0.00000       1.00000       0.00000       0.00000       0.00000              null  
anh_nh3                 0.82000       0.00000       0.00000       0.00000       1.00000              null  
urea                    0.46000       0.00000       0.00000       0.00000       1.00000              null  
46_00_00                0.46000       0.00000       0.00000       0.00000       0.00000              null  
18_46_00                0.18000       0.20200       0.00000       0.00000       0.00000              null  
dairy_fr                0.00700       0.00500       0.03100       0.00300       0.99000              null  
```

Notes on the example:

- The editor writes two trailing columns (`pathogens` and `description`) that are not in `fertilizer_db`. List-directed reads stop once the type is filled, so both trailing tokens are discarded by the reader. The literal `null` in the `pathogens` column has no effect.
- For `urea`, `fminn = 0.46` and `fnh3n = 1.00`. That is, 46% of the product is mineral N, and 100% of that mineral N is in the NH3 form.
- For `18_46_00`, `fminn = 0.18` and `fminp = 0.202`. Note that the third value in the "46" position is the P (P2O5 converted to P), not P2O5 directly.

manure_db.frt

Named manure types. Each entry points to one organic-matter record in `manure_om.frt` and (optionally) to records for pesticides, pathogens, heavy metals, salts, and other constituents that may be carried with the manure. This is the lookup used by manure-application operations and by manure-allocation.md.

Source

- Reader: src/manure_db_read.f90
- Type: src/fertilizer_data_module.f90, `type manure_database` (variable `manure_db`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per manure type. Two passes: count rows, then read.

#	Field	Type	Description
1	name	char(25)	Manure-type name. Referenced by manure-application operations and by `manure_allo.mnu`.
2	org_min	char(25)	Crosswalk into `manure_om.frt`. Resolved into `iorg_min`.
3	pests	char(25)	Pesticide profile name. Pointer field `ipests` is reserved (the crosswalk loop is commented out in the reader). Use `null` when not applicable.
4	paths	char(25)	Pathogen profile name. Pointer `ipaths`.
5	hmets	char(25)	Heavy-metal profile name. Pointer `imets`.
6	salts	char(25)	Salt-ion profile name. Pointer `isalts`.
7	constit	char(25)	Other-constituent profile name. Pointer `iconstit`.
8	descrip	char(80)	Free-text description.

After reading each row, the reader crosswalks `org_min` against `manure_om(:)%name` and stores the matching index in `manure_db(it)%iorg_min`. The other crosswalks are present in the type but not wired in this reader.

Example

`refdata/Osu_1hru/manure_db.frt`:

```
manure_db.frt: manure database
name          org_min            pests  paths  hmets  salts  constit  description
beef_salt     mw_beef_liq        null   null   null   null   null     midwest_beef_liquid_salt
```

One manure type, pointing at `mw_beef_liq` in `manure_om.frt`. No pesticide, pathogen, heavy-metal, salt, or constituent profile.

manure_om.frt

Organic-matter attributes for each manure type. Holds the water fraction and the C, mineral N, mineral P, organic N, organic P, and NH3 fractions used when a manure application is partitioned into the soil pools.

Source

- Reader: src/manure_orgmin_read.f90
- Type: src/fertilizer_data_module.f90, `type manure_attributes` (variable `manure_om`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per manure organic-matter profile.

#	Field	Type	Units	Description
1	name	char(64)	none	Profile name. Used as the crosswalk target from `manure_db.frt`.
2	frac_water	real	kg water / (kg manure + kg water)	Water fraction of the as-applied manure.
3	fcbn	real	kg C / kg manure	Carbon fraction.
4	fminn	real	kg min-N / kg manure	Mineral N fraction (NO3 + NH3).
5	fminp	real	kg min-P / kg manure	Mineral P fraction.
6	forgn	real	kg org-N / kg manure	Organic N fraction.
7	forgp	real	kg org-P / kg manure	Organic P fraction.
8	fnh3n	real	kg NH3-N / kg min-N	Share of mineral N that is NH3.
9	description	char(64)	none	Free-text description.

Example

`refdata/Osu_1hru/manure_om.frt`:

```
manure_om.frt: manure organic matter database
name           frac_water   fcbn          fminn         fminp         forgn         forgp         fnh3n  c
mw_beef_liq    0.988        0.000780576   0.000311751   0.00081295    0.002110312   0.000365707   0.99   r
mw_beef_slur   0.926        0.016786571   0.002697842   0.00081295    0.002110312   0.000365707   0.99   r
```

Two profiles. The liquid form is nearly all water (98.8%); the slurry form is 92.6% water with much higher solids and carbon.

Related files

- `chem_app.ops`. Fertilizer applications in management schedules reference `fertnm` from `fertilizer.frt` or a manure name from `manure_db.frt`.
- `manure-allocation.md`. Allocates manure from named source objects to HRUs and other receivers.
- `codes.bsn`. The active carbon model (`cswat`) controls how N from fertilizer and manure enters the soil pools.

Purpose

`tillage.til` defines each named tillage operation. Each record fills one `tillage_db` entry in the `tilldb` array. Tillage operations referenced in `chem_app.ops` and management decision tables look up the implement by name to get the mixing efficiency, mixing depth, random roughness, and ridge geometry that drive soil-mixing and surface-roughness updates in `mgt_newtillmix_*` and `mgt_tillfactor.f90`.

The reader also scans for the entry named `biomix`. If present, its `effmix` and `deptil` are copied into the module-level `bmix_eff` and `bmix_depth`, which control the daily biological mixing computed in `mgt_biomix.f90`. If no `biomix` entry is present, `bmix_eff` defaults to 0.2 and `bmix_depth` defaults to 50 mm.

Source

- Reader: `src/till_parm_read.f90`
- Type definition: `src/tillage_data_module.f90`, `type tillage_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per tillage operation. List-directed read into `tilldb(itl)` (`type tillage_db`).

#	Field	Type	Units	Default	Description
1	tillnm	char(16)	none	“”	tillage operation name (key from management schedules). The literal name `biomix` sets the biological mixing parameters at the module level
2	effmix	real	none	0.0	mixing efficiency of the tillage operation (fraction of soil mass redistributed within the till depth)
3	deptil	real	mm	0.0	depth of mixing caused by the tillage operation
4	ranrns	real	mm	0.0	random roughness imparted to the soil surface
5	ridge_ht	real	mm	0.0	ridge height
6	ridge_sp	real	mm	0.0	ridge interval (or row spacing)

Example

First lines of `refdata/Osu_1hru/tillage.til` (which has the title and header lines the reader expects):

```
tillage.til: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    mix_eff        mix_dp         rough      ridge_ht      ridge_sp  description
puddle                  0.980       150.000        00.000         0.000         0.000    rice_paddy_puddlin
```

The trailing `description` column is not in `tillage_db`. List-directed reads stop after the six fields are filled, so the description is silently discarded.

The Ames database has many more operations (the first records, after a correct two-line header, are):

```
chiselplow              0.65          225           20              0             0     chiselplow
cltiweed                0.3           100           15              0             0     cultiweeder
moldboardplow           0.95          250           30              0             0     mldboard
biomix                  0.01           50            0              0             0     biological_mixing
```

The `biomix` row above sets `bmix_eff = 0.01` and `bmix_depth = 50` at the module level after the file is read.

Related files

- `chem_app.ops` and `lum.dtl` (and similar decision tables). Tillage operations reference `tillnm` from this file.
- codes.bsn. The active carbon model (`cswat`) selects which `mgt_newtillmix_*` routine performs the soil pool mixing.
- `hru-data.hru` and `landuse.lum`. Identify the HRUs that receive the tillage operations defined here.

Removed

`carb_coefs.cbn` was removed. The keyword-style reader (`carbon_coef_read.f90`) and the dead companion file `basins_carbon.tes` have also been deleted. The 22 daily hard-coded coefficient overrides that previously lived in `cbn_zhang2.f90` are gone; user-supplied values now flow end-to-end.

What replaces it

Carbon inputs have been split into two flat-table files that follow the same layout convention as the rest of the SWAT+ inputs (title line, header line, data rows):

- carbon.bsn: 28 basin-wide scalar parameters in a single data row. Initial pool fractions, water coupling, manure partitioning, temperature and water factor selectors, the `t_cbn_*` triple, biomixing and tillage curves, photodegradation, residue C:N / C:P controls, and the slow-humus initialization method (`mathers_method`).
- carbon_lyr.bsn — per-layer rate constants and CO2 allocation fractions. One row per layer group (1 = top, 2 = subsurface).

Both files are required when `bsn_cc%cswat == 2` (the dynamic carbon model; the activation value moved from 1 to 2). Either missing or malformed file triggers `error stop`; there is no silent fallback to compiled defaults.

Migration

Old keyword in `carb_coefs.cbn`	New location
`cbn_diagnostics`	retired; output gating now lives in print.prt via the `hru_cb_*` and `lsu_cb_*` rows
`hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp`	per-layer rows of carbon_lyr.bsn
`a1co2`, `asco2`, `apco2`, `abco2`	per-layer rows of carbon_lyr.bsn
`org_frac` (`frac_seq`, `frac_hum_microb`, `frac_hum_slow`, `frac_hum_passive`)	`init_seq`, `init_microb`, `init_slow`, `init_passive` columns of carbon.bsn
`prmt_21`	`koc_c` column of carbon.bsn
`prmt_44`	`solc_ratio` column of carbon.bsn
`till_eff_days`	`till_eff_days` column of carbon.bsn

Old keyword in carb_coefs.cbn	New location
rtof	manure_c_frac column of carbon.bsn
cbn_consolidation_factors (2 values)	bio_consol, till_consol columns of carbon.bsn
cbn_factor_approaches (2 ints)	tmpf_eqn, watf_eqn columns of carbon.bsn
tn, top, tx	t_cbn_min, t_cbn_opt, t_cbn_max columns of carbon.bsn
zz_bmix_coefs (3 values)	bmix_a, bmix_b, bmix_c columns of carbon.bsn. zz_ prefix dropped to match the column name.
zz_emix_coefs (3 values)	tillmix_a, tillmix_b, tillmix_c columns of carbon.bsn
photo_degrade_factor	sfc_rsd_photodeg column of carbon.bsn
nmbr_soil_test_layers, soil_test	retired; no replacement (sub-table was unused downstream)

Five new tunables were promoted from hard-coded constants and added to carbon.bsn:

- n_act_frac (formerly the nactfr local in nut_nminrl.f90)
- cnr_cap, cnr_ref (formerly the 500 and 25 hard-coded values in cbn_surfrsd_decomp.f90)
- cpr_cap, cpr_ref (formerly the 5000 and 200 hard-coded values in cbn_surfrsd_decomp.f90)

Related files

- carbon.bsn. Replacement for the basin-wide scalars.
- carbon_lyr.bsn. Replacement for the per-layer table.
- codes.bsn. The cswat switch. cswat == 2 and cswat == 3 are no longer accepted; set 0 (no carbon) or 1 (active carbon).
- Carbon outputs. Replacement for the cbn_diagnostics flag.

Purpose

treatment.trt is a legacy file name carried over from the SWAT2012 treatment operation table. It is written by external setup tools (QSWAT+, SWAT+ Editor) into project directories, but no SWAT+ Fortran reader consumes it. The file plays no part in a SWAT+ run.

The current treatment functionality in SWAT+ lives in the water allocation module and uses water_treat.wal, not treatment.trt. See src/water_treatment_read.f90.

Source

A search of the SWAT+ Fortran tree at commit 8a1edd4a23915a2cac1cd90c788f0d521404700e finds no open, inquire, or filename string referencing treatment.trt or any .trt extension. file.cio does not list it.

```
grep -rn "treatment.trt\|\\.trt" swatplus/src/
# no matches
```

Format

There is no defined format. The file is not parsed.

Example

Both reference datasets ship an empty treatment.trt:

```
$ wc -l refdata/Ames_sub1/treatment.trt refdata/Osu_1hru/treatment.trt
0 refdata/Ames_sub1/treatment.trt
0 refdata/Osu_1hru/treatment.trt
```

The file exists only as a placeholder.

Related files

- water_treat.wal. The active treatment input, read by `water_treatment_read`. Defines water treatment objects with storage, lag, loss fraction, and constituent links used by water allocation.

Purpose

`manure_allo.mnu` defines manure-allocation objects. Each object names one or more source locations (manure storage on a farm, a feedlot, a lagoon) and one or more demand locations (HRUs, municipal treatment, inter-basin diversion). At runtime, manure produced at the sources is distributed to the demand objects under a named rule. Each allocation object holds its own source list, demand list, monthly production, storage, and per-source withdrawal history.

The allocation file ties three databases together:

- fertilizer.frt. Each source's `manure_typ` is a key into `fertdb` (cross-walked at read time into `src%fertdb`).
- lum.dtl. For HRU demands, the rule name is a key into `dtbl_lum` (cross-walked into `trn%dtbl_num`).
- `chem_app.ops`. The first action of the matched lum decision table names a chemical-application profile in `chemapp_db`, cross-walked into `manure_amt%app_method`.

Source

- Reader: src/manure_allocation_read.f90
- Types: src/manure_allocation_module.f90
  - `type manure_allocation` (variable `mallo`)
  - `type manure_source_objects` (`mallo(i)%src(j)`)
  - `type manure_demand_objects` (`mallo(i)%trn(j)`)

Format

- Line 1: title (skipped).
- Line 2: number of allocation objects, `imax`. Stored in `db_mx%mallo_db`.
- Per allocation object, in order:
  1. Header line (skipped).
  2. Object header record: `name`, `rule_typ`, `src_obs`, `trn_obs`.
  3. Header line (skipped).
  4. `src_obs` source records.
  5. Header line (skipped).
  6. `trn_obs` demand records.

Object header

#	Field	Type	Description
1	name	char(25)	Allocation-object name.
2	rule_typ	char(25)	Allocation rule. Selects how unmet demand is split across sources.
3	src_obs	int	Number of source objects in this allocation.
4	trn_obs	int	Number of demand objects in this allocation.

Source record (one per source)

#	Field	Type	Units	Description
1	num	int	none	Sequential source id. Used as the array index.
2	mois_typ	char(3)	none	Moisture class: `wet` or `dry`.

#	Field	Type	Units	Description
3	manure_typ	char(25)	none	Manure-product name. Cross-walked against `fertdb(:)%fertnm` and stored in `fertdb`.
4	lat	real	degrees	Latitude of the source.
5	long	real	degrees	Longitude of the source.
6	stor_init	real	t	Initial manure in storage.
7	stor_max	real	t	Storage capacity.
8..19	prod_mon(1..12)	real	t/month	Average monthly manure production. Twelve values, January through December.

Demand record (one per demand)

#	Field	Type	Description
1	num	int	Sequential demand id. Used as the array index.
2	ob_typ	char(10)	Receiver type: `hru` (apply to an HRU), `muni` (municipal treatment), or `divert` (inter-basin diversion).
3	ob_num	int	Number of the receiving object of `ob_typ`.
4	dtbl	char(25)	Decision-table name. For `hru`, matched against `dtbl_lum`.
5	right	char(2)	Allocation right. `sr` for senior, `jr` for junior.

For an HRU demand, the reader sets `hru(ihru)%man_trn_dtbl` to the resolved decision-table index, and looks up the first action's `option` against `chemapp_db(:)%name` to set the application method on `mallo(imro)%trn(itrn)%manure_amt%app_method`.

Example

A two-line skeleton with one source and one HRU demand:

```
manure_allo.mnu: example
1
name            rule_typ    src_obs  trn_obs
farm_a          fixed             1        1
num  mois_typ  manure_typ  lat       long      stor_init  stor_max   prod_mon(1..12)
  1  wet       beef_salt   41.99   -93.62          0.0       500.0    5.0  5.0  6.0  6.0  7.0  7.0  7.0  7
num  ob_typ    ob_num  dtbl                right
  1  hru             1  apply_manure_dtbl      sr
```

The decision-table named `apply_manure_dtbl` must exist in `lum.dtl` and its first action must name an entry in `chem_app.ops` for the cross-walks to resolve.

Related files

- `fertilizer.frt`. Defines the manure-product names that source records reference, and the manure organic-matter database (`manure_db.frt`, `manure_om.frt`).
- `lum.dtl`. Decision tables that fire manure applications at each demand HRU.
- `chem_app.ops`. Chemical-application profiles (rate, method) selected by the decision-table action.

Five small inputs that the larger pages reference but do not document. Each is a single reader populating a single derived type.

management.sch

Fixed-schedule management. Each schedule lists scheduled operations on calendar dates or heat-unit fractions, plus a set of named auto-operation tables (decision tables in `lum.dtl`). `landuse.lum` selects a schedule by name through `mgt`.

- Reader: `src/mgt_read_mgtops.f90` (header) and `src/read_mgtops.f90` (operation rows).
- Default file name: `management.sch` (from `in_lum%management_sch` in `input_file_module.f90`).
- Target type: `management_schedule` in `src/mgt_operations_module.f90`, array `sched(:)`.

File format

- Line 1: title (skipped).
- Line 2: header (skipped).
- For each schedule:
  - One header row: `name`, `num_ops`, `num_autos`.
  - `num_autos` rows, each naming an auto-operation. If the name is `pl_hv_summer1` or `pl_hv_winter1` the row also lists one crop name; if it is `pl_hv_summer2` it lists two crop names.
  - `num_ops` operation rows, list-directed.

Schedule header

#	Field	Type	Description
1	name	char(40)	schedule name referenced by `landuse.lum`
2	num_ops	int	number of scheduled operation rows
3	num_autos	int	number of auto-operation tables

Operation row

Read into `management_ops` (see `mgt_operations_module.f90`).

#	Field	Type	Description
1	op	char(40)	operation code (see below)
2	mon	int	calendar month
3	day	int	calendar day of month
4	husc	real	heat-unit fraction trigger (0 if calendar-driven)
5	op_char	char(40)	name into the operation database keyed by `op`
6	op_plant	char(40)	name into a secondary database (plant, harvest type, chemical application)
7	op3	real	application amount or override value

Recognised `op` codes and the database that `op_char` (and where noted `op_plant`) resolves against:

Code	Action	op_char database	op_plant database
`pcom`	plant community init	`plant.ini` (`pcomdb`)	none
`plnt`	plant	`plants.plt` (`pldb`)	`transplant.plt` (`transpl`)
`harv`	harvest	(none)	`harv.ops` (`harvop_db`)
`hvkl`	harvest and kill	(none)	`harv.ops`
`kill`	kill	none	none
`till`	tillage	`tillage.til` (`tilldb`)	none
`irrm`, `irrp`	irrigation	`irr.ops` (`irrop_db`)	none
`fert`	fertilizer	`fertilizer.frt` (`fertdb`)	`chem_app.ops` (`chemapp_db`)
`manu`	manure	`manure_db.frt` (`manure_db`)	`chem_app.ops`
`pest`	pesticide	`pesticide.pes` (`pestdb`)	`chem_app.ops`
`graz`	grazing	`graze.ops` (`grazeop_db`)	none
`burn`	burn	`fire.ops` (`fire_db`)	none
`swep`	street sweep	`sweep.ops` (`sweepop_db`)	none
`skip`	end of rotation year	none	none
`prtp`	print plant variables	none	none

A `skip` row advances the internal rotation year counter; subsequent rows belong to the next year of the rotation.

Example

`refdata/Ames_sub1/management.sch`:

```
management.sch file AMES
                          NAME NUMB_OPS NUMB_AUTO OP_TYP   MON   DAY HU_SCH   OP_DATA1
 ↪  OP_DATA2  OP_DATA3
mgt_01                               305         1
                                                 irr_year_irr
                                                     till    4     1    0.0   chisplow
 ↪  null     0.000
                                                     plnt    4    28    0.0       corn
 ↪  null     0
                                                     irrm    5    15    0.0  sprinkler_ilm
 ↪  null     0.000
                                                     fert    6     1    0.0       null
 ↪  elem_n   181.400
                                                     hvkl   10     7    0.0       corn
 ↪  grain     0.000
                                                     skip    0     0    0         null
 ↪  null     0
```

Related files

- `landuse.lum` selects a schedule by name.
- `*.ops` hold the operation parameters that `op_char` and `op_plant` resolve against.
- `*.dtl` decision tables provide the `auto_name` entries.

puddle.ops

Rice paddy puddling parameters. A puddling event resets the upper soil-layer hydraulic conductivity and the in-paddy water chemistry to fixed values.

- Reader: `src/mgt_read_puddle.f90`.
- File name: hard-coded `puddle.ops`.
- Target type: `puddle_operation` in `src/mgt_operations_module.f90`, array `pudl_db(:)`.

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record, list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	wet_hc	real	0	mm/h	upper-layer hydraulic conductivity after puddling
3	sed	real	0	ppm	sediment concentration after puddling
4	orgn	real	0	ppm	organic N concentration after puddling
5	sedp	real	0	ppm	organic P concentration after puddling
6	no3	real	0	ppm	NO3-N concentration after puddling
7	solp	real	0	ppm	mineral (soluble) P concentration after puddling
8	nh3	real	0	ppm	NH3 concentration after puddling
9	no2	real	0	ppm	NO2 concentration after puddling

Related files

- `*.ops` for the other operation databases that follow the same skip-two-and-read pattern.

transplant.plt

Plant transplant database. Initial state for a plant that is established by transplant rather than from seed. Operation rows with `op = plnt` look up `op_plant` against this database.

- Reader: `src/plant_transplant_read.f90`.
- File name: hard-coded `transplant.plt`.
- Target type: `plant_transplant_db` in `src/plant_data_module.f90`, array `transpl(:)`.

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record.

#	Field	Type	Default	Units	Description
1	name	char(40)	`frsd`	none	transplant entry name
2	lai	real	0	$m^{2/m}2$	initial leaf area index
3	bioms	real	0	kg/ha	initial biomass
4	phuacc	real	0	fraction	initial fraction of accumulated heat units
5	fr_yrmat	real	0.05	fraction	fraction of current year of growth to years to maturity
6	pop	real	0	plants/m^2	initial plant population

Related files

- plants.plt for the underlying plant parameter database.
- management.sch resolves the op_plant field against transplant.plt for plnt operations.

co2.out and co2_yr.dat

CO2 driver. Despite the .out extension, co2_read is the routine that establishes the per-year atmospheric CO2 series used by the simulation. The reader reads co2_yr.dat if present, fills the per-year array co2y(:), and writes the resolved series to co2.out as a record of what was used.

- Reader: src/co2_read.f90.
- Input file (optional): co2_yr.dat.
- Output file: co2.out (unit 2222).
- Fallback: when co2_yr.dat is missing, every simulation year is filled with bsn_prm%co2 from parameters.bsn.

co2_yr.dat format

Line	Content
1	title (skipped)
2	integer yrs, the number of year records that follow
3	header (skipped)
4 to 3+yrs	one record per year: iyr (int, calendar year) and co2 (real, ppm)

How the series is built

For a simulation that runs time%nbyr years starting in time%yrc_start:

- If co2_yr.dat is absent or empty, every simulation year is set to bsn_prm%co2.
- If the latest year in co2_yr.dat is at or before the simulation start, every simulation year is set to that final value.
- Otherwise the file is aligned to the simulation start. Simulation years that fall before the first record use the first record's value; years that fall after the last record carry the last value forward.

co2.out format

Header YR CO2(ppm), then one row per simulation year with the calendar year and the resolved ppm value.

Related files

- codes.bsn and parameters.bsn for the co2 fallback in bsn_prm.

object.prt

Per-object output selector. Each row asks the model to write one specific hydrograph or state variable for one spatial object to a named file. Reuses the old saveconc mechanism.

- Reader: src/object_read_output.f90.
- Default file name: object.prt (from in_sim%object_prt in input_file_module.f90).
- Target type: object_output in src/hydrograph_module.f90, array ob_out(:).

File format

Line 1 title and line 2 header are skipped. Each remaining row is one selection.

#	Field	Type	Description
1	k	int	row id (the reader also uses it to size the array)
2	obtyp	char(10)	object type (see below)
3	obtypno	int	object number within `obtyp` (1-based)
4	hydtyp	char(20)	hydrograph or state code (see below)
5	filename	char(26)	output file name to open and write to

Object types recognised by the reader:

Code	Object
`hru`	HRU
`hlt`	HRU LTE
`ru`	routing unit
`res`	reservoir
`cha`	channel
`exc`	export coefficient object
`dr`	delivery ratio object
`out`	outlet
`sdc`	SWAT-Deg channel

Hydrograph or state codes:

Code	Content	Header written
`tot`	total flow	`hyd_hdr_time hyd_hdr`
`rhg`	recharge	`hyd_hdr_time hyd_hdr`
`sur`	surface	`hyd_hdr_time hyd_hdr`
`lat`	lateral	`hyd_hdr_time hyd_hdr`
`til`	tile	`hyd_hdr_time hyd_hdr`
`sol_water`	soil moisture by layer	`hyd_hdr_time sol_hdr`
`solnut_ly`	soil N and P by layer	(no header)
`solnut_pr`	soil N and P for profile	(no header)
`plant`	plant status	`hyd_hdr_time plt_hdr plt_hdr`
`cha_fp`	channel and flood-plain water balance	`hyd_hdr_time fp_hdr`

Each row opens its own file at unit `6100 + i` and writes the matching header. The simulation appends to that file as the run progresses.

Related files

- `print.prt` controls the main output selection. `object.prt` is the side channel for per-object hydrograph files.

Purpose

The initial-conditions files set the starting state of land and water objects before the first simulation day. The general files are:

- `plant.ini`. Named plant communities. One community is a list of plants with initial LAI, biomass, heat-unit accumulation, plant population, fraction of years to maturity, and residue cover. Land-use entries in `landuse.lum` point at a community name through the `plant_cov` field.
- `soil_plant.ini`. Named soil and plant initial-state records combining a starting soil-water fraction with names of nutrient, pesticide, pathogen, salt, heavy-metal, and constituent initial-condition records. `hru-data.hru` references one row of this file per HRU through `soil_plant_init`.
- `om_water.ini`. Initial organic matter and water properties for water bodies (aquifers, channels, ponds, reservoirs, wetlands). One row per named initialisation record.

The solute-specific initial-condition files (pest_hru.ini, pest_water.ini, path_hru.ini, path_water.ini, hmet_hru.ini, hmet_water.ini, salt_hru.ini, salt_water.ini, cs_hru.ini, cs_aqu.ini, cs_channel.ini) are declared in the same type input_init block but documented on the module pages linked at the bottom.

Source

Filename strings: src/input_file_module.f90, type input_init.

File	Reader	Type populated
plant.ini	src/readpcom.f90	type plant_community_db (with type plant_init_db array) in src/plant_data_module.f90
soil_plant.ini	src/soil_plant_init.f90	type soil_plant_initialize in src/hru_module.f90
om_water.ini	src/om_water_init.f90	type hyd_output in src/hydrograph_module.f90

All three readers skip line 1 (title) and line 2 (header), then read records list-directed. Column positions in the reference examples are cosmetic indentation. If the file is missing or the filename in file.cio is null, the reader allocates an empty array and continues.

plant.ini

Each plant community is a header record followed by plt_cnt plant records.

Community record (3 fields)

Read into pcomdb(icom).

#	Field	Type	Units	Default	Description
1	name	char(40)	none	“frsd_frsd”	community name. Referenced from `landuse.lum` (`plant_cov`)
2	plants_com	int	none	1	number of plant records that follow
3	rot_yr_ini	int	none	1	starting year in the rotation

Plant record (8 fields, repeated `plants_com` times)

Read into `pcomdb(icom)%pl(iplt)` (`type plant_init_db`).

#	Field	Type	Units	Default	Description
1	cpnm	char(40)	none	“frsd”	plant name. Must match a row in `plants.plt`. The reader looks the name up in `pldb` and stores the index in `db_num`. If the name is not found a message is written to `9001` (`diagnostics.out`)
2	igro	char(1)	none	“y”	land cover status. `y` = growing, `n` = not growing
3	lai	real	$m^{2/m}2$	0.0	initial leaf area index
4	bioms	real	kg/ha	0.0	initial above-ground biomass
5	phuacc	real	fraction	0.0	fraction of plant heat units accumulated at simulation start
6	pop	real	plants/m^2	0.0	plant population
7	fr_yrmat	real	fraction	1.0	fraction of current year of growth to years to maturity. Used for perennial trees
8	rsdin	real	kg/ha	10000.0	initial surface residue

Example

From `refdata/Ames_sub1/plant.ini`. First two communities, indentation trimmed for width:

```
plant.ini:
pcom_name   plt_cnt  rot_yr_ini  plt_name  lc_status  lai_init  bm_init  phu_init  plnt_pop  yrs_init  rsd
cosy_comm         2           1
                              corn        n   0.00000      0.00000   0.00000   0.00000   0.00000   10000.0
                              soyb        n   0.00000      0.00000   0.00000   0.00000   0.00000   10000.0
corn_comm         1           1
                              corn        n   0.00000      0.00000   0.00000   0.00000   0.00000   10000.0
```

soil_plant.ini

One row per initialisation record. Rows are referenced by name from `hru-data.hru` through the `soil_plant_init` field.

The number of fields read depends on `bsn_cc%nam1`. When `nam1 == 0` (the default) the reader reads 7 fields. When `nam1 /= 0` it reads an eighth field (`csc`) for the constituent-side initial-condition record. `nam1` is declared in `basin_module.f90` as "not used", so in practice the 7-field form is what is written.

Row (7 fields, or 8 with constituents)

Read into `sol_plt_ini(ii)` (`type soil_plant_initialize`).

#	Field	Type	Units	Default	Description
1	name	char(40)	none	“”	record name. Referenced from **hru-data.hru**
2	sw_frac	real	fraction	0.0	initial soil water content as a fraction of field capacity
3	nutc	char(40)	none	“”	name of an initial nutrient record (links to soil nutrient initialisation)
4	pestc	char(40)	none	“”	name of an initial pesticide record. See pest_hru.ini
5	pathc	char(40)	none	“”	name of an initial pathogen record. See path_hru.ini
6	saltc	char(40)	none	“”	name of an initial salt record. See salt_hru.ini
7	hmetc	char(40)	none	“”	name of an initial heavy-metal record. See hmet_hru.ini
8	csc	char(40)	none	“”	name of an initial constituent record. Read only when **bsn_cc%nam1 /= 0**. See cs_hru.ini

Use `null` for any solute-side column that is not active.

Example

From `refdata/Ames_sub1/soil_plant.ini`:

```
soil_plt_ini
       NAME     SW_FRAC    NUT       PEST   PATH   SALT   HMET
soilplant1         0.5    soilnut1   null   null   null   null
```

om_water.ini

One row per named record. Rows are referenced by name from the connectivity files of water bodies (aquifers in `aqu_read_init.f90`, channels, reservoirs, ponds, wetlands).

Row (19 fields)

The reader calls `read (..., om_init_name(ichi), om_init_water(ichi))`. The first field is the name. The remaining 18 fields fill all members of `type hyd_output` in declaration order.

#	Field	Type	Units	Default	Description
1	name	char(16)	none	“”	record name
2	flo	real	m^3 (or fraction for some objects)	0.0	water volume
3	sed	real	metric tons	0.0	sediment
4	orgn	real	kg N	0.0	organic N (PART_N in writer header)
5	sedp	real	kg P	0.0	organic P (PART_P)
6	no3	real	kg N	0.0	NO3-N
7	solp	real	kg P	0.0	mineral soluble P
8	chla	real	kg	0.0	chlorophyll-a
9	nh3	real	kg N	0.0	NH3
10	no2	real	kg N	0.0	NO2
11	cbod	real	kg	0.0	carbonaceous biological oxygen demand
12	dox	real	kg	0.0	dissolved oxygen
13	san	real	tons	0.0	detached sand
14	sil	real	tons	0.0	detached silt
15	cla	real	tons	0.0	detached clay
16	sag	real	tons	0.0	detached small aggregates
17	lag	real	tons	0.0	detached large aggregates
18	grv	real	tons	0.0	gravel
19	temp	real	deg C	0.0	water temperature

The `flo` field is interpreted as a fraction of principal storage for reservoirs and ponds, and as an absolute volume for channels. The interpretation is made by the consumer (`cal_allo_init.f90` and related routines), not the reader.

Example

From `SWATPlusWaterAllocation/documents/wro_test/om_water.ini`:

```
om_wat_ini Generated from ...
       NAME    VOL    SED  PART_N  PART_P    NO3   SOLP  CHL_A    NH3    NO2  CBN_BOD  DIS_OX   SAND   SIL
 aqu_default     1      0       0       0      2      0      0      0      0        0       0      0
 cha_default   0.1   1000      95       0     70      0      0      5      5        0       0      0
pond_default   0.8    100       1     0.1      1   0.03      0    0.1   0.01        0       0      0
 res_default     1    100       1     0.1      1   0.03      0    0.1   0.01        0       0      0
 wet_default   0.5      0       0       0      2      0      0      0      0        0       0      0
```

Related files

- hru-data.hru. Per-HRU pointer into `soil_plant.ini` through `soil_plant_init`.
- landuse.lum. Land-use entries point at a `plant.ini` community through `plant_cov`.
- plants.plt. Plant parameter database. Every `cpnm` in `plant.ini` must resolve here.
- aquifer.aqu, initial.aqu. Aquifers reference `om_water.ini` records for initial storage.
- *.res, *.wet, sd_channel. Water bodies reference `om_water.ini` for initial state.
- Solute-side initial conditions: pesticide module, pathogen module, metals module, salt module, cs module.

Purpose

This page documents the structural BMP parameter files referenced by HRUs and landscape units: `tiledrain.str`, `septic.str`, `filterstrip.str`, `grassedww.str`, and `bmpuser.str`. File names for the first five are set in `type input_structural` in `src/input_file_module.f90`. The page also covers two closely related files declared elsewhere: `septic.sep` (effluent quality, named in `type input_parameter_databases`), `urban.urb` (urban land-cover parameters, same type), and `satbuffer.str` (saturated buffer, file name hard-coded in the reader).

Each file is optional. If the slot in `file.cio` is `null` or the file is missing, the reader allocates a zero-length array and skips on. Every reader skips two header lines, then reads one record per parameter set.

Records in `tiledrain.str`, `filterstrip.str`, `grassedww.str`, and `bmpuser.str` are referenced by name from `hru-data.hru` (columns `tiledrain`, `vfsfield`, `grassww`, `bmpuser`). Records in `septic.str` are referenced from the management schedule when a septic operation is scheduled.

Source

- File names: src/input_file_module.f90, `type input_structural`, `type input_parameter_databases`
- Readers:
  - src/sdr_read.f90 (`tiledrain.str`)
  - src/sep_read.f90 (`septic.str`)
  - src/septic_parm_read.f90 (`septic.sep`)
  - src/scen_read_filtstrip.f90 (`filterstrip.str`)
  - src/scen_read_grwway.f90 (`grassedww.str`)
  - src/scen_read_bmpuser.f90 (`bmpuser.str`)
  - src/sat_buff_read.f90 (`satbuffer.str`)
  - src/urban_parm_read.f90 (`urban.urb`)

tiledrain.str

Subsurface drain parameter sets. Read into `sdr` (`type subsurface_drainage_parameters` in `src/hru_module.f90`). Referenced by the `tiledrain` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“null”	parameter set name
2	depth	real	mm	0.0	depth of drain tube from soil surface
3	time	real	hours	0.0	time to drain soil to field capacity
4	lag	real	hours	0.0	drain tile lag time
5	radius	real	mm	0.0	effective radius of drains
6	dist	real	mm	0.0	distance between two drain tubes or tiles
7	drain_co	real	mm/day	0.0	drainage coefficient
8	pumpcap	real	mm/hr	0.0	pump capacity
9	latksat	real	none	0.0	multiplication factor for lateral saturated hydraulic conductivity

Example from `refdata/Osu_1hru/tiledrain.str`:

```
tiledrain.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        dp          t_fc           lag           rad          dist         drain
mw24_1000          1000.00000      24.00000      96.00000     100.00000      30.00000      10.00000
```

Editor column names (`dp`, `t_fc`, `rad`, `drain`, `pump`, `lat_ksat`) differ from source field names. The reader uses position only.

septic.str

Per-HRU septic system parameter sets (geometry, biozone properties, decay coefficients). Read into `sep` (`type septic_system` in `src/septic_data_module.f90`).

#	Column	Type	Units	Description
1	name	char(13)	none	parameter set name
2	typ	int	none	septic system type
3	yr	int	year	year the septic system became operational
4	opt	int	none	operation flag (1 = active, 2 = failing, 0 = not operated)
5	cap	real	none	number of permanent residents
6	area	real	m^2	average drainfield area per septic system
7	tfail	int	days	time until failing system gets fixed
8	z	real	mm	depth to top of biozone layer
9	thk	real	mm	thickness of biozone layer
10	strm_dist	real	km	distance to stream
11	density	real	per km^2	number of septic systems per square kilometre
12	bd	real	kg/m^3	density of biomass
13	bod_dc	real	m^3/day	BOD decay rate coefficient
14	bod_conv	real	none	mass conversion factor between bacterial growth and BOD degraded
15	fc1	real	none	linear coefficient for biozone field capacity
16	fc2	real	none	exponential coefficient for biozone field capacity
17	fecal	real	m^3/day	fecal coliform decay rate coefficient
18	plq	real	none	plaque conversion factor from TDS
19	mrt	real	none	mortality rate coefficient
20	rsp	real	none	respiration rate coefficient
21	slg1	real	none	slough-off calibration parameter
22	slg2	real	none	slough-off calibration parameter
23	nitr	real	none	nitrification rate coefficient
24	denitr	real	none	denitrification rate coefficient
25	pdistrb	real	L/kg	linear P sorption distribution coefficient
26	psorpmax	real	mg P/kg soil	maximum P sorption capacity
27	solpslp	real	none	slope of linear effluent soluble P equation
28	solpintc	real	none	intercept of linear effluent soluble P equation

Example from `refdata/Osu_1hru/septic.str`:

```
septic.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    typ         yr  operation     residents          area     t_fail       dp_bioz      tl
standard                  1          0          0       2.50000     100.00000         70     500.00000      50
failing                   1          0          0       2.50000     100.00000          1     500.00000      50
```

septic.sep

Effluent quality database. One record per effluent type. Records are referenced by index from elsewhere (no name resolution in the reader). Read into `sepdb` (`type septic_db` in `src/septic_data_module.f90`). File name is in

`in_parmdb%septic_sep`.

#	Column	Type	Units	Description
1	sepnm	char(20)	none	effluent set name
2	qs	real	m^3/day	flow rate per capita
3	bodconcs	real	mg/L	biological oxygen demand
4	tssconcs	real	mg/L	total suspended solids
5	nh4concs	real	mg/L	NH4-N concentration
6	no3concs	real	mg/L	NO3-N concentration
7	no2concs	real	mg/L	NO2-N concentration
8	orgnconcs	real	mg/L	organic N concentration
9	minps	real	mg/L	mineral P concentration
10	orgps	real	mg/L	organic P concentration
11	fcolis	real	mg/L	fecal coliform concentration

The source comment on `nh4concs` reads "concentration of total phosphorus"; the field is the NH4-N concentration based on column position and downstream use. Example from `refdata/Osu_1hru/septic.sep` (first two records shown):

```
septic.sep: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    q_rate           bod           tss          nh4_n          no3_n          no2_n
gcon                   0.22700     170.00000      75.00000      42.40000        0.00000        0.00000
gadv                   0.22700      22.00000      14.00000      18.90000        9.60000        0.00000
```

filterstrip.str

Vegetative filter strip parameter sets. Read into `filtstrip_db` (`type filtstrip_operation` in `src/mgt_operations_module.f9`
Referenced by the `vfsfield` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“”	parameter set name
2	vfsi	int	none	0	on/off flag for vegetative filter strip
3	vfsratio	real	none	0.0	drainage-area to filter-strip-area ratio
4	vfscon	real	none	0.0	fraction of total runoff from the entire field
5	vfsch	real	none	0.0	fraction of flow entering the most concentrated 10% of the VFS that is fully channelized

Example from `refdata/Osu_1hru/filterstrip.str`:

```
filterstrip.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag       fld_vfs       con_vfs         cha_q  description
field_border             0       0.10000       0.00300       0.20000  Field_border
high_engineered          0       0.10000       0.00100       0.05000  Highly_engineered_low_channelized
```

grassedww.str

Grassed waterway parameter sets. Read into `grwaterway_db` (`type grwaterway_operation` in `src/mgt_operations_module.f9`
Referenced by the `grassww` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“”	parameter set name
2	grwat_i	int	none	0	on/off flag for waterway simulation
3	grwat_n	real	none	0.0	Manning’s n for grassed waterway
4	grwat_spcon	real	none	0.0	sediment transport coefficient
5	grwat_d	real	m	0.0	depth of grassed waterway
6	grwat_w	real	m	0.0	width of grassed waterway
7	grwat_l	real	km	0.0	length of grassed waterway
8	grwat_s	real	m/m	0.0	slope of grassed waterway

Example from `refdata/Osu_1hru/grassedww.str`:

```
grassedww.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag          mann        sed_co            dp            wd           len
grwway_high              0       0.05000       0.02000       1.00000       4.00000       0.50000       0.1
grwway_med               0       0.05000       0.02000       0.75000       3.00000       0.75000       0.0
grwway_low               0       0.05000       0.02000       0.50000       2.00000       1.00000       0.0
```

bmpuser.str

User-defined upland BMP removal-efficiency sets. Read into `bmpuser_db` (`type bmpuser_operation` in `src/mgt_operations_module.f90`). Referenced by the `bmpuser` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“”	parameter set name
2	bmp_flag	int	none	0	on/off flag
3	bmp_sed	real	percent	0.0	sediment removal
4	bmp_pp	real	percent	0.0	particulate P removal
5	bmp_sp	real	percent	0.0	soluble P removal
6	bmp_pn	real	percent	0.0	particulate N removal
7	bmp_sn	real	percent	0.0	soluble N removal
8	bmp_bac	real	percent	0.0	bacteria removal

Example from `refdata/Osu_1hru/bmpuser.str`:

```
bmpuser.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag       sed_eff      ptlp_eff      solp_eff      ptln_eff      soln_eff      bact
bmpusr1                  1       0.20000       0.20000       0.20000       0.20000       0.20000       0.2
```

Note: a second type `bmpuser_operation1` in the same module exposes a wider parameter set (surface, subsurface, tile flow and constituents). The reader populates `bmpuser_db` (`bmpuser_operation`); the wider type is not read here.

satbuffer.str

Saturated buffer source-receiver pairs. Read into `satbuff_db` (`type saturated_buffer_parameters` in `src/hru_module.f90`). The file name is hard-coded as `"satbuffer.str"` in the reader, not listed in `type input_structural`. The file is loaded only when it exists.

#	Column	Type	Description
1	name	char(40)	parameter set name
2	hru_src	int	source HRU contributing tile inflow
3	frac_src	real	fraction of source HRU contributing to tile flow
4	flocon_dtbl	char(40)	flow-control decision table name (resolved against `dtbl_flo`)
5	hru_rcv	int	receiving (buffer) HRU
6	lyr	int	soil layer for incoming tile flow (0 = surface)

After reading, the reader copies the parameter set into `hru(hru_src)%sb%sb_db` and `hru(hru_rcv)%sb%sb_db`, then resolves `flocon_dtbl` against the loaded flow-control decision tables (`dtbl_flo`) to set `hru(hru_src)%sb%dtbl`.

urban.urb

Urban land-cover parameter database (build-up, wash-off, percent impervious, urban CN). Read into `urbdb` (`type urban_db` in `src/urban_data_module.f90`). File name is in `in_parmdb%urban_urb`. Urban parameter sets are referenced from `landuse.lum` for HRUs flagged as urban.

#	Column	Type	Units	Default	Description
1	urbnm	char(16)	none	“”	urban land-cover name
2	fimp	real	fraction	0.05	fraction of HRU area that is impervious
3	fcimp	real	fraction	0.05	fraction of HRU classified as directly connected impervious
4	curbden	real	km/ha	0.0	curb length density
5	urbcoef	real	1/mm	0.0	wash-off coefficient for removal of constituents from an impervious surface
6	dirtmx	real	kg/curb-km	1000.0	maximum amount of solids allowed to build up on impervious surfaces
7	thalf	real	days	1.0	time for solids on impervious areas to build up to half the maximum
8	tnconc	real	mg N / kg sed	0.0	total nitrogen concentration in suspended-solid load from impervious areas
9	tpconc	real	mg P / kg sed	0.0	total phosphorus concentration in suspended-solid load from impervious areas
10	tno3conc	real	mg NO3-N / kg sed	0.0	NO3-N concentration in suspended-solid load from impervious areas
11	urbcn2	real	none	98.0	moisture condition II curve number for impervious areas

Example from `refdata/Osu_1hru/urban.urb` (first three records shown):

```
urban.urb: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  frac_imp   frac_dc_imp      curb_den      urb_wash      dirt_max     t_halfmax     c
urhd                   0.60000       0.44000       0.24000       0.18000     225.00000       0.75000     5
urmd                   0.38000       0.30000       0.24000       0.18000     225.00000       0.75000     5
urml                   0.20000       0.17000       0.24000       0.18000     225.00000       0.75000     4
```

Related files

- file.cio. The `structural` and `hru_parm_db` lines select which of these files are read.
- hru-data.hru. References `tiledrain.str`, `filterstrip.str`, `grassedww.str`, and `bmpuser.str` by name.
- landuse.lum. References `urban.urb` entries for urban HRUs.

Purpose

The pesticide module simulates user-listed pesticides across foliage, soil, channel water, benthic sediment, and aquifers. The set of pesticides simulated is declared in constituents.cs, which populates `cs_db%num_pests`. The files below load the parameter database, the per-object initial conditions, and the parent-daughter metabolite chain.

Activation

The pesticide parameter database `pesticide.pes` is referenced from `file.cio` as `in_parmdb%pest` (default `pesticide.pes`). The initial-condition files are referenced from `file.cio` via `in_init%pest_soil` (default `pest_hru.ini`) and `in_init%pest_water` (default `pest_water.ini`).

`pest_parm_read` is called from `proc_db.f90`. `pest_metabolite_read` and `pest_hru_aqu_read` are called from `proc_read.f90`. `pest_cha_res_read` is called later from `main.f90`.

Files

pesticide.pes

Pesticide parameter database. One row per pesticide.

- Reader: src/pest_parm_read.f90
- Type: `pestdb(0:pestparm)` (`type pesticide_db`) in src/pesticide_data_module.f90. The reader also fills `pestcp(ip)%decay_f`, `decay_s`, `decay_a`, `decay_b` from the half-life columns as `exp(-0.693 / hlife)`.
- Format: title line, header line, then one line per pesticide.

#	Field	Units	Description
1	name	-	pesticide name
2	koc	mL/g	soil adsorption coefficient normalized for soil organic carbon
3	washoff	none	fraction of pesticide on foliage washed off by a rainfall event
4	foliar_hlife	days	half-life on foliage
5	soil_hlife	days	half-life in soil
6	solub	mg/L	solubility in water
7	aq_hlife	days	aquatic half-life
8	aq_volat	m/day	aquatic volatilization coefficient
9	mol_wt	g/mol	molecular weight, used for mixing velocity
10	aq_resus	m/day	aquatic resuspension velocity for sorbed pesticide
11	aq_settle	m/day	aquatic settling velocity for sorbed pesticide
12	ben_act_dep	m	depth of the active benthic layer
13	ben_bury	m/day	burial velocity in benthic sediment
14	ben_hlife	days	half-life in benthic sediment

#	Field	Units	Description
15	pl_uptake	none	fraction taken up by plant
16	descrip	-	description string

pest_hru.ini

Initial pesticide concentrations for HRU soils and plants. One block per record.

- Reader: src/pest_hru_aqu_read.f90
- Type: `pest_soil_ini` (`type cs_soil_init_concentrations`) in src/constituent_mass_module.f90. Fields: `name`, `soil(num_pests)` (ppm), `plt(num_pests)` (ppm).
- Format: one title line, then per record: a header line, a name line, then `num_pests` lines each containing `pest_name, soil_value, plt_value` in the order declared in `constituents.cs`.

The same reader populates `cs_pest_solsor(num_pests)` (allocated only).

pest_water.ini

Initial pesticide concentrations for channels and reservoirs (water column and benthic).

- Reader: src/pest_cha_res_read.f90
- Type: `pest_water_ini` (`type cs_water_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `water(num_pests)`, `benthic(num_pests)`. Record names are stored in `pest_init_name`.
- Format: title line, header line, then per record: name line, then `num_pests` pairs of lines (water value, benthic value). The values are then read again on the second pass as a single record line of `name, water(:), benthic(:)`.

pest_metabolite.pes

Parent-daughter metabolite chain. Filename is hard-coded in the reader (not driven by `file.cio`).

- Reader: src/pest_metabolite_read.f90
- Target: `pestcp(ip)%daughter(num_metab)` (`type daughter_decay_fractions`) and `pestcp(ip)%num_metab` in `src/pesticide_data_module.f90`. The daughter `%num` field is set by cross-walk against `cs_db%pests` so the metabolite must also appear in `constituents.cs`.
- Format: title line, header line, then per parent: one line `parent_name num_metab`, followed by `num_metab` lines of `daughter_name, foliar_fr, soil_fr, aq_fr, ben_fr`. The parent name must match an entry in `pesticide.pes`.

Field	Units	Description
name	-	daughter pesticide name
foliar_fr	0-1	fraction of parent on foliage that degrades to this daughter
soil_fr	0-1	fraction of parent in soil that degrades to this daughter
aq_fr	0-1	fraction of parent in aquatic phase that degrades to this daughter
ben_fr	0-1	fraction of parent in benthic phase that degrades to this daughter

Related

- constituents.cs. Declares the pesticide list and sets `cs_db%num_pests`.
- pathogen module. Same per-object initial-condition pattern.
- metals module. Same per-object initial-condition pattern.

Purpose

The pathogen module simulates user-listed pathogens across soil, foliage, channel water, benthic sediment, and aquifers. The set of pathogens simulated is declared in constituents.cs, which populates cs_db%num_paths. The files below load the parameter database and the per-object initial conditions.

Activation

The pathogen database pathogens.pth is referenced from file.cio as in_parmdb%pathcom_db (default pathogens.pth). The initial-condition files are referenced via in_init%path_soil (default path_hru.ini) and in_init%path_water (default path_water.ini).

path_parm_read is called from proc_db.f90. path_hru_aqu_read is called from proc_read.f90. path_cha_res_read is called later from main.f90.

Files

pathogens.pth

Pathogen parameter database. One row per pathogen.

- Reader: src/path_parm_read.f90
- Type: path_db(0:path) (type pathogen_db) in src/pathogen_data_module.f90.
- Format: title line, header line, then one line per pathogen.

#	Field	Units	Description
1	pathnm	-	pathogen name
2	do_soln	1/day	die-off factor in soil solution
3	gr_soln	1/day	growth factor in soil solution
4	do_sorb	1/day	die-off factor sorbed to soil particles
5	gr_sorb	1/day	growth factor sorbed to soil particles
6	kd	none	partition coefficient between solution and sorbed phase in surface runoff
7	t_adj	none	temperature adjustment factor for die-off and growth
8	washoff	none	fraction on foliage washed off by a rainfall event
9	do_plnt	1/day	die-off factor on foliage
10	gr_plnt	1/day	growth factor on foliage
11	fr_manure	none	fraction of manure with active colony forming units
12	perco	none	percolation coefficient, ratio of solution bacteria in the surface layer
13	det_thrshd	cfu/m^2	detection threshold below which soil pathogen levels are zeroed
14	do_stream	1/day	die-off factor in streams
15	gr_stream	1/day	growth factor in streams
16	do_res	1/day	die-off factor in reservoirs
17	gr_res	1/day	growth factor in reservoirs
18	conc_min	-	minimum pathogen concentration

path_hru.ini

Initial pathogen concentrations for HRU soils and plants. One block per record.

- Reader: src/path_hru_aqu_read.f90
- Type: path_soil_ini (type cs_soil_init_concentrations) in src/constituent_mass_module.f90. Fields: name, soil(num_paths), plt(num_paths) in cfu/m^2.
- Format: one title line, then per record: a header line, a name line, then one line containing name, soil(:), plt(:) for all pathogens in constituents.cs order.

The same reader allocates cs_path_solsor(num_paths).

path_water.ini

Initial pathogen concentrations for channels and reservoirs (water column and benthic).

- Reader: src/path_cha_res_read.f90
- Type: `path_water_ini` (`type cs_water_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `water(num_paths)`, `benthic(num_paths)`. Record names are stored in `path_init_name`.
- Format: title line, header line, then per record: name line, then `num_paths` pairs of lines (water value, benthic value). On the second pass the reader reads a single line `name, water(:), benthic(:)` per record.

Related

- constituents.cs. Declares the pathogen list and sets `cs_db%num_paths`.
- pesticide module. Same per-object initial-condition pattern.
- metals module. Same per-object initial-condition pattern.

Purpose

The heavy metals module simulates user-listed metals across HRU soils and plants. The set of metals simulated is declared in constituents.cs, which populates `cs_db%num_metals`. Only initial soil and plant concentrations are loaded by a dedicated reader; no separate metal parameter database is read by `hmet_hru_aqu_read`.

Activation

`in_parmdb%hmetcom_db` is defined in src/input_file_module.f90 with default `metals.mtl`. No reader in `src/` opens this filename, so the parameter database is declared in `file.cio` but not loaded.

The initial-conditions file is referenced from `file.cio` via `in_init%hmet_soil` (default `hmet_hru.ini`). `hmet_hru_aqu_read` is called from `proc_read.f90`.

Files

hmet_hru.ini

Initial metal concentrations for HRU soils and plants.

- Reader: src/hmet_hru_aqu_read.f90
- Type: `hmet_soil_ini` (`type cs_soil_init_concentrations`) in src/constituent_mass_module.f90. Fields: `name`, `soil(num_metals)` (ppm), `plt(num_metals)` (ppm).
- Format: one title line, then per record: a header line, a name line, then `num_metals` pairs of lines (`metal_name, soil_value` then `metal_name, plt_value`) in `constituents.cs` order.

The same reader allocates `cs_hmet_solsor(num_metals)`.

metals.mtl

Declared as `in_parmdb%hmetcom_db` in `input_file_module.f90` (default name `metals.mtl`). No reader in the current source opens this file. The file is reserved but inert.

Related

- constituents.cs. Declares the metal list and sets `cs_db%num_metals`. The metals block in `constit_db_read` is read but its name-database cross-walk is commented out, so listed metals only contribute the count.
- pesticide module. Same per-object initial-condition pattern.
- pathogen module. Same per-object initial-condition pattern.

Purpose

gwflow is the cell-based 2D groundwater module that replaces the lumped aquifer (*.aqu) when active. It discretises the basin into a structured or unstructured mesh and solves a horizontal groundwater balance per cell, with optional couplings to channels, HRUs/LSUs, reservoirs, wetlands, floodplains, tiles, canals, solute transport, and heat transport.

Input is split across several small *.gw files, one concern per file, written by swatplus-editor from the project database and read by the gwflow_*.f90 routines. The one exception is gwflow.con, the channel-cell connect file, which keeps the .con extension (it is a standard SWAT+ connect file listed in file.cio). The older monolithic gwflow.input master file is no longer written or required.

Activation

Set gwflow = 1 in codes.bsn. The model then expects chancell.gw to exist. basin_read_objs opens it to count channel-cell intersections, sets sp_ob%gwflow to that count, switches aqu_con to null, and points gwflow_con at gwflow.con. If chancell.gw is missing, bsn_cc%gwflow is reset to 0 and the lumped aquifer is used.

Format conventions

- **Line 1 of every file** is a description and timestamp line written by the editor; the reader skips it.
- Most tabular files have **2 preamble lines**: the description line, then a column-header row. Data starts on line 3. (gwflow.con follows the SWAT+ .con convention: a title line, then a header row.)
- Numeric tokens are whitespace separated; the reader uses list-directed input, so exact column widths do not matter. The editor right-justifies for readability.
- Optional override columns (strK, strthick, bc_type, tile_*, init_temp) carry the literal null when unset.
- Cell IDs run 1..ncell, one row per active cell, ascending; row position equals cell_id. Every file that references a cell uses that same ID space, so no cell_id may exceed ncell. For unstructured grids row/col are 0.

File overview

The writer's write() driver decides which files to emit.

File	Written when	Purpose
codes.gw	always	Feature flags and run controls (wide table)
zones.gw	always	Aquifer/streambed zone parameters
cells.gw	always	Per-cell static properties and initial state
cellcon.gw	always	Cell-to-cell connection (neighbour) lists
outputs.gw	always	Output times, observation cells, debug cell
chancell.gw	channel cells exist	Channel-to-cell exchange links (incl. depth-zone and observation flags)
gwflow.con	channel cells exist	Channel-cell spatial connections (.con style; keeps .con)
hrucell.gw	conn_type = 1 (HRU)	HRU-to-cell area mapping
lsucell.gw	conn_type = 2 (LSU)	LSU-to-cell area mapping
rescell.gw	reservoir_exchange	Reservoir-cell links
floodplain.gw	floodplain_exchange	Floodplain-cell links
wetland.gw	wetland_exchange	Wetland bed thickness
tile.gw	tile_drainage	Tile drainage parameters and per-cell flag
solute.gw	solute_transport	Solute transport configuration
cell_sol.gw	solute_transport	Per-cell initial solute concentrations
pumpex.gw	pumpex, if pumps defined	External pumping schedule
hru_pump.gw	if HRUs selected	HRUs to report pumping output for

File	Written when	Purpose

The optional flat files (`tvheads.gw`, `ponds.gw` + `pond_cell.gw` + `pond_div.gw`, `phreato.gw` + `phreato_cell.gw`, `sw_group.gw`, `chan_depth.gw`) are each written when their database table is populated. See their sections below.

The files use the `*.gw` extension (content-first SWAT+ naming, like `plants.plt`); `gwflow.con` is the only one that keeps `.con`. When a flag is off, the writer deletes any stale file.

The standalone channel-cell observation and depth-zone files from the earlier format are gone: they are now trailing columns (`obs`, `dep_zone`) of `chancell.gw`. The daily channel-depth series remains a separate file, `chan_depth.gw`.

`codes.gw`: feature flags and run controls

Wide table, same shape as `codes.bsn`: the description line, then a header row of 23 key names, then one value row, right-justified. The reader matches keys by name, so column order does not matter.

Keys in written order:

```
grid_type ncell cell_size n_rows n_cols bc_type conn_type gw_soil satx pumpex
tile res wet fp canal solute heat time_step write_day write_mon write_yr write_aa river_thresh
```

Key	Meaning
`grid_type`	`structured` or `unstructured`
`ncell`	Number of active cells
`cell_size`	Cell size, m (0 = unstructured)
`n_rows`	Grid rows (0 = unstructured)
`n_cols`	Grid cols (0 = unstructured)
`bc_type`	Default boundary condition (1 = const head, 2 = no flow)
`conn_type`	Recharge connection (1 = HRU, 2 = LSU)
`gw_soil`	Groundwater-to-soil transfer (0/1)
`satx`	Saturation-excess flow (0/1)
`pumpex`	External pumping (0/1)
`tile`	Tile drainage (0/1)
`res`	Reservoir exchange (0/1)
`wet`	Wetland exchange (0/1)
`fp`	Floodplain exchange (0/1)
`canal`	Canal seepage (0/1)
`solute`	Solute transport (0/1)
`heat`	Heat transport (0/1)
`time_step`	Solver time step (days)
`write_day`	Daily output (0/1)
`write_mon`	Monthly output (0/1)
`write_yr`	Annual output (0/1)
`write_aa`	Average-annual output (0/1)
`river_thresh`	River-bed elevation threshold (m)

Descriptions are not stored in the file; the doc-of-record for the key list is `mergeWork/inputWork/gwflow_codes_reference.md`, kept in sync by the editor. Heat is plumbed but off by default (`heat 0`).

Output frequency is hybrid: the `write_day`/`write_mon`/`write_yr`/`write_aa` flags here set the gwflow output frequency, but `print.prt` overrides them when it carries `gwflow_wb`, `gwflow_obs`, or `gwflow_pump` rows. With no such rows in `print.prt`, the `codes.gw` flags win.

`zones.gw`: aquifer and streambed zones

2 preamble lines. One row per zone, `zone_id` ascending. 6 columns:

#	Column	Meaning
1	zone_id	Zone index
2	aquifer_K	Aquifer hydraulic conductivity (m/day)
3	aquifer_Sy	Specific yield
4	streambed_K	Streambed hydraulic conductivity (m/day)
5	streambed_thick	Streambed thickness (m)
6	thermal_K	Thermal conductivity (heat batch; 0 when off)

cells.gw: cell properties and initial state

2 preamble lines. One row per active cell, id running 1..ncell. 23 columns, leading with the SWAT+ object convention id / name / gis_id:

#	Column	Meaning
1	id	Cell index, 1..ncell, equal to row position
2	name	Cell name, cellNNNN (zero-padded, e.g. cell0001)
3	gis_id	Authoritative GIS cell id from gwflowcells.shp, equal to id (1..ncell)
4	status	Cell status code
5	elev	Ground-surface elevation (m)
6	thck	Aquifer thickness (m)
7	K_zone	Aquifer K zone id
8	Sy_zone	Specific-yield zone id
9	delay	Recharge delay (days)
10	exdp	ET extinction depth (m)
11	init	Initial groundwater head (m)
12	x	Cell-centre x coordinate
13	y	Cell-centre y coordinate
14	area	Cell area (m^2)
15	strK	Streambed K override (null if unset)
16	strthick	Streambed thickness override (null if unset)
17	bc_type	Per-cell boundary condition override (null if unset)
18	tile_depth	Tile depth override (null if unset)
19	tile_area	Tile area override (null if unset)
20	tile_K	Tile K override (null if unset)
21	row	Structured grid row (0 if unstructured)
22	col	Structured grid col (0 if unstructured)
23	init_temp	Initial groundwater temperature (null when heat off)

gis_id is the cell id carried straight from the QSWAT+ gwflowcells.shp layer (it equals id, since the shapefile numbers active cells 1..ncell). It is used for the gis_id output column and for cell and pump naming, replacing an earlier row/col computation. The reader consumes the columns by position, so column order is fixed.

cellcon.gw: cell-to-cell connections

2 preamble lines (description, then cell_id neighbor_tot neighbor_ids). One variable-width row per cell 1..ncell:

- cell_id
- neighbor_tot (neighbour count)
- then neighbor_tot neighbour cell_ids

This defines the solver stencil (which cells are adjacent).

outputs.gw: output control

2 preamble lines (description, then category value note). One row per item:

category	value
head_output_time	output time, YYYYDDD (one row per requested time)
observation_cell	cell_id of an observation well (one row each)
detail_debug_cell	a cell_id, or 0 for none

chancell.gw: channel-to-cell links

2 preamble lines (description, then column header). One row per channel-cell link. 7 columns:

#	Column	Meaning
1	cell_id	Cell exchanging with the channel (1..ncell)
2	elev_m	Channel-bed elevation (m)
3	channel	Channel id (matches chandeg.con)
4	riv_length_m	Channel length within the cell (m)
5	zone	Depth-zone index
6	dep_zone	Channel depth-zone index, links the cell to its series in chan_depth.gw
7	obs	1 to report gw-channel exchange output for this cell, else 0

A cell may appear on multiple rows when more than one channel crosses it. The dep_zone and obs columns fold in two files from the earlier format (the separate channel-depth-zone file and the channel-observation file).

gwflow.con: channel-cell spatial connections

SWAT+ .con-style file, keeping the .con extension. Line 1 is the title gwflow.con: channel-cell spatial connections; line 2 is the .con header:

```
NUMB NAME GISID AREA LAT LONG ELEV CELL WST CONST OVER RULE SRC_TOT OBTYPE_OUT1 OBTYPNO_OUT1 HTYPE_OUT1 FR
```

One row per channel cell. CELL is the gwflow cell_id, OBTYPNO_OUT1 is the channel connect index, with fixed sdc / tot / 1.00 in the type/total/fraction fields.

hrucell.gw: HRU-to-cell mapping

2 preamble lines (description, then column header). One row per HRU-cell overlap. 4 columns:

#	Column	Meaning
1	hru	HRU id
2	area_m2	HRU area (m^2)
3	cell_id	Cell overlapping the HRU
4	overlap_m2	Overlap area (m^2)

lsucell.gw is the LSU analogue, written when conn_type is 2.

wetland.gw: wetland bed thickness

2 preamble lines (description, then name bed_thick). One row per wetland: wetland name, bed thickness (m).

hru_pump.gw: HRU pump-observation list

Selector list of HRUs to report pumping output for. Existence-gated: the editor writes it when the source table has rows, and removes it otherwise; no codes.gw flag. Meta line, then a count, then one hru_id per line (ascending).

Other conditional files

Written only when the matching `codes.gw` flag is on. These keep a title/description line then a blank line before data (no column-header row):

- `lsucell.gw`: LSU-to-cell mapping, one row per overlap: `lsu_id  cell_id  area_m2`.
- `rescell.gw`: reservoir-cell links: bed thickness and bed K, count, then `cell_id  res_id  res_stage`. (Not present when no reservoir cells.)
- `floodplain.gw`: floodplain-cell links: count, then `cell_id  channel_id  K  area_m2`. (Not present when no floodplain cells.)
- `tile.gw`: tile drainage: a defaults row `tile_depth  tile_area  tile_k`, then a tile-group count, then a per-cell `cell_id` list.
- `solute.gw`: solute transport config: transport steps, dispersion coefficient, solute count `nsolute`, then one row per solute `name  sorption_coef  rate_const  init_conc_aquifer  init_conc_recharge`.

`cell_sol.gw`: per-cell initial solute concentrations

Written when solute transport is on. 2 preamble lines (description, then `cell_id  conc_1 … conc_nsolute`). One row per cell `1..ncell`, with one concentration column per solute in `solute.gw` order:

```
cell_sol.gw: ...
 cell_id        conc_1        conc_2   ...   conc_10
       1      12.50000       0.00000   ...   0.00000
       2       7.70000       0.00000   ...   0.00000
```

New flat optional files

These optional files were harmonized to the flat convention: a description line, a header row, then rows to end-of-file with no embedded count lines. Each is existence-gated, written from its database table when populated. The formats below match the editor's on-disk output. The deferred items `minerals.gw` (salt-mineral parameters) and `transit.gw` (transit-time tracking) are reader-supported but not exported by the editor.

`tvheads.gw`: time-varying boundary heads

One row per boundary cell: `cell_id` then one head value per simulation year (positional, `1..Nbyr`; the reader takes `Nbyr` from the run period, so the file has no count line). A `0` means no specified head that year.

```
tvheads.gw: ...
   cell_id      head_yr1      head_yr2      head_yr3      head_yr4      head_yr5      head_yr6
         1       0.00000      12.50000       0.00000       9.00000       0.00000       0.00000
         5       0.00000       0.00000       7.00000       0.00000       0.00000       4.40000
```

`ponds.gw` + `pond_cell.gw`: recharge ponds

`ponds.gw` is one row per pond: 11 fixed columns then one `unl_conc` per solute (`unl_conc_1 … unl_conc_Nsolute`, unsaturated-zone concentration of each solute):

```
ponds.gw: ...
 pond_id  area  chan  canal  unl  bed_k  wsta  evap_co  start_yr  start_mo  start_day  unl_conc_1..unl_con
       1   25000.00   5   0   1   0.000100   3   0.70000   1981   4   1    2.50  0.30  ...
```

Column	Meaning
`pond_id`	Pond index
`area`	Pond area (m²)
`chan`	Target channel id
`canal`	Target canal id
`unl`	Unsaturated-zone flag (0/1)
`bed_k`	Pond-bed hydraulic conductivity (m/day)

Column	Meaning
`wsta`	Weather-station id
`evap_co`	Evaporation coefficient
`start_yr` / `start_mo` / `start_day`	Divert start date
`unl_conc_1..N`	Unsaturated-zone concentration per solute

`pond_cell.gw` lists pond-to-cell connections, one per row: `pond_id  cell_id  conn_area` (m^2). The number of connection rows defines the per-pond cell count; there is no count line.

`pond_div.gw`: pond daily diversion series

Daily diversion volume into each pond. Dense: one row per simulation day (0-filled on days with no diversion). The columns after the date are one `div_pondN` per pond, in `ponds.gw` pond-id order:

```
pond_div.gw: ...
    year   month     day     div_pond1     div_pond2
    1980       1       1       0.00000       0.00000
    1980       1       2       0.00000       0.00000
```

- `year` / `month` / `day`: calendar date of the row. The reader keys on row order (positional), so every simulation day must be present, including leap days.
- `div_pond1 … div_pondN`: diversion volume into each pond (m^3) that day.
- Existence-gated: written when pond diversions are defined; if absent, diversion is 0.

`sw_group.gw`: channel-cell groups

Groups of channel cells for aggregated reach gw-channel exchange output. One variable-width row per group:

```
sw_group.gw: channel-cell groups
   group_id   ncell   cell_ids
   1   2   42   43
   2   2   44   45
```

- `group_id`, then `ncell` (cells in the group), then `ncell` cell ids (1..ncell space).

`phreato.gw` + `phreato_cell.gw`: phreatophyte ET

Split into a depth-ET curve and the cells it applies to.

`phreato.gw` is the depth-rate curve, one row per point:

```
phreato.gw: depth-ET curve
     depth_m     et_rate_m
   0.0   0.005
   1.0   0.003
   2.0   0.001
```

`phreato_cell.gw` lists the cells, one per row:

```
phreato_cell.gw: phreatophyte cells
   cell_id   area_m2
   22   25000000.0
   23   25000000.0
```

`chan_depth.gw`: daily channel depth per zone

Per-zone daily channel-depth series, one row per simulation day. Wide: `jday  yr` then one depth column per channel depth zone (N = number of `dep_zone` values in `chancell.gw`):

```
chan_depth.gw: daily channel depth per zone
   jday   yr   depth_z1   depth_z2
   1   1980   0.50   0.80
   2   1980   0.50   0.80
```

pumpex.gw format

External pumping schedule, enabled by `pumpex = 1` in `codes.gw`. 2 preamble lines (meta line, then a column header), then one row per pump-period. 7 columns:

```
gwflow.pumpex: external groundwater pumping
      name    cell_id        rate   yr_start   dy_start     yr_end     dy_end
  cell0700        700      150.00       1981          1       1981        180
  cell0700        700       80.00       1981        181       1981        365
  cell0912        912      250.00       1982         90       1984        270
```

#	Column	Meaning
1	`name`	Cell name (`cellNNNN`, matching `cells.gw` col 2); label only, the reader keys on `cell_id`
2	`cell_id`	Pumped cell (1..ncell)
3	`rate`	Pumping rate (m^3/day)
4	`yr_start`	Start year
5	`dy_start`	Start day-of-year (1..365)
6	`yr_end`	End year
7	`dy_end`	End day-of-year (1..365)

- One row per pump-period; rows for the same cell are consecutive, and consecutive same-cell rows form one pump.
- Dates are calendar (year + day-of-year). The reader converts each to a simulation day-count as `(year - sim_start_year) * 365 + day_of_year`, the same 365-day convention as `ponds.gw`.

The editor writes this file (`write_pumpex`) when `pumpex = 1` and pumps are defined in the project database, one row per period straight from the database. It is skipped when no pumps exist.

Example

A 2-zone structured grid (excerpts from a generated project; values are illustrative).

codes.gw (wide, abbreviated)

```
grid_type ncell cell_size n_rows n_cols ... solute heat time_step write_aa river_thresh
structured 1454   5000.00     52     53 ...      0    0      1.00        1         5.00
```

zones.gw

```
   zone_id     aquifer_K    aquifer_Sy   streambed_K streambed_thick
         1       0.50000       0.20000       0.00500         0.50000
         2       2.00000       0.20000       0.00500         0.50000
```

cells.gw (abbreviated)

```
 id  name      gis_id  status   elev   thck K_zone Sy_zone ...  row  col init_temp
  1  cell0001       1       2 395.00  34.99      1       1 ...   52    1      null
```

cellcon.gw

```
 cell_id  neighbor_tot  neighbor_ids
       1             3       2      20      21
```

chancell.gw

```
 cell_id   elev_m  channel  riv_length_m  zone  dep_zone  obs
      42   326.00       66       1853.55     1         1    1
```

Related

- codes.bsn: gwflow = 1 activates the module.
- object.cnt: sp_ob%gwflow is set from the channel-cell count and replaces sp_ob%aqu.
- aquifer.aqu, initial.aqu: not used when gwflow is active.

Purpose

The rtb-salt module simulates the eight salt ions (so4, ca, mg, na, k, cl, co3, hco3) across the soil profile, plant, channel, aquifer, urban, irrigation, fertilizer, atmospheric, and road-salt pathways. Each input file listed below feeds one piece of that state: initial concentrations, applied loads, or per-plant tolerance parameters.

Activation

Salt simulation is gated by the master constituent file constituents.cs referenced from file.cio (in_sim%cs_db, default constituents.cs). The number of salt ions in that file is held in cs_db%num_salts. If num_salts = 0, salt readers that test it (salt_roadsalt_read, salt_uptake_read, salt_urban_read) skip silently. The remaining readers test only for file existence and read whatever is present.

All readers are called from proc_read.f90 except salt_cha_read, which is called later from main.f90.

Files

salt_hru.ini

Initial salt ion concentrations and mineral fractions for HRU soils and plants.

- Reader: src/salt_hru_read.f90
- Type: salt_soil_ini (type cs_soil_init_concentrations) in src/constituent_mass_module.f90
- Format: 4 header lines, then for each entry three lines: name, then num_salts + 5 soil values, then num_salts + 5 plant values. First num_salts values are ion concentrations (ppm), next 5 are salt mineral fractions.

salt_aqu.ini

Initial salt ion concentrations and mineral fractions for aquifers.

- Reader: src/salt_aqu_read.f90
- Type: salt_aqu_ini (type salt_aqu_init_concentrations) in src/constituent_mass_module.f90. Fields: name, conc(num_salts) in g/m3, frac(5) mineral fractions.
- Format: 4 header lines, then one line per aquifer init record: name, num_salts concentrations, 5 mineral fractions.

salt_channel.ini

Initial salt ion concentrations for channels.

- Reader: src/salt_cha_read.f90
- Type: salt_cha_ini (type salt_cha_init_concentrations) in src/constituent_mass_module.f90. Fields: name, conc(num_salts) in g/m3.
- Format: 2 header lines, then one line per record: name, num_salts concentrations.

salt_irrigation

Salt ion concentrations for irrigation water sourced from outside the watershed.

- Reader: src/salt_irr_read.f90
- Type: salt_water_irr (type cs_irrigation_concentrations) in src/constituent_mass_module.f90. Fields: name, water(num_salts) in ppm.

- Format: 2 header lines, then one line per source: `name`, `num_salts` concentrations.

salt_plants

Plant salt-tolerance parameters and stress control switches.

- Reader: src/salt_plant_read.f90
- Targets in src/salt_data_module.f90:

Field	Type	Units	Description
salt_tds_ec	real	none	TDS to electrical conductivity conversion factor
salt_tol_sim	int	flag	0 = off, 1 = simulate salt effect on plant growth
salt_soil_type	int	code	1 = CaSO4 soils, 2 = NaCl soils
salt_effect	int	code	1 = applied after other stresses, 2 = combined with other stresses via min
salt_stress_a(plantparm)	real	none	a parameter in the salinity relative-yield equation
salt_stress_b(plantparm)	real	none	b parameter in the salinity relative-yield equation

Format: title line, header, `salt_tds_ec`, blank, `salt_tol_sim`, `salt_soil_type`, `salt_effect`, four skipped lines, header, then one line per plant in plant database order with `name, a, b`.

salt_road

Road salt loadings (kg/ha) per atmospheric deposition station.

- Reader: src/salt_roadsalt_read.f90
- Target: `rdapp_salt(:)%salt(:)` (`%road`, `%roadmo`, `%roadyr`, `%roadday`). Allocated for `0:atmodep_cont%num_sta` stations.
- Format: 4 commentary lines skipped, then per station and per salt ion either an annual value (`aa`), a monthly series (`mo`), or a yearly series (`yr`). For `yr`, the reader internally splits the annual mass to daily loadings on days with precipitation and sub-zero temperature.

salt_uptake

Specified daily salt uptake (kg/ha) per plant community per ion.

- Reader: src/salt_uptake_read.f90
- Targets in `src/salt_module.f90`: `salt_uptake_kg(plantparm, num_salts)`, `salt_uptake_on` (flag set to 1 when the file is present).
- Format: 3 header lines, then one line per plant: `name`, `num_salts` uptake values.

salt_urban

Salt ion concentration in suspended solids from urban impervious areas (mg salt / kg sediment).

- Reader: src/salt_urban_read.f90
- Target: `salt_urban_conc(urban, num_salts)` in `src/salt_module.f90`.
- Format: 2 header lines, then one line per urban land-use type: `urb_type`, `num_salts` concentrations. The urban name must match an entry in `urban.urb`.

salt_fertilizer.frt

Per-fertilizer salt ion loading (kg/ha).

- Reader: src/salt_fert_read.f90
- Type: `fert_salt(fertparm)` (`type fert_db_salt`) in `src/salt_module.f90`. Sets `fert_salt_flag = 1`.

#	Field	Units	Description
1	fertnm	-	fertilizer name
2	so4	kg/ha	sulfate loading
3	ca	kg/ha	calcium loading
4	mg	kg/ha	magnesium loading
5	na	kg/ha	sodium loading
6	k	kg/ha	potassium loading
7	cl	kg/ha	chloride loading
8	co3	kg/ha	carbonate loading
9	hco3	kg/ha	bicarbonate loading

Format: title, header, then one line per fertilizer in the fertilizer database order.

Related

- constituents.cs. Master constituent list. Sets `num_salts` and the ion names.
- fertilizer.frt. The base fertilizer database. `salt_fertilizer.frt` must have the same row count.
- urban.urb. Source of urban land-use names referenced by `salt_urban`.
- atmodep.cli. Defines the atmospheric deposition stations used by `salt_road`.

Purpose

The rtb-cs module simulates user-listed water-quality constituents (selenium SeO4 and SeO3, boron, and additional species defined in the constituent database). It runs in parallel with rtb-salt and shares the same master file, `constituents.cs`.

Activation

`constituents.cs` is the master file referenced from `file.cio` as `in_sim%cs_db` (default `constituents.cs`). It lists the pesticides, pathogens, metals, salt ions, and other constituents in the simulation, and populates `cs_db%num_pests`, `cs_db%num_paths`, `cs_db%num_metals`, `cs_db%num_salts`, and `cs_db%num_cs`. The cs and salt readers downstream allocate their arrays from these counts.

The readers `cs_roadsalt_read` does not exist for cs. The cs readers `cs_uptake_read`, `cs_urban_read`, and `cs_reactions_read` only proceed when `cs_db%num_cs > 0`; the remaining readers test only for file existence.

All cs readers are called from `proc_read.f90` except `cs_cha_read`, which is called later from `main.f90`.

Files

constituents.cs

Master list of constituents.

- Reader: src/constit_db_read.f90
- Type: `cs_db` (`type cs_db_type`) in src/constituent_mass_module.f90.
- Format: title, then five pairs of `count` followed by `count` names, in order: pesticides, pathogens, metals, salts, other constituents. The metals block is read but is unused downstream (the matching loop is commented out in the source). The salt names are matched against the database in salt_plants by plant-parameter index, not by name.

cs_hru.ini

Initial constituent concentrations for HRU soils and plants.

- Reader: src/cs_hru_read.f90

- Type: `cs_soil_ini` (`type cs_soil_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `soil(2*num_cs)`, `plt(2*num_cs)` in ppm.
- Format: 4 header lines, then for each entry three lines: name, soil values, plant values.

cs_aqu.ini

Initial constituent concentrations for aquifers.

- Reader: src/cs_aqu_read.f90
- Type: `cs_aqu_ini` (`type cs_aqu_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `aqu(2*num_cs)` (dissolved plus sorbed).
- Format: 3 header lines, then one line per entry: `name`, `2*num_cs` values.

cs_channel.ini

Initial constituent concentrations for channels.

- Reader: src/cs_cha_read.f90
- Type: `cs_cha_ini` (`type cs_cha_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `conc(num_cs)` in g/m3.
- Format: 2 header lines, then one line per record: `name`, `num_cs` concentrations.

The same reader also processes the optional `cs_streamobs` file, which lists channels for which daily concentration and load output is written to `cs_streamobs_output`.

cs_irrigation

Constituent concentrations for irrigation water sourced from outside the watershed.

- Reader: src/cs_irr_read.f90
- Type: `cs_water_irr` (`type cs_irrigation_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `water(num_cs)` in ppm.
- Format: 2 header lines, then one line per source: `name`, `num_cs` concentrations.

cs_plants_boron

Plant boron tolerance parameters.

- Reader: src/cs_plant_read.f90
- Targets in src/cs_data_module.f90:

Field	Type	Units	Description
bor_tol_sim	int	flag	0 = off, 1 = simulate boron effect on plant growth
bor_stress_a(plantparm)	real	none	a parameter in the boron relative-yield equation
bor_stress_b(plantparm)	real	none	b parameter in the boron relative-yield equation

Format: title, blank, `bor_tol_sim`, four header lines, then one line per plant in plant database order with `name, a, b`.

cs_uptake

Specified daily constituent uptake (kg/ha) per plant community per constituent.

- Reader: src/cs_uptake_read.f90
- Targets in src/cs_module.f90: `cs_uptake_kg(plantparm, num_cs)`, `cs_uptake_on` (flag set to 1 when the file is present).
- Format: 3 header lines, then one line per plant: `name`, `num_cs` uptake values.

cs_reactions

Selenium and boron chemical reaction parameters for soils and aquifers, organised by reaction group and shale geology.

- Reader: src/cs_reactions_read.f90
- Type: `cs_rct_soil(sp_ob%hru)` and `cs_rct_aqu(sp_ob%aqu)` (`type constituent_rct`) in `src/cs_data_module.f90`. Also fills the module-level arrays `rct(num_rct, num_groups)` and `rct_shale(num_geol_shale, 3)`.
- Format: title, header, `num_rct num_groups`, `num_rct` lines of group values, header, `num_geol_shale`, `num_geol_shale` lines of three shale parameters, header, one line per HRU (`hru_id, group, shale_fractions...`), header, one line per aquifer (`aqu_id, group, shale_fractions...`). The eight `rct` rows hold (in order): inorganic-N to Se ratio, soil oxygen, aquifer oxygen, kd_seo4, kd_seo3, kd_boron, kseo4 in soil, kseo4 in aquifer. See the source for the full assignment table.

cs_urban

Constituent concentration in suspended solids from urban impervious areas (mg cs / kg sediment).

- Reader: src/cs_urban_read.f90
- Target: `cs_urban_conc(urban, num_cs)` in `src/cs_module.f90`.
- Format: 2 header lines, then one line per urban land-use type: `urb_type`, `num_cs` concentrations. The urban name must match an entry in `urban.urb`.

fertilizer.frt_cs

Per-fertilizer constituent loading (kg/ha).

- Reader: src/cs_fert_read.f90
- Type: `fert_cs(fertparm)` (`type fert_db_cs`) in `src/cs_module.f90`. Sets `fert_cs_flag = 1`.

#	Field	Units	Description
1	fertnm	-	fertilizer name
2	seo4	kg/ha	selenate loading
3	seo3	kg/ha	selenite loading
4	boron	kg/ha	boron loading

Format: title, header, then one line per fertilizer in the fertilizer database order.

Related

- salt module. The salt readers share `constituents.cs` for ion counts and use the same per-object pattern.
- fertilizer.frt. Base fertilizer database. `fertilizer.frt_cs` must have the same row count.
- urban.urb. Source of urban land-use names for `cs_urban`.
- atmodep.cli. Atmospheric deposition stations used by the cs deposition reader `cli_read_atmodep_cs`.

Purpose

The water allocation module moves water between spatial objects (channels, reservoirs, aquifers, HRUs) and an auxiliary transport network (canals, pipes, towers, treatment plants, end-use sinks, out-of-basin sources and receivers). It is the mechanism SWAT+ uses for irrigation withdrawals, municipal and industrial supply, inter-basin transfer, and any rule-driven diversion. The driver type is `water_allocation` (`type water_allocation`, alias `wallo`) defined in `water_allocation_module.f90`. The entry-point file is `water_allocation.wro`. All other water-allocation files supply objects referenced from that file.

How it fits together

A single water allocation object (`wallo(iwro)`) owns one or more transfer objects (`wallo%trn(itrn)`). Each transfer object has:

- a transfer type (`trn_typ`): `outflo`, `ave_day`, `div_min`, `div_frac`, `dtbl_con`, `dtbl_lum`. The first five draw from the source list directly; `dtbl_lum` and `dtbl_con` defer the amount to a decision table.
- one or more source objects (`src`), each with a type from `cha`, `res`, `aqu`, `osrc`, `osrc_a`, `wtp`, `use`, `can`, plus a conveyance pointer (`pipe` or `pump`) and an optional withdrawal-limit decision table.
- exactly one receiving object (`rcv`), with a type from `hru`, `cha`, `res`, `aqu`, `wtp`, `use`, `stor` (tower), `can`, `orcv`.

The transport network is a set of independent files, each read by its own subroutine:

```
                    water_allocation.wro
                             |
            +----------------+----------------+
            |                |                |
         sources          transfer         receivers
            |                |                |
cha res aqu osrc       (rule type)      hru cha res aqu wtp
can wtp use osrc_a                       use stor can orcv
            |                                 |
            +--conveyance: pipe / pump --------+
                       |
                    canal
                (water_canal.wal)
                       |
                    pipe
                (water_pipe.wal)
                       |
                    tower
                (water_tower.wal)

wtp  : water_treat.wal     (treatment plant; lag-and-loss store)
use  : water_use.wal       (domestic/industrial/commercial sink)
osrc : out_src.wal         (out-of-basin source, recall-driven)
orcv : outside_rcv.wal     (out-of-basin receiver)
```

The water allocation routine runs per day. `wallo_control` advances the current transfer object, computes demand (`wallo_demand`), withdraws from each source in turn (`wallo_withdraw`), allows compensation between sources that share a `comp = y` flag, transports water through any pipe/pump conveyance (`wallo_transfer`), and delivers it to the receiver. Receiver-side post-processing (`wallo_treatment`, `wallo_use`, `wallo_canal`) handles lag, loss and outflow for `wtp`, `use` and `can` receivers.

Activation

The water allocation files are read at startup from `main.f90`, in this order:

```
call om_treat_read
call om_use_read
call om_osrc_read
call water_treatment_read
call water_use_read
!call water_osrc_read
call water_tower_read
call water_pipe_read
call water_canal_read
call water_allocation_read
```

The module is active whenever `water_allocation.wro` exists and is not `null`. The `wallo` array is then allocated from the count on line 2. If the file is absent the array is allocated as `wallo(0:0)` and the module is inert.

Per-step execution of `wallo_control` is triggered from `command.f90` (channel and reservoir commands) and from `sd_channel_control3.f90` (channel routing). The check `wallo(iwallo)%trn_cur <= wallo(iwallo)%trn_obs` advances the current transfer object until all are processed for the day.

Output (`water_allo_*.txt` / `.csv`) is produced when `db_mx%wallo_db > 0` and the corresponding `pco%water_allo` switch in `print.prt` is `y`.

Files

`water_allocation.wro`

Master file. Reader: src/water_allocation_read.f90. Pointed to by `in_watrts%transfer_wro` in `input_file_module.f90`.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`, the number of water allocation objects. Allocates `wallo(imax)`.
- For each allocation object:
  - Header line (skipped).
  - One record with the object header: `name`, `rule_typ`, `trn_obs`.
  - Header line (skipped).
  - `trn_obs` transfer-object records.

Allocation-object header columns (`wallo(iwro)`):

#	Field	Type	Description
1	name	char(25)	name of the water allocation object
2	rule_typ	char(25)	rule type to allocate water
3	trn_obs	int	number of transfer objects that follow

Each transfer-object record is variable-width. The reader reads it twice: first to learn `src_num`, then, after allocating `src(num_src)` and `osrc(num_src)`, to read the source and receiving fields. The full record is:

`id  trn_typ  trn_typ_name  amount  right  src_num  dtbl_src  src(1)..src(src_num)  rcv`

#	Field	Type	Description
1	id	int	transfer object index (also stored as `trn%num`)
2	trn_typ	char(10)	one of `outflo`, `ave_day`, `div_min`, `div_frac`, `dtbl_con`, `dtbl_lum`
3	trn_typ_name	char(40)	name of the dtbl row (for `dtbl_lum` / `dtbl_con`) or recall (for `outflo` with `osrc`); otherwise free text
4	amount	real	demand amount. m3/day for urban objects; mm for HRU; m3/s for `ave_day`, `div_min`, `div_frac`
5	right	char(2)	water right priority. `sr` (senior) or `jr` (junior)
6	src_num	int	number of source objects in this record
7	dtbl_src	char(25)	decision table name to allocate among sources (may be `null`)
8 to 7+8*src_num	src block	source-object tuple, repeated `src_num` times. See below.	
last three	rcv block	receiving-object tuple. See below.	

Source-object tuple (`transfer_source_objects`, 8 columns per source):

#	Field	Type	Description
1	typ	char(10)	source type: `cha`, `res`, `aqu`, `osrc`, `osrc_a`, `wtp`, `use`, `can`
2	num	int	sequential number of the source object in its own database
3	conv_typ	char(10)	conveyance type: `pipe`, `pump`, or none
4	conv_num	int	sequential number of the conveyance object
5	dtbl_lim	char(25)	decision-table name that sets the withdrawal limit (may be `null`)
6	wdraw_lim	real	actual withdrawal limit. Units depend on source type: reservoir uses principal-storage fraction; aquifer uses maximum depth (m); channel uses minimum flow (m3/s)
7	frac	real	fraction of total transfer supplied by this source
8	comp	char(1)	compensate from this source if other sources are past their threshold. `y` or `n`

Receiving-object tuple (`transfer_receiving_objects`, 3 columns):

#	Field	Type	Description
1	typ	char(10)	receiver type: `hru`, `cha`, `res`, `aqu`, `wtp`, `use`, `stor`, `can`, `orcv`
2	num	int	sequential number of the receiver in its own database
3	frac	int	for an HRU receiver, the soil layer that receives incoming tile flow. Declared `integer :: frac = 0.` in the module (initializer is a real literal).

Synthetic example (no example exists in `refdata/`):

```
water_allocation.wro: synthetic example
1
name                      rule_typ          trn_obs
muni_supply               demand            1
id  trn_typ      trn_typ_name      amount    right  src_num  dtbl_src  [src...]
1   ave_day      none              0.10      sr     1        null      cha 5  none 0  null  0.0   1.0   n
```

`TODO:` verify exact whitespace and `null` keyword handling in `dtbl_src`. The reader uses list-directed input, so any whitespace separates fields, but the `null` token convention is inferred from other SWAT+ inputs.

`water_canal.wal`

Reader: src/water_canal_read.f90. Filename is hard-coded in the reader; not pointed to from `file.cio`.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`, the number of canal objects.
- Line 3: header (skipped).
- `imax` canal records, each read twice (back-spaced) to learn `num_aqu` before allocating `aqu_loss(num_aqu)`.

Canal record (`water_canal_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	canal name
3	w_sta	char(25)	none	nearby weather station
4	init	char(25)	none	initial concentrations record
5	dtbl	char(25)	none	decision table that determines canal outflow
6	ddown_days	real	days	days to drawdown storage to zero
7	w	real	m	top width
8	d	real	m	depth

#	Field	Type	Units	Description
9	s	real	m/m	slope
10	ss	real	m/m	side slope (trapezoidal)
11	sat_con	real	varies	parameter used to compute percolation to groundwater
12	loss_fr	real	fraction	water loss during conveyance
13	bed_thick	real	m	bed-sediment thickness for Darcy seepage in `gwflow`; 0 if not used
14	div_id	int	none	recall diversion ID (gwflow). 0 if routed by `wallo`
15	day_beg	int	Julian	canal operation start day (gwflow external)
16	day_end	int	Julian	canal operation end day (gwflow external)
17	num_aqu	int	none	number of aquifers that receive seepage loss
18 .. 17+2*num_aqu	aqu_loss block	int, real pairs	each tuple is `aqu_num` (aquifer index) and `frac` (fraction of loss to that aquifer)	

`water_pipe.wal`

Reader: src/water_pipe_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` pipes:
  - Header line (skipped).
  - One record, read twice (back-spaced) to learn `num_aqu`.

Pipe record (`water_transfer_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	pipe name
3	stor_mx	real	m3	maximum storage
4	ddown_days	real	days	drawdown time
5	loss_fr	real	fraction	water loss during conveyance
6	num_aqu	int	none	number of aquifers that receive loss
7 .. 6+2*num_aqu	aqu_loss block	int, real	aquifer index and loss fraction	

The `init` field exists in the type definition (`character (len=25) :: init`) but is not read by `water_pipe_read.f90`. `TODO: verify` whether this is by design or an oversight.

`water_tower.wal`

Reader: src/water_tower_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` towers:
  - Header line (skipped).
  - One record.

Tower record (subset of `water_transfer_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	tower name
3	stor_mx	real	m3	maximum storage
4	ddown_days	real	days	drawdown time
5	loss_fr	real	fraction	storage loss

The reader does not allocate or read `aqu_loss` for towers even though the type carries the array. The tower is treated as a passive storage node.

`water_treat.wal`

Reader: src/water_treatment_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` plants:
  - One main record.
  - If `cs_db%num_pests > 0`: a header line then one record of `num_pests` pesticide concentrations into `wtp_cs_treat(iwtp)%pest`.
  - If `cs_db%num_paths > 0`: a header line then one record of `num_paths` pathogen concentrations into `wtp_cs_treat(iwtp)%path`.

Main record (`water_treatment_use_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	plant name
3	stor_mx	real	m3	maximum storage
4	lag_days	real	days	treatment time (outflow lag)
5	loss_fr	real	fraction	water loss during treatment
6	org_min	char(25)	none	name of `om_treat` record for sediment, carbon, nutrients. Cross-walked to `iorg_min`
7	pests	char(25)	none	name of pesticide-set record (`ipests`)
8	paths	char(25)	none	name of pathogen-set record (`ipaths`)
9	salts	char(25)	none	name of salt-ion record (`isalts`)
10	constit	char(25)	none	name of other-constituent record (`iconstit`)
11	descrip	char(80)	none	free-text description

`init` is declared in the type but commented out in the source. The reader does not read it. The cross-walk loop sets `iorg_min` only; `ipests`, `ipaths`, `isalts`, `iconstit` are left at 0. `TODO: verify` whether the missing cross-walks are intentional.

`water_use.wal`

Reader: src/water_use_read.f90. Filename hard-coded.

Same `water_treatment_use_data` type and same layout as `water_treat.wal`, including the conditional `pest` and `path` concentration blocks. Records are stored in `wuse` instead of `wtp`. Cross-walk is against `om_use_name` rather than `om_treat_name`.

out_src.wal

Reader: src/water_osrc_read.f90. The reader checks `inquire(file='outside_src.wal')` but then opens `'out_src.wal'`. See the Important section.

Layout (as written, assuming the open succeeds):

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` sources: one main record, then conditional pest and path blocks (header + values) as in `water_treat.wal`.

Main record (`outside_basin_source`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	out-of-basin source name
3	stor_mx	real	m3	maximum storage
4	lag_days	real	days	lag
5	loss_fr	real	fraction	loss fraction

outside_rcv.wal

Reader: src/water_orcv_read.f90. Filename hard-coded. The reader is not called from `main.f90`. See the Important section.

Layout (as written):

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` receivers: one record.

Main record (`outside_basin_receive`):

#	Field	Type	Description
1	id	int	record index
2	name	char(25)	out-of-basin receiver name
3	filename	char(25)	name of the output file written for this receiver

element.wro and water_rights.wro

Declared in `input_file_module.f90` (`in_watrts%element`, `in_watrts%water_rights`). No reader in `water_*_read.f90` or anywhere else under `src/` references either filename. `TODO: verify` whether these are legacy placeholders or are read by code outside the water-allocation subroutines.

Related

- `*.dtl` decision tables drive transfers via the `water_rights` action. `dtbl_lum` rows are looked up by `trn_typ_name` from `dtbl_lum` and supply irrigation amount; `dtbl_con` rows are looked up from `dtbl_flo` and supply flow-control amount, source availability and source/receiver allocation. `dtbl_src` and `dtbl_lim` are matched to decision-table names at run time.
- `object.cnt` has a `wro` column (position 21). It is currently noted in the source as "not used"; the actual count of water allocation objects is taken from line 2 of `water_allocation.wro`.
- `recall_db` is cross-walked for `osrc` sources (daily/monthly/yearly recall) and `exco_db` for `osrc_a` (annual constant from an `exco` record).
- [`om_treat.om`, `om_use.om`, `om_osrc.om`] supply the organic and mineral concentration sets named in `org_min` fields of `water_treat.wal`, `water_use.wal` and `out_src.wal`.